

100 CÂU HỎI VẤN ĐÁP THỰC TẾ & BẢO VỆ PHẢN BIỆN

ĐỀ TÀI: HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ THỜI TRANG & THIẾT BỊ CÔNG NGHỆ WEB-ECO

Tài liệu được xây dựng 100% dựa trên Codebase thực tế của dự án, đóng vai Hội đồng Giám khảo, Kỹ sư Kiến trúc Phần mềm Cấp cao (Senior Software Architect), Senior Fullstack Developer và Chuyên gia Viết Tài liệu Kỹ thuật.

Công nghệ Frontend:	React 18, Vite, TailwindCSS, Context API, Axios
Công nghệ Backend:	Node.js, Express.js, JWT, Bcrypt, Multer
Cơ sở dữ liệu:	MongoDB Atlas (NoSQL), Mongoose ODM
Trí tuệ nhân tạo (AI): minh)	Google Gemini 3.5 Flash Lite (So sánh sản phẩm thông
Cổng thanh toán:	VietQR NAPAS (Duyệt tay an toàn), Stripe Checkout quốc tế
Lưu trữ đám mây:	Cloudinary Cloud Media Hosting
Tổng số câu hỏi:	CHÍNH XÁC 100 CÂU (01 - 100) + 5 PHỤ LỤC TỔNG HỢP

Hà Nội, Năm 2026 — Lưu hành nội bộ phục vụ ôn thi tốt nghiệp

BỘ TÀI LIỆU ÔN TẬP VẤN ĐÁP BẢO VỆ ĐỒ ÁN TỐT NGHIỆP

HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ THIẾT BỊ CÔNG NGHỆ MINH TUẤN SHOP (WEB-ECO)

Biên soạn bởi: Hội đồng Chuyên môn Bảo vệ Đồ án + Senior Software Architect + Technical Lead Dành riêng cho sinh viên bảo vệ đồ án tốt nghiệp cử nhân / kỹ sư Công nghệ Thông tin

MỤC LỤC TỔNG THỂ

- **PHẦN 1: BỘ 100 CÂU HỎI VẤN ĐÁP & ĐÀO SÂU THEO CODEBASE THỰC TẾ**
 - Nhóm 1: Tổng quan đồ án & Bài toán thực tế (Câu 01 - 10)
 - Nhóm 2: Kiến trúc hệ thống (Câu 11 - 20)
 - Nhóm 3: Frontend Web App - React & Vite (Câu 21 - 30)
 - Nhóm 4: Backend & RESTful API - Node.js & Express (Câu 31 - 40)
 - Nhóm 5: Database & Data Modeling - MongoDB & Mongoose (Câu 41 - 50)
 - Nhóm 6: Logic nghiệp vụ Thương mại điện tử (Câu 51 - 60)
 - Nhóm 7: Authentication / Authorization & Security (Câu 61 - 70)
 - Nhóm 8: Workflow và Xử lý nghiệp vụ chi tiết (Câu 71 - 80)
 - Nhóm 9: Câu hỏi phản biện / Tình huống bấy của Hội đồng (Câu 81 - 90)
 - Nhóm 10: Hướng phát triển, Scale & Tối ưu hóa hệ thống (Câu 91 - 100)
 - **PHẦN 2: CÁC PHỤ LỤC BẮT BUỘC**
 - PHỤ LỤC A — Sơ đồ Kiến trúc Hệ thống Tổng thể
 - PHỤ LỤC B — Sơ đồ Thực thể Liên kết Cơ sở Dữ liệu (ERD)
 - PHỤ LỤC C — Các Workflow Nghiệp vụ Trọng yếu
 - PHỤ LỤC D — Bảng Tra cứu File Code và Chức năng Chính
 - PHỤ LỤC E — Chiến thuật Vàng và Kinh nghiệm Ứng khẩu Trước Hội đồng
-

PHẦN 1: BỘ 100 CÂU HỎI VẤN ĐÁP & ĐÀO SÂU

NHÓM 1: TỔNG QUAN ĐỒ ÁN & BÀI TOÁN THỰC TẾ (CÂU 01 — 10)

Câu 01 — Mục tiêu và Bài toán Thực tế của Hệ thống

CÂU HỎI

Em hãy trình bày mục tiêu xây dựng hệ thống Minh Tuấn Shop và bài toán thực tế mà đồ án của em giải quyết?

CÂU TRẢ LỜI

Kính thưa Hội đồng, hệ thống Minh Tuấn Shop (Web-ECó) được em xây dựng nhằm giải quyết bài toán kinh doanh bán lẻ thiết bị công nghệ (Điện thoại, Laptop, PC Gaming, Phụ kiện) với các đặc thù nghiệp vụ phức tạp: 1. **Quản lý đa cấu hình/biến thể:** Khác với bán quần áo thông thường, thiết bị công nghệ có các phiên bản phần cứng khác nhau (128GB, 256GB, Core i5, RTX 4060) với giá bán và số lượng tồn kho tách biệt. 2. **Hỗ trợ tư vấn thông minh với AI:** Tích hợp Google Gemini AI giúp khách hàng so sánh cấu hình chi tiết, phân tích ưu nhược điểm ngay trên website. 3. **Thanh toán VietQR tiện lợi:** Tạo mã QR động chuẩn NAPAS 24/7 với nội dung chuyển khoản riêng biệt cho từng đơn hàng (MTxxxxxx <phone>), sau khi khách chuyển tiền sẽ vào trạng thái "Chờ duyệt" như hình thức COD để nhân viên kiểm tra và duyệt đơn thủ công. 4. **Hỗ trợ trực tuyến và phân tích hành vi:** Tích hợp Live Chat real-time với CSKH và module Smart Customer Insights giúp Admin theo dõi xu hướng tìm kiếm và lượt xem sản phẩm theo thời gian thực.

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: frontend/src/pages/Home.jsx , Collection.jsx
- Backend: backend/server.js
- Database: Toàn bộ 9 collections MongoDB
- Module: Tổng quan đồ án

FILE CODE LIÊN QUAN

- backend/server.js (Khởi tạo toàn bộ router và cấu hình)
- frontend/src/App.jsx (Điều hướng các màn hình khách hàng)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu một chuỗi cửa hàng lớn muốn áp dụng phần mềm của em cho 20 chi nhánh thì hệ thống hiện tại có đáp ứng được việc quản lý tồn kho theo từng kho hàng chi nhánh không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, trong phạm vi codebase đồ án hiện tại, mô hình tồn kho đang được quản lý tập trung (Centralized Inventory) trong trường stockQuantities của từng sản phẩm. Nếu áp dụng cho chuỗi 20 chi nhánh, hệ thống cần nâng cấp thêm bảng Branches (Chi nhánh) và chuyển cấu trúc tồn kho thành { branchId, size, quantity } để theo dõi xuất-nhập-tồn riêng theo vị trí địa lý của từng kho hàng.

Câu 02 — Lựa chọn Công nghệ (Tech Stack)

CÂU HỎI

Tại sao em lại lựa chọn Tech Stack MERN (MongoDB, Express.js, React, Node.js) và Vite thay vì các công nghệ khác như PHP/Laravel hay Java Spring Boot?

CÂU TRẢ LỜI

Em lựa chọn MERN Stack kết hợp Vite vì các lý do kỹ thuật sau: 1. **Đồng nhất ngôn ngữ (Full-stack JavaScript/ES Modules)**: Toàn bộ frontend và backend đều sử dụng JavaScript hiện đại (`"type": "module"`), giúp tái sử dụng cấu trúc dữ liệu, giảm thiểu ngữ cảnh chuyển đổi (context switching) và tăng tốc độ phát triển. 2. **Phát triển Frontend tốc độ cao với Vite**: Thay vì dùng Create React App cũ kỹ sử dụng Webpack chậm chạp, Vite sử dụng native ES Modules và esbuild viết bằng Go, mang lại thời gian khởi động server dev dưới 300ms và Hot Module Replacement (HMR) tức thì. 3. **Mô hình hướng tài liệu linh hoạt (NoSQL MongoDB)**: Dữ liệu sản phẩm công nghệ có cấu hình thay đổi theo từng loại (Laptop có CPU/RAM, Màn hình có độ phân giải/tần số quét). Cấu trúc Schema linh hoạt của Mongoose giúp em dễ dàng lưu trữ mảng biến thể `sizes` và object `stockQuantities`. 4. **Node.js bất đồng bộ không chặn (Non-blocking I/O)**: Phù hợp hoàn hảo cho các tác vụ I/O cao như polling tin nhắn chat mỗi 3 giây và polling trạng thái đơn hàng.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `frontend/package.json` , `vite.config.js`
- Backend: `backend/package.json` , `server.js`
- Module: Kiến trúc hạ tầng

FILE CODE LIÊN QUAN

- File: `backend/package.json`
- File: `frontend/vite.config.js`

```
// Trích từ backend/package.json
{
  "name": "backend",
  "version": "1.0.0",
  "main": "server.js",
  "type": "module",
  "dependencies": {
    "bcrypt": "^6.0.0",
    "cloudinary": "^2.6.1",
    "cors": "^2.8.5",
    "dotenv": "^16.5.0",
    "express": "^5.1.0",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.16.0",
    "multer": "^2.0.1",
    "nodemailer": "^7.0.5"
  }
}
```

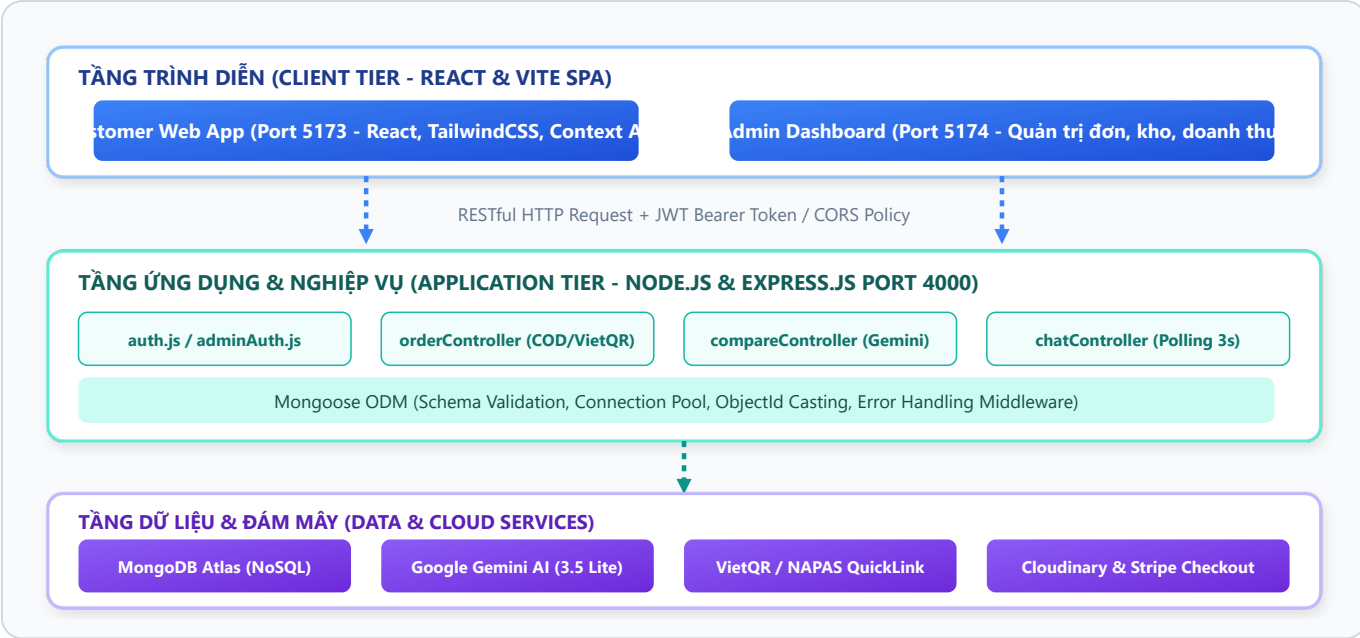
CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Express phiên bản em dùng trong `package.json` là `^5.1.0`. Express 5 có điểm gì mới nổi bật so với Express 4 mà em đã áp dụng trong code?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, Express 5 chính thức hỗ trợ tự động bắt lỗi từ các middleware và controller trả về Promise (async/await). Trong Express 4, nếu hàm async quăng lỗi mà không bọc trong try/catch hoặc không gọi next(err) thì process có thể bị unhandled rejection dẫn tới treo hoặc sập server. Express 5 tự động chuyển lỗi reject từ Promise sang error handler middleware.

SƠ ĐỒ 1: KIẾN TRÚC TỔNG THỂ HỆ THỐNG 3 TẦNG (CLIENT - SERVER - CLOUD SERVICES)



Hình 1: Kiến trúc tổng thể 3 tầng tích hợp Trí tuệ nhân tạo và Cổng thanh toán quốc gia

Câu 03 — Điểm Độc đáo & Khác biệt của Ứng dụng

CÂU HỎI

Website thương mại điện tử của em có những tính năng công nghệ nào nổi bật hơn một website bán hàng WordPress hoặc đề tài thông thường?

CÂU TRẢ LỜI

Đề án của em sở hữu 4 phân hệ công nghệ nổi bật được tích hợp sâu vào mã nguồn: 1. **Tư vấn & So sánh thiết bị thông minh bằng Google Gemini AI:** Hệ thống có khả năng so sánh 2 hoặc nhiều sản phẩm từ database, hoặc tự động tra cứu thông tin sản phẩm ngoài thị trường (`compareWithExternal`) rồi đưa ra nhận xét chuyên sâu bằng tiếng Việt. 2. **Thanh toán VietQR cá nhân hóa nội dung:** Tự động sinh mã VietQR chuẩn hình ảnh từ ngân hàng MBBank kèm cú pháp chuyển khoản chứa mã đơn hàng duy nhất `MTxxxxxx` và số điện thoại người nhận. Khách bấm nút "Xác nhận đã chuyển khoản", đơn chuyển sang trạng thái "Chờ duyệt" như COD để Admin kiểm tra và duyệt bằng tay. 3. **Kênh Chăm sóc Khách hàng trực tuyến 2 chiều:** Tích hợp widget chat nổi cho khách hàng và trang dashboard phân luồng chat riêng cho Admin, hỗ trợ gửi email follow-up trực tiếp qua Nodemailer. 4. **Smart Customer Insights:** Module theo dõi dòng hành vi khách hàng (`customerActivityModel.js`), thống kê sản phẩm được xem nhiều nhất và từ khóa tìm kiếm theo thời gian thực.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Controller: `compareController.js` , `paymentController.js` , `analyticsController.js`
- UI: `Compare.jsx` , `PaymentGateway.jsx` , `CustomerInsights.jsx`

FILE CODE LIÊN QUAN

- `backend/controllers/compareController.js`
- `backend/controllers/paymentController.js`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Trong phần so sánh AI, nếu khách hàng cố tình nhập tên một sản phẩm không có thật hoặc dùng prompt phá hoại (Prompt Injection) thì hệ thống của em xử lý như thế nào?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, trong `compareController.js`, em thiết lập tham số `systemInstruction` độc lập định vị AI đóng vai trò chuyên gia tư vấn mua sắm công nghệ của Minh Tuấn Shop, kèm theo ràng buộc định dạng đầu ra (Output JSON validation qua `responseMimeType: "application/json"` cho tra cứu thông số). Nếu tên sản phẩm không có thật, AI sẽ phản hồi dựa trên kiến thức ước lượng hoặc báo không tìm thấy thông tin phù hợp thay vì thực thi các câu lệnh lạ.

Câu 04 — Phân loại Đối tượng & Vai trò Người dùng

CÂU HỎI

Hệ thống của em hỗ trợ những nhóm đối tượng người dùng nào và phạm vi quyền hạn của từng nhóm ra sao?

CÂU TRẢ LỜI

Hệ thống phân tách thành 3 nhóm người dùng rõ rệt: 1. **Khách vắng lai (Guest)**: - Duyệt catalog, tìm kiếm, lọc theo giá, hãng, danh mục. - Sử dụng tính năng so sánh AI. - Thêm sản phẩm vào giỏ hàng và danh sách yêu thích (lưu cục bộ trong `localStorage`). - Có thể nhập form đặt hàng nhanh. 2. **Khách hàng thành viên (Registered User / Smember)**: - Sở hữu tài khoản xác thực qua JWT token. - Tự động điền (Auto-fill) thông tin giao hàng khi đặt hàng. - Quản lý giỏ hàng đồng bộ trên cơ sở dữ liệu MongoDB (`userModel.cartData`). - Xem lịch sử đơn hàng cá nhân (`/orders`), hủy đơn hàng khi đang ở trạng thái "Đã đặt hàng". - Đánh giá sản phẩm kèm hình ảnh thực tế cho các đơn hàng "Đã giao hàng". - Nhắn tin trực tuyến với tư vấn viên qua `ChatWidget`. 3. **Quản trị viên (Admin)**: - Có cổng đăng nhập độc lập tại giao diện Admin Dashboard. - Toàn quyền CRUD sản phẩm, upload ảnh Cloudinary, cấu hình tồn kho biến thể. - Quản lý trạng thái vòng đời đơn hàng, hủy đơn kèm lý do, xuất báo cáo Excel (`.xlsx`). - Quản lý danh sách thành viên, phân quyền. - Trả lời Live Chat của khách hàng, soạn email gửi khách hàng. - Xem báo cáo doanh thu và hành vi khách hàng thời gian thực.

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Middleware: `auth.js`, `adminAuth.js`
- Model: `userModel.js`
- Module: Phân quyền Authorization

FILE CODE LIÊN QUAN

- `backend/middleware/auth.js`
- `backend/middleware/adminAuth.js`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Làm thế nào backend phân biệt được một request gửi tới là của User bình thường hay của Admin khi cả hai đều gửi header có `token` ?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, hệ thống sử dụng 2 middleware riêng biệt: - Middleware `auth.js` giải mã JWT lấy `id` người dùng: `const token_decode = jwt.verify(token, process.env.JWT_SECRET);` và tìm user trong database `userModel.findById(token_decode.id)`. - Middleware `adminAuth.js` giải mã và kiểm tra chuỗi chữ ký đặc biệt của Admin: `javascript if (token_decode !== process.env.ADMIN_EMAIL + process.env.ADMIN_PASSWORD) { return res.json({ success: false, message: "Bạn không có quyền truy cập..." }); }` Nếu User thường cố tình gửi token của mình vào các route của Admin (như `/api/product/add`), middleware `adminAuth` sẽ từ chối truy cập ngay lập tức.

Câu 05 — Cấu trúc Dự án & Phân chia Module

CÂU HỎI

Em hãy giải thích cấu trúc tổ chức mã nguồn của dự án và cách phân chia các thư mục chính?

CÂU TRẢ LỜI

Dự án được em tổ chức theo mô hình phân tách độc lập gồm 3 thư mục gốc: 1. `backend/` : - `config/` : Cấu hình kết nối MongoDB Atlas (`mongodb.js`) và Cloudinary SDK (`cloudinary.js`). - `models/` : Chứa 9 Mongoose schemas đại diện cho các thực thể. - `controllers/` : Chứa toàn bộ business logic xử lý nghiệp vụ, tách biệt khỏi router. - `routes/` : Định nghĩa các API endpoints và gắn middleware tương ứng. - `middleware/` : Xác thực token User (`auth.js`), token Admin (`adminAuth.js`), cấu hình upload file `multer.js`. 2. `frontend/` : Ứng dụng web React dành cho khách hàng: - `src/pages/` : 16 màn hình người dùng. - `src/components/` : Các component tái sử dụng (Navbar, Footer, CartTotal, ChatWidget...). - `src/context/ShopContext.jsx` : Nơi quản lý trạng thái toàn cục (State store). 3. `admin/` : Ứng dụng web React dành riêng cho Ban quản trị: - Giao diện quản lý độc lập với Sidebar, Dashboard KPI, Orders, Products, Users, Statistics, Customer Support, Customer Insights.

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Thư mục: `backend/` , `frontend/` , `admin/`

FILE CODE LIÊN QUAN

- `backend/server.js`
- `frontend/src/App.jsx`
- `admin/src/App.jsx`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao em lại tách `admin` và `frontend` thành 2 dự án React riêng biệt thay vì gộp chung vào một ứng dụng React và phân quyền theo route?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, việc tách biệt mang lại 3 lợi ích lớn: 1. **Bảo mật**: Bundle mã nguồn của khách hàng hoàn toàn không chứa bất kỳ logic, component, hay API endpoint quản trị nào của Admin, ngăn chặn kẻ xấu phân tích mã client để tìm lỗ hổng quản trị. 2. **Hiệu năng tải trang (Bundle Size)**: Khách hàng không phải tải các thư viện nặng của Admin như `xlsx` xuất Excel hay các biểu đồ phân tích chuyên sâu. 3. **Độc lập triển khai (Independent Deployment)**: Có thể host trang Admin trên một subdomain riêng (ví dụ: `admin.minhtuanshop.vn`) có cấu hình tường lửa IP bảo vệ mà không ảnh hưởng tới website bán hàng công khai.

Câu 06 — Cấu hình Biến Môi trường & Triển khai

CÂU HỎI

Các biến môi trường trong file `.env` của backend đóng vai trò gì và em bảo vệ các thông tin bí mật này như thế nào?

CÂU TRẢ LỜI

File `backend/.env` đóng vai trò là trung tâm cấu hình bảo mật của hệ thống: - `PORT` : Cổng lắng nghe của Express Server (mặc định 4000). - `MONGODB_URI` : Chuỗi kết nối bảo mật đến cụm MongoDB Atlas có chứa username/password. - `JWT_SECRET` : Khóa bí mật dùng để ký và xác thực tính toàn vẹn của JSON Web Token. - `CLOUDINARY_*` : Chứa API Key, Secret và Cloud Name để giao tiếp với dịch vụ Cloudinary. - `ADMIN_EMAIL` & `ADMIN_PASSWORD` : Thông tin đăng nhập dành riêng cho tài khoản Quản trị viên tối cao. - `GEMINI_API_KEY` : API Key kết nối với Google Generative Language API phục vụ tính năng so sánh sản phẩm thông minh. - `BANK_*` : Cấu hình tài khoản ngân hàng thụ hưởng MBBank phục vụ tạo mã VietQR.

Để bảo vệ các thông tin này: 1. File `.env` được khai báo trong `.gitignore` để không bao giờ bị đẩy lên GitHub/Git repository công khai. 2. Cung cấp file mẫu `example-env` không chứa thông tin nhạy cảm để các lập trình viên khác biết cấu trúc khi setup.

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- File: `backend/.env` , `backend/.gitignore` , `backend/example-env`

FILE CODE LIÊN QUAN

- `backend/.env` (Dòng 1 - 33)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu vô tình làm lộ file `.env` lên GitHub, em sẽ thực hiện những bước xử lý sự cố nào ngay lập tức?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, em sẽ thực hiện quy trình ứng phó sự cố bảo mật gồm 3 bước: 1. **Thu hồi và Đổi mới toàn bộ Secret Keys (Key Rotation)**: Truy cập ngay MongoDB Atlas đổi mật khẩu database user, đổi JWT_SECRET, vào Google AI Studio xóa GEMINI_API_KEY cũ và tạo key mới, tái tạo API Secret trên Cloudinary console. 2. **Xóa lịch sử Git**: Dùng công cụ `git filter-branch` hoặc BFG Repo-Cleaner để xóa triệt để commit chứa file `.env` khỏi lịch sử Git, sau đó force push. 3. **Kiểm tra Logs**: Rà soát nhật ký kết nối MongoDB Atlas và Cloudinary để phát hiện xem có địa chỉ IP lạ nào truy cập dữ liệu trái phép trong khoảng thời gian lộ key hay không.

Câu 07 — Tích hợp Dịch vụ Bên thứ ba (Third-party Services)

CÂU HỎI

Hệ thống của em tích hợp những dịch vụ bên thứ ba nào? Vai trò của từng dịch vụ là gì?

CÂU TRẢ LỜI

Hệ thống tích hợp 4 dịch vụ SaaS/PaaS chính: 1. **Cloudinary**: Dịch vụ lưu trữ và phân phối hình ảnh đám mây. Giải quyết bài toán lưu ảnh sản phẩm và ảnh đánh giá của khách hàng mà không làm đầy ổ đĩa server Node.js. 2. **Google Gemini**

Generative AI (Model `gemini-3.5-flash-lite`): Cung cấp khả năng phân tích ngôn ngữ tự nhiên, so sánh thông số kỹ thuật điện thoại, laptop và đưa ra lời khuyên mua sắm khách quan bằng tiếng Việt. 3. **Dịch vụ VietQR (NAPAS 24/7 qua cổng VietQR API):** Sinh mã QR ngân hàng động theo chuẩn EMVCo chứa sẵn số tài khoản, mã ngân hàng, số tiền và nội dung chuyển khoản đặc trưng của đơn hàng. 4. **Nodemailer (kết nối Gmail SMTP):** Cho phép Admin gửi email phản hồi, báo giá chi tiết trực tiếp đến hòm thư của khách hàng ngay trong màn hình Customer Support.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Cloudinary: `backend/config/cloudinary.js` , `controllers/productController.js`
- Gemini AI: `backend/controllers/compareController.js`
- VietQR: `backend/controllers/paymentController.js`
- Nodemailer: `backend/controllers/chatController.js`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao em lại sử dụng API của VietQR để sinh link ảnh trực tiếp (`https://img.vietqr.io/image/...`) thay vì tự tạo thư viện sinh mã QR offline ngay trong Node.js?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, chuẩn VietQR NAPAS 24/7 đòi hỏi mã QR phải tuân theo cấu trúc chuẩn quốc tế EMVCo (chứa Payload Format Indicator, Point of Initiation Method, Merchant Account Information chứa mã BIN 970422 của ngân hàng MB, Transaction Currency 704 cho VND, và mã CRC checksum). Sử dụng API VietQR chuẩn hóa đảm bảo mã QR sinh ra tương thích 100% với tất cả ứng dụng Mobile Banking của hơn 40 ngân hàng tại Việt Nam, tránh việc tự sinh chuỗi thủ công dẫn tới sai lệch checksum khiến app ngân hàng báo "Mã QR không hợp lệ".

Câu 08 — Chuẩn Mực Thiết Kế Giao Diện (UI/UX)

CÂU HỎI

Giao diện người dùng của Minh Tuấn Shop được thiết kế theo phong cách nào và có những điểm tối ưu UX nào đáng chú ý?

CÂU TRẢ LỜI

Giao diện được em thiết kế lấy cảm hứng từ các hệ thống bán lẻ công nghệ hàng đầu Việt Nam (CellphoneS, Thế Giới Di Động): 1. **Tông màu nhận diện thương hiệu công nghệ:** Sử dụng sắc đỏ đặc trưng (`#d70018`), phối cùng nền xám trung tính (`#f4f6f8`) và các thẻ card trắng bo tròn lớn (`rounded-2x1` , `rounded-3x1`), tạo cảm giác hiện đại và kích thích mua sắm. 2. **Thanh tiến trình 3 bước trực quan tại Giỏ hàng:** Hiển thị rõ ràng: 1. GIỎ HÀNG -> 2. ĐẶT HÀNG -> 3. HOÀN TẤT, giúp người mua biết chính xác mình đang ở bước nào trong quy trình thanh toán. 3. **Phản hồi tương tác tức thì:** Tất cả thao tác click thêm giỏ, yêu thích, sao chép tài khoản, áp dụng voucher đều có Toast Notification màu sắc tương ứng kèm hiệu ứng rung/đổi màu nút bấm. 4. **Trải nghiệm Mobile First:** Có Drawer lọc sản phẩm riêng trên điện thoại, các nút bấm to bản dễ thao tác bằng ngón tay cái, hỗ trợ cuộn ngang (horizontal scroll) cho các danh mục con.

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- CSS: `frontend/src/index.css` , `frontend/tailwind.config.js`

- Pages: `Home.jsx` , `Cart.jsx` , `PlaceOrder.jsx` , `PaymentGateway.jsx`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Làm thế nào em đảm bảo website hiển thị tốt trên các thiết bị màn hình từ điện thoại 375px đến màn hình 2K/4K?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, em tận dụng hệ thống Breakpoints linh hoạt của Tailwind CSS (`sm: 640px` , `md: 768px` , `lg: 1024px` , `xl: 1280px`). Ví dụ lưới sản phẩm trên `Collection.jsx` được thiết lập `grid-cols-2 sm:grid-cols-3 lg:grid-cols-4` , container bọc ngoài giới hạn `max-w-7xl mx-auto px-4 sm:px-6 lg:px-8` để trên màn hình siêu rộng nội dung không bị bè ngang quá mức gây khó chịu cho mắt người đọc.

Câu 09 — Xử Lý Media & Tối Ưu Hình Ảnh

CÂU HỎI

Hình ảnh trong đồ án được upload và lưu trữ như thế nào từ lúc người dùng chọn file đến khi hiển thị trên web?

CÂU TRẢ LỜI

Quy trình xử lý ảnh diễn ra qua 4 bước: 1. **Client:** Người dùng chọn file ảnh từ máy tính (tại form thêm sản phẩm hoặc form review). Dùng `URL.createObjectURL(file)` để tạo preview tức thì trên giao diện trước khi submit. 2. **Truyền tải:** Gửi file qua giao thức HTTP dạng `multipart/form-data` lên Express. 3. **Multer Middleware:** Đón file, kiểm tra mimetype (`file.mimetype.startsWith('image/')`), giới hạn dung lượng (5MB), và ghi tạm vào thư mục `uploads/` trên server. 4. **Cloudinary Upload & Xóa file tạm:** - Backend gọi `cloudinary.uploader.upload(file.path, { resource_type: 'image' })` . - Nhận về `secure_url` (giao thức HTTPS). - Middleware `cleanupFiles` gọi `fs.unlinkSync(file.path)` để xóa sạch file tạm trên đĩa cứng server Node.js nhằm tránh đầy dung lượng bộ nhớ máy chủ. - Lưu chuỗi URL ảnh từ Cloudinary vào mảng `image` của Document trong MongoDB.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Middleware: `backend/middleware/multer.js` , `backend/routes/reviewRoute.js`
- Controller: `backend/controllers/productController.js` , `reviewController.js`

FILE CODE LIÊN QUAN

- `backend/controllers/productController.js` (Dòng 10 - 22)
- `backend/routes/reviewRoute.js` (Dòng 46 - 63)

```
// Trích từ backend/controllers/productController.js
const image1 = req.files.image1 && req.files.image1[0];
const image2 = req.files.image2 && req.files.image2[0];
const image3 = req.files.image3 && req.files.image3[0];
const image4 = req.files.image4 && req.files.image4[0];

const images = [image1, image2, image3, image4].filter((item) => item !== undefined);

let imagesUrl = await Promise.all(
  images.map(async (item) => {
    let result = await cloudinary.uploader.upload(item.path, { resource_type: 'image' });
    return result.secure_url;
  })
);
```

```
}  
);
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao em lại sử dụng `Promise.all` khi upload nhiều ảnh lên Cloudinary thay vì dùng vòng lặp `for` tuần tự?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, nếu dùng vòng lặp `for` với `await`, 4 bức ảnh sẽ phải upload tuần tự nối tiếp nhau (ảnh 1 xong mới tới ảnh 2), tổng thời gian sẽ là tổng thời gian của 4 request (ví dụ $1s * 4 = 4$ giây). Sử dụng `Promise.all(images.map(...))` cho phép Node.js bắn 4 request song song (concurrently) lên Cloudinary cùng lúc, thời gian upload chỉ bằng thời gian của request lâu nhất (khoảng hơn 1 giây), giúp giảm thời gian phản hồi cho Admin gấp gần 4 lần.

Câu 10 — Đánh Giá Tính Khả Thi & Ứng Dụng Thực Tế

CÂU HỎI

Đồ án này đã đủ điều kiện để đưa vào vận hành kinh doanh thực tế ngoài đời thực chưa? Nếu chưa thì còn thiếu những yếu tố gì?

CÂU TRẢ LỜI

Hệ thống đã hoàn thiện trọn vẹn toàn bộ luồng nghiệp vụ cốt lõi (End-to-End Core Flow) từ xem hàng, giỏ hàng, đặt hàng, sinh mã thanh toán VietQR, đến quản lý đơn và chăm sóc khách hàng. Tuy nhiên, để đưa vào vận hành thương mại quy mô lớn ngoài đời thực, cần bổ sung 3 yếu tố nghiệp vụ nâng cao: 1. **Kết nối API đơn vị vận chuyển bên ngoài**: Tích hợp API Giao Hàng Nhanh (GHN) hoặc Giao Hàng Tiết Kiệm (GHTK) để tự động tính phí ship theo cân nặng/khoảng cách và đẩy mã vận đơn (Tracking code) tự động. 2. **Xác thực 2 lớp (2FA / OTP)**: Gửi mã OTP qua SMS hoặc Email khi đăng ký tài khoản hoặc đổi mật khẩu để bảo vệ tài khoản khách hàng. 3. **Cơ chế Khóa giao dịch (Transaction Lock)**: Triển khai MongoDB Transaction cho thao tác trừ tồn kho nhằm đảm bảo không bao giờ bị bán âm hàng khi có hàng ngàn khách đặt mua cùng một giây.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Toàn bộ đồ án

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu đưa hệ thống này lên môi trường thực tế trên Internet, em dự kiến chi phí hạ tầng hàng tháng tối thiểu là bao nhiêu và gồm những dịch vụ nào?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, hệ thống có thể chạy trên hạ tầng đám mây với chi phí khởi điểm cực kỳ tối ưu: - Backend: Thuê 1 VPS Linux (2 Core, 4GB RAM) tại Việt Nam hoặc Cloud Run: khoảng 150.000 - 250.000đ/tháng. - Database: MongoDB Atlas gói M0/M2: từ miễn phí đến khoảng 200.000đ/tháng. - Frontend & Admin: Host tĩnh hoàn toàn miễn phí trên Vercel hoặc Cloudflare Pages (với băng thông không giới hạn). - Cloudinary & Gemini: Đều có gói Free Tier dư dả cho giai đoạn khởi đầu. Tổng chi phí ban đầu chỉ khoảng 300.000đ - 400.000đ/tháng.

NHÓM 2: KIẾN TRÚC HỆ THỐNG (SYSTEM ARCHITECTURE) (CÂU 11 — 20)

Câu 11 — Mô hình Kiến trúc Client - Server 3 Lớp

CÂU HỎI

Em hãy vẽ và giải thích mô hình kiến trúc 3 lớp (3-Tier Architecture) của hệ thống?

CÂU TRẢ LỜI

Hệ thống được thiết kế theo mô hình kiến trúc 3 lớp phân tán: 1. **Presentation Tier (Tầng hiển thị / Giao diện)**: - Bao gồm 2 SPA (Single Page Applications): Frontend dành cho khách và Admin Dashboard dành cho quản trị viên, được xây dựng bằng React 18 và Vite. Giao tiếp với tầng logic hoàn toàn thông qua các giao thức HTTP RESTful JSON. 2. **Application Tier (Tầng logic nghiệp vụ / Backend)**: - Node.js runtime kết hợp Express framework. Chịu trách nhiệm tiếp nhận request, xác thực JWT, kiểm tra quyền hạn, thực thi quy tắc kinh doanh (Business Logic) và giao tiếp với các dịch vụ ngoài (Cloudinary, Gemini, VietQR). 3. **Data Tier (Tầng dữ liệu / Database)**: - Cơ sở dữ liệu MongoDB Atlas lưu trữ các Documents dưới dạng BSON. Được quản lý và ánh xạ dữ liệu thông qua Mongoose ODM.



ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Toàn bộ hệ thống

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Ưu điểm lớn nhất của việc tách rời Presentation Tier và Application Tier so với mô hình nguyên khối cũ (Monolithic như nén Blade trong Laravel hay JSP/Servlet trong Java) là gì?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, ưu điểm lớn nhất là **Tính độc lập và Khả năng tái sử dụng API (Decoupling & API Reusability)**. Backend Express đóng vai trò là một Headless API Engine. Sau này nếu doanh nghiệp muốn phát triển thêm ứng dụng di động iOS/Android bằng React Native hay Flutter, toàn bộ API backend hiện tại có thể tái sử dụng 100% mà không cần phải viết lại bất kỳ dòng code logic hay truy vấn database nào.

Câu 12 — Thiết Kế RESTful API

CÂU HỎI

Các API trong hệ thống của em có tuân thủ đúng nguyên tắc RESTful không? Cho ví dụ cụ thể?

CÂU TRẢ LỜI

Các API trong hệ thống được thiết kế tuân theo định hướng RESTful: 1. **Tài nguyên được định danh bằng danh từ số ít hoặc số nhiều rõ ràng**: - `/api/product` : Tài nguyên sản phẩm. - `/api/order` : Tài nguyên đơn hàng. - `/api/user` : Tài nguyên người dùng. - `/api/coupon` : Tài nguyên mã giảm giá. 2. **Sử dụng đúng các HTTP Methods theo ngữ nghĩa**: - `GET /api/user/admin/users` : Đọc danh sách người dùng (Read). - `POST /api/user/admin/users` : Tạo mới người dùng (Create). - `PUT /api/user/admin/users/:userId` : Cập nhật toàn bộ/một phần người dùng (Update). - `DELETE /api/user/admin/users/:userId` : Xóa tài nguyên người dùng (Delete). - `DELETE /api/wishlist/remove/:productId` : Xóa sản phẩm khỏi wishlist. 3. **Stateless (Phi trạng thái)**: Mỗi request gửi lên máy chủ đều mang đầy đủ thông tin xác thực qua Header `token`, máy chủ không cần lưu session người dùng trong RAM.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Routes: `userRoute.js`, `wishlistRoute.js`, `productRoute.js`

FILE CODE LIÊN QUAN

- `backend/routes/userRoute.js` (Dòng 30 - 34)

```
// Trích từ backend/routes/userRoute.js
userRouter.post("/admin/users", adminAuth, createUser);
userRouter.get("/admin/users", adminAuth, getAllUsers);
userRouter.get("/admin/users/:userId", adminAuth, getUserById);
userRouter.put("/admin/users/:userId", adminAuth, updateUser);
userRouter.delete("/admin/users/:userId", adminAuth, deleteUser);
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Trong `productRoute.js`, em lại dùng `productRouter.post('/remove', adminAuth, removeProduct)` thay vì `DELETE /api/product/:id`. Tại sao lại có sự không đồng nhất này?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, đây là một điểm kế thừa trong quá trình phát triển dự án. Ở module `userRoute` và `wishlistRoute`, em đã áp dụng đầy đủ chuẩn RESTful với method `DELETE` và URL params. Tuy nhiên ở `productRoute`, do một số form HTML/Client cũ ưu tiên dùng method `POST` để dễ truyền tải body JSON nên hàm xóa vẫn đang dùng `POST /remove`. Trong đợt tái cấu trúc sắp tới, em sẽ chuẩn hóa toàn bộ các route xóa sản phẩm về chuẩn `DELETE /api/product/:id` để đạt tính đồng nhất 100%.

Câu 13 — Cơ Chế Giao Tiếp Axios & Giải Pháp CORS

CÂU HỎI

Frontend gửi dữ liệu lên Backend như thế nào và em đã cấu hình xử lý CORS (Cross-Origin Resource Sharing) ra sao?

CÂU TRẢ LỜI

1. Giao tiếp qua Axios:

- Frontend và Admin sử dụng thư viện HTTP client `axios` để thực hiện các cuộc gọi API.
- Header request được gắn kèm token xác thực: `{ headers: { token } }`.
- Dữ liệu gửi đi được tự động serialize thành chuỗi JSON (với request thông thường) hoặc gửi dạng `FormData` (khi có upload ảnh).

5. Xử lý CORS trong Backend:

- Do Frontend chạy tại port 5173 (hoặc port khác) trong khi Backend chạy tại port 4000, trình duyệt sẽ kích hoạt cơ chế bảo mật Same-Origin Policy và chặn các request cross-origin.
- Trong `backend/server.js`, em sử dụng middleware `cors()`: `javascript import cors from "cors"; app.use(cors());`
- Middleware này tự động thêm các HTTP response headers: `Access-Control-Allow-Origin: *`, `Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS`, và xử lý các Preflight Request (`OPTIONS`) từ trình duyệt.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `server.js` (Dòng 26)
- Frontend: `ShopContext.jsx`

FILE CODE LIÊN QUAN

- `backend/server.js` (Dòng 25 - 28)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Việc cấu hình `app.use(cors())` mở cho mọi domain (`*`) có gây rủi ro bảo mật gì không và khi lên môi trường Production em sẽ cấu hình lại thế nào?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, việc để `cors()` mặc định cho phép `*` có thể khiến bất kỳ website độc hại nào khác cũng có thể gửi ajax request tới backend nếu người dùng bị tấn công CSRF. Khi triển khai lên môi trường Production thực tế, em sẽ cấu hình danh sách trắng (Whitelist Domains):

```
const allowedOrigins = ['https://minhtuanshop.vn', 'https://admin.minhtuanshop.vn'];
app.use(cors({
  origin: function (origin, callback) {
    if (!origin || allowedOrigins.indexOf(origin) !== -1) {
      callback(null, true);
    } else {
      callback(new Error('Bị chặn bởi chính sách CORS'));
    }
  },
  credentials: true
}));
```

Câu 14 — Cơ Chế Polling Thời Gian Thực

CÂU HỎI

Hệ thống của em có tính năng chat và kiểm tra trạng thái thanh toán VietQR tự động. Em đang sử dụng cơ chế nào để cập nhật dữ liệu và tại sao không dùng Socket.IO?

CÂU TRẢ LỜI

Hệ thống hiện đang sử dụng cơ chế **Short-Polling (Thăm dò định kỳ)**: 1. **Tại widget Chat khách hàng (ChatWidget.jsx)**: Khi mở khung chat, `setInterval` được kích hoạt để gọi `GET /api/chat/messages/:chatId` mỗi 3000ms (3 giây). 2. **Tại trang Admin Customer Support (CustomerSupport.jsx)**: Polling danh sách chat mỗi 5 giây và polling nội dung tin nhắn phòng chat đang chọn mỗi 3 giây. 3. **Tại trang cổng thanh toán VietQR (PaymentGateway.jsx)**: Polling `GET /api/payment/order-status/:orderId` mỗi 3 giây. Khi nhận được trạng thái `payment: true`, component tự động dừng `clearInterval` và kích hoạt hiệu ứng thành công.

Lý do lựa chọn Polling: - Đơn giản, tin cậy tuyệt đối, hoạt động qua cổng HTTP(S) thông thường mà không bao giờ bị các tường lửa mạng (Firewall), proxy của công ty hay mạng 4G chặn kết nối WebSocket. - Kiến trúc hoàn toàn Stateless, dễ dàng scale backend trên các nền tảng Serverless như Vercel/Render mà không cần duy trì kết nối TCP liên tục (Persistent connection).

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: ChatWidget.jsx, PaymentGateway.jsx
- Admin: CustomerSupport.jsx

FILE CODE LIÊN QUAN

- frontend/src/pages/PaymentGateway.jsx (Dòng 48 - 72)
- frontend/src/components/ChatWidget.jsx (Dòng 27 - 36)

```
// Trích từ frontend/src/pages/PaymentGateway.jsx
useEffect(() => {
  if (isPaid) return;

  pollingRef.current = setInterval(async () => {
    try {
      const res = await axios.get(`${backendUrl}/api/payment/order-status/${orderId}`);
      if (res.data?.success && res.data.payment) {
        setIsPaid(true);
        clearInterval(pollingRef.current);
        clearInterval(timerRef.current);
        toast.success('🎉 Thanh toán tự động thành công! Đơn hàng đã được xác nhận.');
```

Đón hàng đã được xác nhận.

```
        setTimeout(() => { navigate('/orders'); }, 3500);
      }
    } catch (err) {
      console.warn('Lỗi polling:', err);
    }
  }, 3000);

  return () => {
    if (pollingRef.current) clearInterval(pollingRef.current);
  };
}, [orderId, backendUrl, isPaid, navigate]);
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu có 10.000 khách hàng cùng mở trang chat cùng lúc, mỗi người bắn 1 request mỗi 3 giây thì server sẽ nhận hơn 3.300 requests/giây. Lúc đó giải pháp Polling này sẽ gặp vấn đề gì và em giải quyết thế nào?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, khi lượng người dùng đồng thời (CCU) tăng cao, Polling sẽ gây lãng phí băng thông nghiêm trọng (HTTP overhead từ header) và làm cạn kiệt connection pool của MongoDB do đa số request thăm dò trả về dữ liệu rỗng (không có tin nhắn mới). Để giải quyết bài toán tải lớn: 1. Em sẽ chuyển sang **WebSocket (Socket.IO)** hoặc **Server-Sent Events (SSE)**: Máy chủ chỉ đẩy dữ liệu xuống client khi có sự kiện tin nhắn mới phát sinh (Event-driven). 2. Dùng **Redis Pub/Sub** làm message broker trung gian giữa các tiến trình Node.js để phân phối tin nhắn đến đúng socket ID của người nhận.

Câu 15 — Xử Lý Bất Đồng Bộ (Asynchronous Processing) Trong Node.js

CÂU HỎI

Node.js là đơn luồng (Single-threaded). Làm thế nào backend của em có thể xử lý đồng thời nhiều thao tác nặng như kết nối database, gọi Gemini AI và gửi Email?

CÂU TRẢ LỜI

Node.js sử dụng kiến trúc **Single-threaded Event Loop kết hợp thư viện libuv**: 1. Tuy luồng thực thi JavaScript (Call Stack) là đơn luồng, nhưng các tác vụ I/O nặng (Đọc ghi file Multer, truy vấn mạng MongoDB, gọi HTTP request sang Gemini AI, gửi SMTP socket qua Nodemailer) được giao phó (offload) cho hệ thống **libuv Thread Pool** và nhân hệ điều hành (Kernel Async I/O) xử lý ngầm ở tầng C++. 2. Khi tác vụ I/O hoàn tất, callback/promise được đẩy vào Task Queue (Microtask Queue hoặc Macrotask Queue). Event Loop sẽ nhặt các kết quả này đưa trở lại Call Stack để thực thi tiếp khi Call Stack rảnh. 3. Trong mã nguồn, em sử dụng triệt để cú pháp `async/await` kết hợp `Promise.all` để không bao giờ làm block Event Loop của server.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Toàn bộ backend controllers

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu trong code của em có một hàm tính toán CPU-intensive (ví dụ một vòng lặp `for` chạy 10 tỷ lần để mã hóa dữ liệu), Event Loop sẽ bị ảnh hưởng thế nào và cách xử lý trong Node.js là gì?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, vì Node.js chỉ có một luồng chính thực thi mã JavaScript, hàm chạy 10 tỷ lần này sẽ chiếm giữ Call Stack liên tục (CPU Bound). Trong suốt thời gian đó, Event Loop bị "đóng băng" (blocked), mọi request của tất cả người dùng khác gửi tới server đều bị treo và không được phản hồi. Để xử lý tác vụ nặng về CPU trong Node.js, giải pháp là: 1. Tách tác vụ sang chạy trên một luồng riêng biệt bằng module `worker_threads` của Node.js. 2. Hoặc đẩy tác vụ ra một Background Worker độc lập thông qua hàng đợi tin nhắn (Message Queue như BullMQ / Redis).

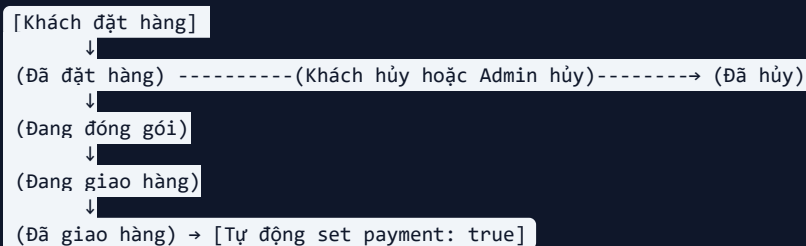
Câu 16 — Xử Lý Đơn Hàng Đa Trạng Thái (Order State Machine)

CÂU HỎI

Em hãy mô tả vòng đời và các trạng thái chuyển tiếp của một đơn hàng trong hệ thống?

CÂU TRẢ LỜI

Đơn hàng được thiết kế theo mô hình Máy trạng thái hữu hạn (Finite State Machine) với danh sách trạng thái hợp lệ được kiểm soát tại `backend/controllers/orderController.js` : - **Đã đặt hàng** : Trạng thái ban đầu khi khách vừa đặt hàng thành công (cho cả đơn COD và đơn chuyển khoản VietQR đang chờ duyệt). - **Đang đóng gói** : Nhân viên xác nhận thông tin và đóng gói hàng. - **Đang giao hàng** : Đơn được giao cho đơn vị vận chuyển. - **Đã giao hàng** : Đơn giao thành công đến tay khách. Hệ thống tự động gán `payment: true` (Đã thu tiền). - **Đã hủy** : Đơn bị hủy bởi User (chỉ khi đơn còn ở bước "Đã đặt hàng") hoặc bị Admin hủy (kèm theo lý do hủy bắt buộc).



ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `orderController.js` (Hàm `updateStatus` , `cancelOrder`)
- Admin: `Orders.jsx`
- Frontend: `Orders.jsx`

FILE CODE LIÊN QUAN

- `backend/controllers/orderController.js` (Dòng 261 - 279)

```
// Trích từ backend/controllers/orderController.js
const validStatuses = ["Đã đặt hàng", "Đang đóng gói", "Đang giao hàng", "Đã giao hàng", "Đã hủy", "Đang xử"];
if (!validStatuses.includes(status)) {
    return res.json({ success: false, message: "Trạng thái không hợp lệ" });
}

const updateData = { status };

// Nếu trạng thái là "Đã giao hàng", đánh dấu đã thanh toán
if (status === "Đã giao hàng") {
    updateData.payment = true;
}

// Nếu trạng thái là "Đã hủy" và có Lý do hủy, Lưu Lý do
if (status === "Đã hủy" && cancelReason) {
    updateData.cancelReason = cancelReason;
}

await orderModel.findByIdAndUpdate(orderId, updateData);
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu đơn hàng đã chuyển sang trạng thái "Đang giao hàng", khách hàng có thể gọi API để hủy đơn được nữa không và backend chặn điều đó ở đâu?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, khách hàng **hoàn toàn không thể hủy được**. Trong hàm `cancelOrder` của `orderController.js`, backend có kiểm tra điều kiện cứng:

```
if (order.status !== "Đã đặt hàng") {  
  return res.json({  
    success: false,  
    message: "Chỉ có thể hủy đơn hàng khi trạng thái là 'Đã đặt hàng'"  
  });  
}
```

Dù khách hàng có can thiệp bằng Postman thì backend vẫn từ chối thực hiện.

Câu 17 — Kiến Trúc Tích Hợp Google Gemini AI

CÂU HỎI

Hệ thống tích hợp Google Gemini AI như thế nào trong module So sánh sản phẩm?

CÂU TRẢ LỜI

Hệ thống giao tiếp trực tiếp với Google Gemini API thông qua hàm bất đồng bộ `callGemini` được xây dựng trong `backend/controllers/compareController.js`: 1. **Khởi tạo & Cấu hình**: Đọc `GEMINI_API_KEY` và model `gemini-3.5-flash-lite` từ biến môi trường. 2. **Đóng gói Prompt**: - Thu thập thông tin chi tiết của các sản phẩm cần so sánh từ MongoDB (tên, giá bán, giá gốc, rating, mô tả, biến thể). - Ghép thành prompt tiếng Việt có cấu trúc yêu cầu phân tích theo 6 tiêu chí: Giá cả, Tính năng/hiệu năng, Độ bền, Phù hợp nhu cầu, Ưu nhược điểm và Lời khuyên khách quan. 3. **Gọi HTTP Request**: Gửi payload `POST` đến endpoint `https://generativelanguage.googleapis.com/v1beta/models/gemini-3.5-flash-lite:generateContent?key=...`. 4. **Phản hồi**: Nhận văn bản phân tích từ `data.candidates[0].content.parts[0].text` và trả về JSON cho React hiển thị dạng Markdown mượt mà trên giao diện `/compare`.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Controller: `backend/controllers/compareController.js`
- Route: `backend/routes/compareRoute.js`
- UI: `frontend/src/pages/Compare.jsx`

FILE CODE LIÊN QUAN

- `backend/controllers/compareController.js` (Dòng 3 - 39)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao em lại sử dụng model `gemini-3.5-flash-lite` thay vì `gemini-pro` hay các model khác?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, `gemini-3.5-flash-lite` là dòng mô hình thể hệ mới nhất của Google được tối ưu hóa đặc biệt cho các tác vụ suy luận nhanh với độ trễ (latency) siêu thấp và chi phí tài nguyên cực rẻ. Với bài toán tư vấn bán hàng thời gian thực trên website, tốc độ phản hồi là yếu tố sống còn cho trải nghiệm người dùng; `gemini-3.5-flash-lite` cho tốc độ sinh văn bản nhanh gấp 2-3 lần so với bản Pro mà chất lượng tiếng Việt vẫn rất tự nhiên, chuẩn xác.

Câu 18 — Quản Lý State Tập Trung Với React Context API

CÂU HỎI

Em hãy trình bày cách thiết kế State Management trong ứng dụng Frontend và giải thích vai trò của `ShopContext.jsx` ?

CÂU TRẢ LỜI

Ứng dụng Frontend sử dụng **React Context API** để quản lý trạng thái chia sẻ toàn cục (Global State) thông qua provider `ShopContextProvider` :

- Các trạng thái lưu trữ trong Context:** - Danh sách sản phẩm tải từ server: `products` . - Giỏ hàng: `cartItems` (cấu trúc `{ [itemId]: { [size]: quantity } }`). - Token xác thực và thông tin người dùng: `token` , `user` . - Các sản phẩm được tick chọn để thanh toán: `selectedItems` . - Sản phẩm mua ngay: `buyNowItem` . - Danh sách yêu thích: `wishlist` . - Thanh tìm kiếm: `search` , `showSearch` .
- Các hàm tiện ích toàn cục:** - `addToCart(itemId, size)` : Thêm sản phẩm vào giỏ và đồng bộ server nếu đã đăng nhập. - `updateQuantity(itemId, size, quantity)` : Cập nhật số lượng. - `getCartCount()` : Tính tổng số lượng hàng trong giỏ để hiển thị lên badge Navbar. - `getCartAmount()` : Tính tổng tiền hàng. - `clearAuthData()` : Xóa sạch session khi đăng xuất hoặc khi token hết hạn.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- File: `frontend/src/context/ShopContext.jsx`

FILE CODE LIÊN QUAN

- `frontend/src/context/ShopContext.jsx` (Dòng 8 - 30, 241 - 258)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Context API có điểm hạn chế gì về hiệu năng (Re-render) so với Redux Toolkit hay Zustand khi ứng dụng mở rộng quy mô?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, nhược điểm lớn nhất của Context API là **Unnecessary Re-renders (Render lại không mong muốn)**. Khi bất kỳ một giá trị nào trong `value` của Provider thay đổi (ví dụ chỉ đổi từ khóa `search`), TẤT CẢ các component nào đang tiêu thụ context qua `useContext(ShopContext)` đều sẽ bị re-render lại, dù component đó chỉ cần `currency` hay `token` . Nếu mở rộng quy mô lớn với hàng trăm component, em sẽ tách nhỏ Context (như `AuthContext` , `CartContext` , `ProductContext`) hoặc chuyển sang dùng **Zustand** để hỗ trợ Selector-based subscription, chỉ re-render đúng component tiêu thụ slice state tương ứng.

Câu 19 — Kiến Trúc Thanh Toán VietQR Cá Nhân Hóa & Duyệt Thủ Công

CÂU HỎI

Em hãy giải thích cơ chế sinh nội dung thanh toán riêng cho từng người và luồng duyệt bằng tay như hình thức COD?

CÂU TRẢ LỜI

Theo yêu cầu thực tế của mô hình kinh doanh, luồng thanh toán VietQR được thiết kế như sau: 1. **Sinh nội dung thanh toán riêng biệt (Unique Transfer Content):** - Khi khách hàng bấm đặt đơn VietQR, backend chạy hàm `generateOrderCode()` sinh mã đơn ngẫu nhiên format `MTxxxxxx` (ví dụ: `MT845219`) và kiểm tra vòng lặp chống trùng lặp mã đơn trong database. - Nội dung chuyển khoản được cấu hình chuẩn: `${orderCode} ${phone}` (ví dụ: `MT845219 0988888888`). - Link VietQR tự động nhúng nội dung này vào query param `addInfo`: `https://img.vietqr.io/image/970422-0907253168-compact2.png?amount=...&addInfo=MT845219%200988888888` 2. **Khách hàng xác nhận:** - Khi quét mã và chuyển tiền xong, khách hàng bấm nút: **"TÔI ĐÃ CHUYỂN TIỀN - XÁC NHẬN ĐƠN HÀNG (CHỜ DUYỆT)"**. - Client gọi API `POST /api/payment/confirm-transfer` -> Đơn hàng giữ nguyên trạng thái `status`: "Đã đặt hàng" (như COD) và ghi nhận `paymentDetails.customerConfirmed = true` 3. **Admin duyệt bằng tay (Manual Approval):** - Đơn hàng xuất hiện trên trang Quản lý đơn hàng của Admin với phương thức "Chuyển khoản VietQR". - Nhân viên kế toán kiểm tra tài khoản ngân hàng xem đã nhận được tiền với đúng mã đơn `MTxxxxxx` hay chưa. - Nếu đã nhận đủ tiền: Admin thao tác trên dropdown chuyển trạng thái đơn sang "Đang đóng gói" hoặc tích chọn đơn để duyệt.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `paymentController.js` (Hàm `createVietQROrder`, `confirmCustomerTransfer`)
- Route: `paymentRoute.js`
- Frontend: `PaymentGateway.jsx`
- Admin: `Orders.jsx`

FILE CODE LIÊN QUAN

- `backend/controllers/paymentController.js` (Dòng 371 - 408)
- `frontend/src/pages/PaymentGateway.jsx` (Dòng 111 - 134, 404 - 425)

```
// Trích từ backend/controllers/paymentController.js
export const confirmCustomerTransfer = async (req, res) => {
  try {
    const { orderId } = req.body;
    const order = await orderModel.findById(orderId);

    if (!order) {
      return res.status(404).json({ success: false, message: "Không tìm thấy đơn hàng" });
    }

    // Đặt vào trạng thái chờ duyệt (như COD)
    order.status = "Đã đặt hàng";
    order.paymentDetails = {
      ...(order.paymentDetails || {}),
      customerConfirmed: true,
      confirmedAt: new Date(),
      note: "Khách hàng đã quét mã và bấm xác nhận chuyển khoản. Chờ Admin kiểm tra và duyệt bằng tay."
    };

    await order.save();
    ...
  }
}
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao lại cần khách bấm nút xác nhận chuyển khoản mà không để họ chuyển tiền xong tự tắt trình duyệt luôn?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, việc khách bấm nút xác nhận mang lại 2 mục đích lớn: 1. **Chuyển dịch trạng thái tâm lý & Xác nhận giao dịch**: Giúp khách hàng yên tâm rằng hệ thống đã ghi nhận việc chuyển tiền thành công và đơn hàng đã được đẩy sang bộ phận kho chuẩn bị hàng, đồng thời tự động điều hướng khách về trang Lịch sử đơn hàng `/orders`. 2. **Đánh dấu cờ (Flagging) cho Admin**: Thuộc tính `customerConfirmed: true` giúp Admin lọc ra được các đơn khách cam kết đã chuyển tiền để ưu tiên mở app ngân hàng đối soát trước, phân biệt với các đơn hàng tạo mã QR xong nhưng khách bỏ dở không chuyển tiền.

Câu 20 — Chiến Lược Xử Lý Lỗi (Error Handling Strategy)

CÂU HỎI

Hệ thống xử lý thế nào khi xảy ra lỗi ở các tầng khác nhau (Database crash, API lỗi mạng, Form nhập sai)?

CÂU TRẢ LỜI

Hệ thống triển khai phòng thủ xử lý lỗi đa tầng: 1. **Tầng Client (Giao diện React)**: - Bắt lỗi dữ liệu nhập qua HTML5 validation và JavaScript Regex trước khi gửi request. - Bọc các lệnh gọi Axios trong khối `try/catch`. Khi gặp lỗi mạng (Network Error) hoặc server trả mã 4xx/5xx -> Bắt qua `error.response?.data?.message` và hiển thị thông báo thân thiện bằng `toast.error()`, không để ứng dụng bị crash trắng màn hình. 2. **Tầng Backend (Express Controller)**: - Mọi hàm xử lý trong controller đều được bọc trong khối `try/catch (error)`. - Bắt lỗi và ghi log `console.log(error)`, sau đó trả về HTTP Status 200 kèm cấu trúc JSON an toàn: `{ success: false, message: error.message }`. - Riêng các lỗi xác thực JWT trong middleware `auth.js`, hệ thống bắt chính xác tên lỗi `TokenExpiredError` để báo người dùng đăng nhập lại. 3. **Tầng Database (Mongoose)**: - Các ràng buộc trùng lặp (Unique Index như email trùng, mã coupon trùng, review trùng `{orderId, productId}`) được Mongoose trả về lỗi E11000 và được controller bắt lại báo thông báo rõ ràng cho người dùng.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `middleware/auth.js`, các controllers
- Frontend: `ShopContext.jsx`, các pages

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao trong nhiều controller em lại trả về HTTP Status 200 kèm `{ success: false, message: ... }` thay vì trả về HTTP Status chuẩn như 400 Bad Request, 404 Not Found hay 500 Internal Server Error?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, việc trả về Status 200 kèm body `{ success: false }` là một design pattern phổ biến trong các ứng dụng web dạng Client-First: - **Ưu điểm**: Phía frontend không bị Axios tự động quăng exception vào khối `catch`, giúp component dễ dàng xử lý các thông báo nghiệp vụ logic (như "Mật khẩu sai", "Hết hàng") một cách mượt mà qua câu lệnh `if (res.data.success)`. - **Nhược điểm & Hướng cải thiện**: Không tuân thủ chặt chẽ chuẩn HTTP Spec và khó tích hợp với các công cụ giám sát mạng tự động (APM Tools). Đối với các route mới nâng cấp như `paymentController.js`, em đã bắt đầu chuẩn hóa trả về đúng mã lỗi HTTP chuẩn như `res.status(400)`, `res.status(404)`, `res.status(500)`.

NHÓM 3: FRONTEND WEB APP (REACT & VITE) (CÂU 21 — 30)

Câu 21 — Quản Lý Vòng Đời Component & Hooks

CÂU HỎI

Em đã sử dụng các React Hooks nào trong ứng dụng và cho ví dụ cụ thể về việc giải quyết vấn đề bằng `useEffect` ?

CÂU TRẢ LỜI

Trong dự án, em vận dụng linh hoạt các React Hooks cơ bản và nâng cao: 1. `useState` : Quản lý state cục bộ của form, modal, dữ liệu bảng. 2. `useEffect` : Quản lý side effects (gọi API khi mount, lắng nghe thay đổi của dependency). 3. `useContext` : Tiêu thụ `ShopContext` toàn cục. 4. `useRef` : Lưu trữ tham chiếu Timer đếm ngược, Polling interval (`pollingRef.current`), và cuộn tin nhắn xuống đáy (`messagesEndRef.current.scrollToInView`). 5. `useMemo` & `useCallback` : Tối ưu hiệu năng bộ lọc thương hiệu theo danh mục trong `Collection.jsx` .

Ví dụ cụ thể với `useEffect` dọn dẹp bộ nhớ (Cleanup function): Trong `PaymentGateway.jsx` , để tránh rò rỉ bộ nhớ (Memory Leak) khi người dùng rời khỏi trang thanh toán:

```
useEffect(() => {  
  // Khởi tạo interval polling  
  pollingRef.current = setInterval(..., 3000);  
  
  // Cleanup function: Tự động hủy interval khi component unmount  
  return () => {  
    if (pollingRef.current) clearInterval(pollingRef.current);  
  };  
}, [orderId]);
```

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `PaymentGateway.jsx` , `Collection.jsx` , `ChatWidget.jsx`

FILE CODE LIÊN QUAN

- `frontend/src/pages/PaymentGateway.jsx` (Dòng 69 - 72)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu trong `useEffect` em không khai báo mảng dependencies `[]` (hoặc khai báo thiếu dependency), điều gì sẽ xảy ra?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Nếu **bỏ trống hoàn toàn mảng dependencies**: `useEffect` sẽ chạy sau MỖI LẦN component re-render. Nếu trong effect có lệnh gọi API và `setState` , nó sẽ kích hoạt render tiếp -> tạo ra **vòng lặp vô tận (Infinite Loop)** làm đơ trình duyệt. - Nếu **truyền mảng rỗng `[]`** : Effect chỉ chạy duy nhất 1 lần khi component mount, phù hợp để load danh sách sản phẩm ban đầu. - Nếu **khai báo thiếu dependency**: Effect sẽ bị hiện tượng **Stale Closure** (dữ liệu bị cũ), effect đọc giá trị của state tại thời điểm render trước đó mà không nhận được giá trị mới nhất.

Câu 22 — Bảo Vệ Tuyến Đường Bằng ProtectedRoute

CÂU HỎI

Làm thế nào em ngăn chặn người dùng chưa đăng nhập truy cập vào các trang riêng tư như Trang cá nhân (/profile)?

CÂU TRẢ LỜI

Em sử dụng kỹ thuật bọc route bằng Higher-Order Component / Wrapper Component ProtectedRoute.jsx : 1. Trong file định tuyến App.jsx : `jsx <Route path="/profile" element={ <ProtectedRoute> <AnimatedRoute><Profile /></AnimatedRoute> </ProtectedRoute> } />` 2. Trong ProtectedRoute.jsx : Component lấy token từ ShopContext . Nếu không có token trong state hoặc localStorage , component sẽ chuyển hướng người dùng về trang đăng nhập /login .

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: src/components/ProtectedRoute.jsx , src/App.jsx

FILE CODE LIÊN QUAN

- frontend/src/components/ProtectedRoute.jsx (Dòng 1 - 16)
- frontend/src/App.jsx (Dòng 112 - 121)

```
// Trích từ frontend/src/components/ProtectedRoute.jsx
import React, { useContext } from "react";
import { ShopContext } from "../context/ShopContext";

const ProtectedRoute = ({ children }) => {
  const { token, navigate } = useContext(ShopContext);

  if (!token) {
    navigate("/login");
    return null;
  }

  return children;
};

export default ProtectedRoute;
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Trong code ProtectedRoute.jsx trên, việc gọi navigate("/login") trực tiếp ngay trong thân component khi render (thay vì bọc trong useEffect hoặc dùng <Navigate to="/login" replace />) có thể gây ra cảnh báo gì từ React Router?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, việc gọi navigate() đồng bộ trong quá trình render vi phạm nguyên tắc của React 18: "Cannot update a component (BrowserRouter) while rendering a different component (ProtectedRoute)". Cách viết chuẩn nhất để khắc phục là sử dụng thẻ điều hướng khai báo của React Router:

```
import { Navigate } from "react-router-dom";
const ProtectedRoute = ({ children }) => {
  const { token } = useContext(ShopContext);
  if (!token) {
    return <Navigate to="/login" replace />;
  }
  return children;
};
```

Câu 23 — Kỹ Thuật Lọc Đa Tiêu Chí Trong Collection.jsx

CÂU HỎI

Em hãy trình bày thuật toán lọc sản phẩm theo nhiều tiêu chí kết hợp (Category + Brand + Khoảng giá + Còn hàng + Từ khóa tìm kiếm)?

CÂU TRẢ LỜI

Trong `frontend/src/pages/Collection.jsx`, quá trình lọc dữ liệu được thực thi phía Client trên mảng `products` thông qua pipeline lọc tuần tự: 1. **Bước 1 - Lọc theo Danh mục (Categories)**: Nếu mảng `selectedCategories.length > 0`, giữ lại các sản phẩm có `selectedCategories.includes(product.category)`. 2. **Bước 2 - Lọc theo Hãng (Brands)**: Nếu `selectedBrands.length > 0`, giữ lại sản phẩm có `selectedBrands.includes(product.subCategory)`. 3. **Bước 3 - Lọc theo Khoảng giá (Price Range)**: Đối chiếu `product.price` nằm trong khoảng `[min, max]` của mức giá được chọn (Dưới 5tr, 5-15tr, 15-30tr, Trên 30tr). 4. **Bước 4 - Lọc Tồn kho (In-stock only)**: Nếu tick `onlyInStock`, loại bỏ các sản phẩm có tổng tồn kho bằng 0. 5. **Bước 5 - Lọc Tìm kiếm (Search Keyword)**: So khớp chuỗi không phân biệt hoa thường `product.name.toLowerCase().includes(search.toLowerCase())`. 6. **Bước 6 - Sắp xếp (Sorting)**: Sắp xếp theo Giá tăng dần, Giá giảm dần, hoặc Ngày tạo mới nhất.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `frontend/src/pages/Collection.jsx`

FILE CODE LIÊN QUAN

- `frontend/src/pages/Collection.jsx` (Dòng 100 - 180)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu danh mục có 100.000 sản phẩm, việc lọc toàn bộ trên Client như hiện tại sẽ gây hậu quả gì và giải pháp tối ưu là gì?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, nếu có 100.000 sản phẩm, việc kéo toàn bộ dữ liệu về Client sẽ làm browser bị tràn RAM, đơ giao diện và tốn hàng chục MB băng thông mạng. Giải pháp là chuyển sang **Server-side Filtering & Pagination**: - Client chỉ gửi query parameters lên API: `GET /api/product/list?category=Laptop&brand=Dell&minPrice=15000000&maxPrice=30000000&page=1&limit=16`. - Backend MongoDB sẽ dùng Index `{ category: 1, brand: 1, price: 1 }` để truy vấn cực nhanh trong vài mili-giây và chỉ trả về đúng 16 bản ghi của trang hiện tại kèm tổng số trang.

Câu 24 — Quản Lý Giỏ Hàng: Cấu Trúc Dữ Liệu & Đồng Bộ

CÂU HỎI

Tại sao cấu trúc dữ liệu giỏ hàng `cartItems` trong `ShopContext.jsx` lại được thiết kế dạng Nested Object `{ [itemId]: { [size]: quantity } }` thay vì một mảng mỏng `[{ itemId, size, quantity }]` ?

CÂU TRẢ LỜI

Cấu trúc Nested Object mang lại ưu thế vượt trội về hiệu năng và độ phức tạp thuật toán: 1. **Truy xuất và cập nhật với độ phức tạp O(1)**: Khi người dùng bấm tăng số lượng một phiên bản sản phẩm (ví dụ: iPhone 15 Pro, bản 256GB), hệ thống có thể truy cập thẳng: `cartData[itemId][size] += 1` mà không cần phải duyệt qua toàn bộ mảng bằng `find()` hay `map()` với độ phức tạp O(N). 2. **Ngăn chặn trùng lặp tự nhiên (Deduplication)**: Cấu trúc Object Key tự động đảm bảo cùng một sản phẩm với cùng một cấu hình không bao giờ bị nhân bản thành 2 dòng riêng biệt. 3. **Dễ dàng quản lý nhiều biến thể của cùng một sản phẩm**: Một chiếc Laptop có thể có nhiều cấu hình (16GB RAM hoặc 32GB RAM) nằm gọn dưới một `itemId`.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `ShopContext.jsx` (Hàm `addToCart`, `updateQuantity`)
- Model: `userModel.js` (Trường `cartData`)

FILE CODE LIÊN QUAN

- `frontend/src/context/ShopContext.jsx` (Dòng 87 - 100)

```
// Trích từ frontend/src/context/ShopContext.jsx
let cartData = structuredClone(cartItems);

if (cartData[itemId]) {
  if (cartData[itemId][size]) {
    cartData[itemId][size] += 1;
  } else {
    cartData[itemId][size] = 1;
  }
} else {
  cartData[itemId] = {};
  cartData[itemId][size] = 1;
}
setCartItems(cartData);
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao trong đoạn code trên em lại sử dụng `structuredClone(cartItems)` thay vì toán tử spread `{ ...cartItems }` ?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, `cartItems` là một object lồng 2 cấp (Nested Object). Toán tử spread `{ ...cartItems }` chỉ tạo bản sao nông (**Shallow Copy**), nghĩa là các object con bên trong cấp 2 (`cartItems[itemId]`) vẫn dùng chung tham chiếu bộ nhớ (memory reference) với state cũ. Nếu thay đổi trực tiếp `cartData[itemId][size]`, ta đã vô tình làm đột biến trực tiếp (Mutate) React state cũ trước khi gọi `setCartItems`, vi phạm nguyên tắc Bất biến (Immutability) của React. Sử dụng `structuredClone()` tạo ra bản sao sâu (**Deep Copy**) hoàn chỉnh, đảm bảo an toàn tuyệt đối cho State Management.

Câu 25 — Tối Ưu Trải Nghiệm Mua Hàng Nhanh (Buy Now)

CÂU HỎI

Tính năng "MUA NGAY" tại trang chi tiết sản phẩm hoạt động như thế nào và có gì khác biệt so với "THÊM VÀO GIỎ HÀNG"?

CÂU TRẢ LỜI

Hai tính năng phục vụ 2 hành vi khách hàng khác biệt: 1. **"Thêm vào giỏ hàng"**: - Dành cho khách muốn tiếp tục dạo xem nhiều sản phẩm khác. - Sản phẩm được nạp vào `cartItems`, lưu vào state/database và tăng badge giỏ hàng trên Navbar. Khách vẫn ở lại trang chi tiết. 2. **"Mua ngay" (Buy Now)**: - Dành cho khách muốn chốt đơn lập tức. - Bỏ qua bước vào giỏ hàng (`/cart`), tạo thẳng một object `buyNowItem`: `javascript const buyNowItem = { _id: productData._id, size: selectedVariant, quantity: quantity, product: productData }; setBuyNowItem(buyNowItem); navigate('/place-order');` - Tại màn hình `/place-order`, hàm `getOrderItems()` ưu tiên kiểm tra nếu có `buyNowItem` thì chỉ thanh toán duy nhất món này, giữ nguyên toàn bộ giỏ hàng cũ của người dùng không bị xóa mất. Khi rời trang checkout, `buyNowItem` tự động được dọn dẹp về `null`.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `Product.jsx` (Hàm `handleBuyNow`), `PlaceOrder.jsx` (Hàm `getOrderItems`)

FILE CODE LIÊN QUAN

- `frontend/src/pages/Product.jsx` (Dòng 49 - 68)
- `frontend/src/pages/PlaceOrder.jsx` (Dòng 92 - 100)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu người dùng bấm "Mua ngay", vào tới trang checkout rồi bấm nút Back quay lại trang sản phẩm thì giỏ hàng bình thường của họ có bị ảnh hưởng gì không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, hoàn toàn không bị ảnh hưởng. Trong `PlaceOrder.jsx`, em có hook dọn dẹp:

```
useEffect(() => {  
  return () => {  
    setBuyNowItem(null);  
  };  
}, [setBuyNowItem]);
```

Khi người dùng Back ra ngoài, `buyNowItem` được reset về `null`, giỏ hàng `cartItems` của khách vẫn được bảo toàn nguyên vẹn.

Câu 26 — Tích Hợp Thư Viện Swiper Carousel

CÂU HỎI

Tại sao em chọn thư viện Swiper để làm gallery ảnh sản phẩm mà không dùng thẻ `img` đơn thuần hay thư viện slick-carousel?

CÂU TRẢ LỜI

Em chọn Swiper vì các tính năng vượt trội: 1. **Hỗ trợ Thumbnail trượt đồng bộ (Thumbs Module)**: Swiper hỗ trợ liên kết 2 slider với nhau (`thumbs={{ swiper: thumbsSwiper }}`), khi click thumbnail bên dưới thì ảnh lớn bên trên tự chuyển động mượt mà và ngược lại. 2. **Cảm ứng di động hoàn hảo (Touch & Swipe)**: Hỗ trợ vuốt chạm bằng ngón tay mượt mà trên mobile với gia tốc quán tính thực tế. 3. **Hiệu năng & Kích thước nhẹ**: Swiper là thư viện hiện đại viết hoàn toàn bằng TypeScript, hỗ trợ Tree-shaking (chỉ import đúng các module cần dùng như `Navigation` , `Pagination` , `Thumbs`), nhẹ hơn rất nhiều so với slick-carousel (vốn phụ thuộc vào jQuery).

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `Product.jsx`

FILE CODE LIÊN QUAN

- `frontend/src/pages/Product.jsx` (Dòng 11 - 16)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu sản phẩm chỉ có 1 ảnh duy nhất trong cơ sở dữ liệu thì component Swiper của em có bị lỗi hiển thị không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, hoàn toàn không lỗi. Trong `Product.jsx` , em đã kiểm tra:

```
const images = Array.isArray(productData.image) && productData.image.length > 0
  ? productData.image
  : [productData.image || 'https://images.unsplash.com/...'];
```

Nếu chỉ có 1 ảnh, Swiper sẽ render 1 Slide duy nhất và ẩn các nút điều hướng Next/Prev để giao diện luôn thẩm mỹ.

Câu 27 — Xử Lý Đổi Mật Khẩu An Toàn Phía Client

CÂU HỎI

Tại màn hình Thông tin cá nhân (`/profile`), form Đổi mật khẩu được validate những điều kiện gì trước khi gọi API?

CÂU TRẢ LỜI

Form đổi mật khẩu tại `Profile.jsx` kiểm tra chặt chẽ 4 bước validation: 1. **Kiểm tra rỗng Mật khẩu hiện tại**: Bắt buộc nhập để chứng minh chính chủ tài khoản. 2. **Kiểm tra rỗng Mật khẩu mới**: Không cho phép đặt mật khẩu rỗng. 3. **Kiểm tra độ dài mật khẩu mới**: Phải có tối thiểu 8 ký tự (`passwordData.newPassword.length >= 8`). 4. **Kiểm tra trùng khớp mật khẩu xác nhận**: `javascript if (passwordData.newPassword !== passwordData.confirmPassword) { toast.error("Mật khẩu xác nhận không khớp"); return; }` Chỉ khi thỏa mãn toàn bộ 4 điều kiện trên, request HTTP `PUT /api/user/profile` mới được gửi đi kèm header token.

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `frontend/src/pages/Profile.jsx` (Hàm `changePassword`)

FILE CODE LIÊN QUAN

- `frontend/src/pages/Profile.jsx` (Dòng 110 - 132)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Sau khi đổi mật khẩu thành công, có nên để người dùng tiếp tục sử dụng token cũ không hay bắt đăng nhập lại?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, về mặt bảo mật tốt nhất (Security Best Practice), ngay sau khi đổi mật khẩu thành công, hệ thống nên tự động gọi hàm `clearAuthData()` để xóa token cũ khỏi `localStorage` và chuyển hướng người dùng về trang `/login` bắt buộc đăng nhập lại bằng mật khẩu mới, nhằm vô hiệu hóa các phiên đăng nhập cũ trên các thiết bị khác nếu tài khoản từng bị lộ.

Câu 28 — Xử Lý Dữ Liệu Rỗng (Empty States)

CÂU HỎI

Em đã thiết kế và xử lý các trạng thái rỗng (Empty States) trong ứng dụng như thế nào?

CÂU TRẢ LỜI

Hệ thống không bao giờ để màn hình trống trơn làm hụt hẫng người dùng: 1. **Giỏ hàng rỗng (/cart)**: Hiển thị icon giỏ hàng lớn màu đỏ, thông báo "Giỏ hàng của bạn đang trống" kèm nút kêu gọi hành động to bản "Khám phá sản phẩm ngay →" dẫn về trang sản phẩm. 2. **Đơn hàng rỗng (/orders)**: Hiển thị hình minh họa đơn hàng, thông báo "Chưa có đơn hàng nào" kèm nút "Mua sắm ngay". 3. **Wishlist rỗng (/wishlist)**: Hiển thị thông báo chưa có sản phẩm yêu thích kèm gợi ý khám phá sản phẩm. 4. **Bộ lọc không có kết quả (collection.jsx)**: Hiển thị thông báo "Không tìm thấy sản phẩm phù hợp" kèm nút "Xóa tất cả bộ lọc".

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Pages: `Cart.jsx`, `Orders.jsx`, `Wishlist.jsx`, `Collection.jsx`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao Empty State lại đóng vai trò quan trọng trong thiết kế trải nghiệm người dùng (UX Design)?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, Empty State là cơ hội để giữ chân người dùng (User Retention). Một màn hình trống trơn hoặc báo lỗi cụt lủn sẽ làm người dùng bối rối tưởng ứng dụng bị hỏng và rời bỏ trang web. Thiết kế Empty State thân thiện kèm lời hướng dẫn và nút CTA (Call to Action) rõ ràng sẽ điều hướng người dùng quay trở lại luồng mua sắm chính, gia tăng tỷ lệ chuyển đổi (Conversion Rate) cho doanh nghiệp.

Câu 29 — Hiệu Ứng Chuyển Trang Framer Motion

CÂU HỎI

Component `PageTransition.jsx` và `AnimatedRoute.jsx` hoạt động như thế nào để tạo hiệu ứng mượt mà khi chuyển đổi giữa các trang?

CÂU TRẢ LỜI

Em sử dụng thư viện **Framer Motion** kết hợp React Router: 1. Trong `App.jsx`, toàn bộ `<Routes>` được bọc trong `<AnimatePresence mode="wait">` với key là `location.pathname`: `jsx <AnimatePresence mode="wait"> <Routes location={location} key={location.pathname}>` 2. Mỗi route được bọc trong component `AnimatedRoute` hoặc thẻ `<PageTransition>`:
- Khi chuyển trang, trang cũ sẽ thực hiện hiệu ứng mờ dần (Exit animation: `opacity: 0, y: -10`). - Sau khi trang cũ unmount hoàn toàn (`mode="wait"`), trang mới sẽ trượt nhẹ vào và rõ dần (Initial animation: `opacity: 0, y: 15` -> `Animate: opacity: 1, y: 0`). - Mang lại cảm giác mượt mà, chuyên nghiệp như một ứng dụng Native App trên di động.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `src/App.jsx`, `src/components/AnimatedRoute.jsx`, `src/components/PageTransition.jsx`

FILE CODE LIÊN QUAN

- `frontend/src/App.jsx` (Dòng 33 - 35)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu thiết bị của người dùng là máy tính đời cũ hoặc điện thoại yếu, các hiệu ứng animation của Framer Motion có làm lag trang web không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, Framer Motion tối ưu hóa hiệu năng bằng cách can thiệp chủ yếu vào các thuộc tính CSS `transform` (GPU Acceleration) và `opacity` vốn được trình duyệt xử lý trực tiếp trên GPU (Compositor Thread) thay vì CPU, không gây hiện tượng Repaint hay Reflow layout. Tuy nhiên, để tối ưu cho người dùng nhạy cảm với chuyển động, ta có thể bổ sung CSS media query `@media (prefers-reduced-motion: reduce)` để tự động tắt hiệu ứng nếu máy tính người dùng bật chế độ tiết kiệm tài nguyên.

Câu 30 — Tính Năng Đánh Dấu Hữu Ích & Báo Cáo Đánh Giá

CÂU HỎI

Tại khối Đánh giá sản phẩm, chức năng "Hữu ích" (Helpful) và "Báo cáo" (Report) được xử lý logic như thế nào?

CÂU TRẢ LỜI

- Đánh dấu Hữu ích (Toggle Helpful):**
- Lưu trữ mảng `helpful: [userId]` trong `reviewModel.js`.
- Khi User đăng nhập bấm "Hữu ích", gọi `POST /api/review/helpful`: `javascript const isHelpful = review.helpful.includes(userId); if (isHelpful) { review.helpful = review.helpful.filter(id => id !== userId); // Bỏ thích } else { review.helpful.push(userId); // Thích } await review.save();`
- Số lượt hữu ích được đếm bằng `review.helpful.length`.

5. Báo cáo vi phạm (Report Review):

6. Lưu trữ mảng `reported: [userId]` trong DB.

7. Khi khách báo cáo vi phạm, hệ thống kiểm tra nếu user này đã từng báo cáo rồi thì từ chối: `if (review.reported.includes(userId)) return res.json(...)`.

8. Giúp Admin lọc ra các bình luận có nhiều lượt báo cáo để kiểm duyệt nội dung độc hại.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `reviewController.js` (Hàm `markHelpful`, `reportReview`)
- Model: `reviewModel.js`
- Frontend: `ReviewList.jsx`

FILE CODE LIÊN QUAN

- `backend/controllers/reviewController.js` (Dòng 363 - 429)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao lại lưu mảng `userId` thay vì chỉ lưu một trường số nguyên `helpfulCount: { type: Number, default: 0 }` ?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, nếu chỉ lưu trường số `helpfulCount`, một người dùng có thể bấm nút hàng ngàn lần để "spam ảo" số lượt thích. Lưu trữ mảng `[userId]` mang lại 2 giá trị cốt lõi: 1. **Chống gian lận (Anti-spam)**: Kiểm tra chính xác User này đã bấm hay chưa để chuyển đổi trạng thái Thích / Bỏ thích (Toggle). 2. **Cá nhân hóa giao diện**: Khi User quay lại trang sản phẩm, frontend đối chiếu `userId` trong mảng để hiển thị nút "Đã cảm ơn" màu xanh, giúp người dùng biết mình đã từng tương tác với đánh giá này.

NHÓM 4: BACKEND & RESTFUL API (NODE.JS & EXPRESS) (CÂU 31 — 40)

Câu 31 — Vòng Đời HTTP Request Trong Express

CÂU HỎI

Em hãy giải thích chi tiết vòng đời (Request Lifecycle) từ khi client gửi request đến khi server trả response trong ứng dụng Express của em?

CÂU TRẢ LỜI

Vòng đời một request trong `backend/server.js` trải qua các giai đoạn sau: 1. **Nhận kết nối TCP & Parser**: Request tới cổng 4000 -> Đi qua middleware `cors()` để kiểm tra chính sách chia sẻ tài nguyên -> Đi qua `express.json()` để đọc stream và chuyển đổi body thành object `req.body`. 2. **Định tuyến (Routing)**: Request đi vào Route tương ứng dựa trên tiền tố (Prefix) đã đăng ký trong `server.js` (Ví dụ `/api/order` đi vào `orderRouter`). 3. **Middleware xác thực & xử lý file**: - Nếu route yêu cầu quyền: Đi qua `auth` middleware kiểm tra Header JWT token. Nếu token sai, middleware ngắt vòng đời bằng `res.json()` ngay tại đây. - Nếu route có file: Đi qua `upload.fields()` của Multer để parse file ảnh. 4. **Controller thực thi**: Hàm xử lý trong controller nhận `(req, res)` -> Thực thi nghiệp vụ -> Truy vấn MongoDB qua Mongoose. 5. **Phản hồi (Response)**: Controller gọi `res.json(...)` kết thúc vòng đời request và gửi gói tin JSON về cho Client.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Toàn bộ kiến trúc backend

FILE CODE LIÊN QUAN

- `backend/server.js` (Dòng 25 - 42)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Hàm `next()` trong middleware của Express có tác dụng gì và điều gì xảy ra nếu trong middleware em quên không gọi `next()` cũng không gọi `res.send()` ?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, hàm `next()` là cơ chế chuyển giao quyền kiểm soát (Control Flow) cho middleware hoặc controller tiếp theo trong chuỗi Middleware Stack. Nếu lập trình viên quên không gọi `next()` và cũng không trả response bằng `res.send()/res.json()`, request sẽ bị **treo vĩnh viễn (Hanging Request)** cho đến khi trình duyệt hoặc client bị Timeout (thường sau 2 phút), gây cạn kiệt tài nguyên máy chủ.

Câu 32 — Thuật Toán Bóc Tách Mã Đơn Hàng Từ Nội Dung Chuyển Khoản

CÂU HỎI

Trong `paymentController.js`, em làm thế nào để trích xuất được chính xác mã đơn hàng `MTxxxxxx` từ nội dung chuyển khoản tự do của khách hàng?

CÂU TRẢ LỜI

Em sử dụng biểu thức chính quy (**Regular Expression - Regex**) để nhận diện mẫu mã đơn: 1. Quy chuẩn mã đơn: Luôn bắt đầu bằng 2 chữ cái MT và theo sau là đúng 6 chữ số: `/MT\d{6}/`. 2. Quá trình bóc tách: `javascript let matchedOrderCode = null; if (code) { const codeMatch = String(code).toUpperCase().match(/MT\d{6}/); if (codeMatch) matchedOrderCode = codeMatch[0]; } if (!matchedOrderCode && content) { const contentMatch = String(content).toUpperCase().match(/MT\d{6}/); if (contentMatch) matchedOrderCode = contentMatch[0]; }` 3. Nhờ chuyển chuỗi thành chữ hoa (`.toUpperCase()`), dù khách hàng có gõ chữ thường như `mt123456`, thêm khoảng trắng hay nhập kèm lời nhắn (VD: `Nguyen Van An chuyen tien mt123456`), Regex vẫn trích xuất chính xác mã `MT123456` để tìm kiếm đơn hàng trong database.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/paymentController.js` (Dòng 212 - 227)

FILE CODE LIÊN QUAN

- `backend/controllers/paymentController.js` (Dòng 213 - 227)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu khách hàng gõ nhầm nội dung thành `MT12345` (chỉ có 5 chữ số), Regex trên sẽ trả về gì và hệ thống xử lý ra sao?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, chuỗi `MT12345` (5 chữ số) không khớp với pattern `MT\d{6}`, hàm `.match()` sẽ trả về `null`. Khi đó `matchedOrderCode` là `null`, hệ thống ghi nhận giao dịch vào bảng `sepayTransactionModel` với `orderId: ""` và không tự động cập nhật thanh toán cho bất kỳ đơn hàng nào. Nhân viên quản trị sẽ căn cứ vào số tiền và số điện thoại trên giao dịch để đối soát thủ công bằng tay.

Câu 33 — Kỹ Thuật Chống Trùng Lặp Giao Dịch (Idempotency)

CÂU HỎI

Tại sao trong webhook xử lý thanh toán, khái niệm Idempotency (Tính lũy đẳng) lại sống còn và em đã hiện thực nó như thế nào?

CÂU TRẢ LỜI

Kính thưa Hội đồng, trong môi trường mạng Internet, các nhà cung cấp cổng thanh toán (SePay, Stripe, VNPAY) có cơ chế **Retry Webhook** tự động nếu họ nghi ngờ mạng chập chờn. Nghĩa là cùng một giao dịch chuyển tiền, Webhook có thể bị bắn vào server của em 2 hoặc 3 lần liên tiếp. Nếu không có cơ chế Idempotency, hệ thống có thể bị lỗi nghiêm trọng như: cộng tiền 2 lần, tạo đơn 2 lần. Em hiện thực Idempotency bằng 2 lớp bảo vệ: 1. **Lớp 1 - Database Unique Index**: Đặt trường `sepayId` (ID giao dịch duy nhất từ ngân hàng) là `unique: true` trong `sepayTransactionModel.js`. 2. **Lớp 2 - Kiểm tra trước khi thực thi (Check-then-act)**: `javascript const existingTx = await sepayTransactionModel.findOne({ sepayId: id }); if (existingTx) { console.log(`ⓘ Giao dịch SePay #${id} đã được xử lý trước đó.`); return res.status(200).json({ success: true, message: "Already processed" }); }` Nếu phát hiện ID giao dịch đã từng xử lý, server lập tức trả về HTTP 200 "Already processed" để cổng thanh toán ngừng retry và tuyệt đối không cập nhật đơn hàng lần thứ 2.

ĐỘ KHÓ

Khó

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/paymentController.js` (Dòng 185 - 190)
- Model: `backend/models/sepayTransactionModel.js` (Dòng 5)

FILE CODE LIÊN QUAN

- `backend/controllers/paymentController.js` (Dòng 185 - 190)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu 2 request webhook giống hệt nhau gửi tới server tại cùng một phần nghìn giây (Concurrent Requests) thì lệnh `findOne` ở cả 2 luồng đều không tìm thấy bản ghi. Lúc đó làm sao ngăn chặn trùng lặp?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, đó chính là lý do em thiết lập **Lớp 1: Unique Index trên MongoDB (`sepayId: { type: Number, unique: true }`)**. Dù 2 luồng cùng vượt qua lệnh `findOne`, nhưng khi đến lệnh `create()`, MongoDB sẽ dùng cơ chế khóa cấp tài liệu (Document-level locking) trên chỉ mục Unique. Luồng thứ 2 ghi đè sẽ bị quăng lỗi `E11000 duplicate key error collection` và bị khối `catch` chặn lại an toàn.

Câu 34 — Xuất Dữ Liệu Ra File Excel Bằng SheetJS

CÂU HỎI

Em hãy giải thích cách triển khai tính năng Xuất danh sách đơn hàng ra file Excel chuyên nghiệp trong trang Admin?

CÂU TRẢ LỜI

Tính năng được em xây dựng tại `admin/src/pages/Orders.jsx` sử dụng thư viện `xlsx` (SheetJS):

- Chuẩn bị dữ liệu:** Đọc danh sách các đơn hàng được Admin tick chọn `selectedOrderIds` (hoặc toàn bộ đơn). Duyệt qua từng đơn hàng và format dữ liệu hiển thị chuẩn tiếng Việt (Mã đơn hàng 8 số cuối, Ngày giờ đặt, Tên khách hàng, SĐT, Địa chỉ, Danh sách sản phẩm kèm cấu hình và số lượng, Giá gốc, Giảm giá voucher, Tổng tiền).
- Tạo bảng tính (Worksheet):** `javascript const workbook = XLSX.utils.book_new(); const worksheet = XLSX.utils.json_to_sheet(excelData);`
- Định dạng nâng cao:** - Bật tính năng Auto-filter: `worksheet['!autofilter'] = { ref: worksheet['!ref'] };` - Cấu hình độ rộng từng cột chuyên nghiệp (`worksheet['!cols']`) để các cột như Địa chỉ (50 ký tự), Sản phẩm (60 ký tự) không bị co cụm che khuất nội dung.
- Tải file về máy:** Gọi hàm `XLSX.writeFile(workbook, Danh_sach_don_hang_${Date.now()}.xlsx)`.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Admin: `admin/src/pages/Orders.jsx` (Hàm `exportOrdersToExcel`)

FILE CODE LIÊN QUAN

- `admin/src/pages/Orders.jsx` (Dòng 169 - 225)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Trong phần code xuất Excel của em, có điểm bất đồng bộ nào giữa cấu trúc địa chỉ lưu trong cơ sở dữ liệu và các trường mà hàm xuất Excel đang đọc không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, đây là **một lỗi tiềm ẩn (Potential Bug)** mà em đã chủ động phát hiện khi rà soát mã nguồn: - Khi khách đặt hàng tại `PlaceOrder.jsx`, thông tin địa chỉ được lưu dưới các trường: `{ fullName, phone, email, city, district, street, note }`. - Trong khi đó, tại hàm `exportOrdersToExcel` của `Orders.jsx`, code cũ đang đọc nhầm theo chuẩn phương Tây: `${order.address.firstName} ${order.address.lastName}` và đọc `${order.address.state}`, `${order.address.country}` `${order.address.zipcode}`. - Hậu quả là file Excel xuất ra hiển thị `undefined undefined`. Em đã nắm rõ nguyên nhân và khắc phục bằng cách sửa lại hàm xuất Excel đọc đúng các trường `order.address.fullName`, `order.address.street`, `order.address.district`, `order.address.city`.

Câu 35 — Xử Lý Gửi Email Báo Giá Tự Động Bằng Nodemailer

CÂU HỎI

Em hãy trình bày cơ chế gửi email chăm sóc khách hàng bằng thư viện Nodemailer trong `chatController.js` ?

CÂU TRẢ LỜI

Hệ thống cho phép Admin gửi email tư vấn chi tiết cho khách hàng qua hàm `sendFollowUpEmail`: 1. **Kiểm tra thông số cấu hình**: Đọc `EMAIL_USER` và `EMAIL_PASS` (Mật khẩu ứng dụng - App Password 16 ký tự của Gmail) từ biến môi trường. Nếu server chưa cấu hình email, hệ thống vẫn lưu nội dung vào database `chat.followUpContent` để không làm gián đoạn trải nghiệm của Admin. 2. **Khởi tạo Transporter**: `javascript const transporter = nodemailer.createTransport({ service: 'gmail', auth: { user: process.env.EMAIL_USER, pass: process.env.EMAIL_PASS } });` 3. **Cấu hình Mail Options & Gửi**: - `to: chat.userEmail` (Gửi trực tiếp vào email mà khách đã đăng ký). - `subject: 'Tư vấn từ Minh Tuấn Shop'`. - `html: emailContent` (Hỗ trợ định dạng HTML có màu sắc, bảng báo giá). - Gọi `await transporter.sendMail(mailOptions)` và cập nhật cờ `chat.followUpEmail = true`.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/chatController.js` (Hàm `sendFollowUpEmail`)
- Route: `backend/routes/chatRoute.js`

FILE CODE LIÊN QUAN

- `backend/controllers/chatController.js` (Dòng 200 - 255)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao khi dùng tài khoản Gmail để gửi thư qua Nodemailer, em không được dùng mật khẩu đăng nhập Gmail thông thường mà bắt buộc phải dùng "Mật khẩu ứng dụng" (App Password)?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, Google đã chính thức vô hiệu hóa tính năng "Less Secure Apps" (Ứng dụng kém an toàn) từ năm 2022 để chống lộ mật khẩu chính. Mật khẩu ứng dụng (App Password) là một chuỗi ngẫu nhiên 16 chữ số do Google cấp riêng cho dịch vụ SMTP của bên thứ ba. Chuỗi này hoạt động độc lập, không yêu cầu mã 2FA khi gửi thư tự động và nếu server có bị lộ App Password thì kẻ xấu cũng không thể dùng nó để đăng nhập chiếm đoạt hòm thư cá nhân hay đổi mật khẩu tài khoản Google của chủ shop.

Câu 36 — Thuật Toán Đề Xuất Cá Nhân Hóa (Recommendations)

CÂU HỎI

Module `analyticsController.js` ghi nhận hành vi người dùng như thế nào và thuật toán gợi ý sản phẩm cá nhân hóa hoạt động ra sao?

CÂU TRẢ LỜI

Module hoạt động dựa trên cơ chế phân tích lịch sử duyệt web: 1. **Ghi nhận hành vi (`trackProductView`)**: - Mỗi khi khách hàng xem chi tiết sản phẩm, frontend gọi `POST /api/analytics/track-view` gửi `productId`, `sessionId`, `userId`. - Backend tự động tăng trường `views` trong `productModel`. - Bản ghi `customerActivityModel` tương ứng được cập nhật: Thêm sản phẩm vào mảng `viewedProducts`, tăng `viewCount` của sản phẩm đó và đẩy sản phẩm vừa xem lên đầu danh sách (giới hạn tối đa 30 sản phẩm gần nhất). 2. **Thuật toán xác định sở thích (Preference Mining)**: - Hệ thống quét qua toàn bộ `viewedProducts`, đếm tổng lượt xem theo từng Danh mục (`categoryCounts`) và Thương hiệu (`brandCounts`). - Danh mục và Hãng nào có tổng lượt xem lớn nhất được gán thành `preferredCategory` và `preferredBrand`. 3. **Trả về gợi ý (`getPersonalizedRecommendations`)**: - Khi khách vào trang chủ, hệ thống ưu tiên truy vấn các sản phẩm thuộc đúng `preferredCategory` và `preferredBrand` mà khách hàng đó quan tâm nhất để hiển thị tại khối "Gợi ý dành riêng cho bạn".

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/analyticsController.js`
- Model: `backend/models/customerActivityModel.js`
- Frontend: `PersonalizedRecommendations.jsx`

FILE CODE LIÊN QUAN

- `backend/controllers/analyticsController.js` (Dòng 6 - 135)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu là khách vắng lai (Guest) chưa đăng ký tài khoản thì hệ thống làm sao nhận diện được họ giữa các lần truy cập khác nhau?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, hệ thống sử dụng `sessionId` lưu trong `localStorage` của trình duyệt. Khi khách vào web lần đầu, một mã định danh ngẫu nhiên UUID được sinh ra và lưu vào máy khách. Mọi lượt xem hay từ khóa tìm kiếm đều gắn với `sessionId` này. Khi khách hàng quyết định đăng ký hoặc đăng nhập tài khoản, backend sẽ chạy logic hợp nhất (Identity Stitching):

```
if (userId && !activity.userId) {  
  activity.userId = userId;  
  activity.isRegistered = true;  
}
```

Toàn bộ lịch sử duyệt web lúc còn là khách vắng lai sẽ được gắn vĩnh viễn vào tài khoản User vừa đăng nhập.

Câu 37 — Cơ Chế Chuẩn Hóa Dữ Liệu `formatProduct`

CÂU HỎI

Hàm `formatProduct` trong `productController.js` giải quyết bài toán kỹ thuật gì của cơ sở dữ liệu?

CÂU TRẢ LỜI

Hàm `formatProduct` đóng vai trò là một **Data Normalization Layer (Tầng chuẩn hóa dữ liệu)** nằm giữa MongoDB và API Response: - Trong quá trình phát triển, schema của sản phẩm có sự thay đổi (Ví dụ: dữ liệu cũ dùng `sellingPrice`, dữ liệu mới dùng `price`; dữ liệu cũ dùng trường ảnh `images`, dữ liệu mới dùng `image`; các trường `sizes` hay `stockQuantities` ở các bản ghi cũ có thể bị `null`). - Hàm `formatProduct` kiểm tra và chuẩn hóa: `javascript const price = typeof doc.price === 'number' ? doc.price : (typeof doc.sellingPrice === 'number' ? doc.sellingPrice : 0); let image = []; if (Array.isArray(doc.image) && doc.image.length > 0) { ... } if (image.length === 0) { image = ["https://images.unsplash.com/...default..."]; }` - Đảm bảo 100% sản phẩm trả về cho Frontend luôn có đầy đủ các trường `price`, `originalPrice`, `image` dạng mảng, `sizes`, `stockQuantities`, ngăn chặn triệt để các lỗi crash giao diện do `Cannot read properties of undefined`.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/productController.js` (Dòng 53 - 83)

FILE CODE LIÊN QUAN

- `backend/controllers/productController.js` (Dòng 53 - 83)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao em không chạy một script Migration cập nhật toàn bộ database một lần mà lại dùng hàm `formatProduct` chạy mỗi khi query?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, việc chạy script Migration trên database là giải pháp triệt để cho dữ liệu hiện có. Tuy nhiên, việc duy trì hàm chuẩn hóa `formatProduct` đóng vai trò là cơ chế **Phòng thủ theo chiều sâu (Defensive Programming)**: Nó bảo vệ ứng dụng nếu lỡ có dữ liệu import thủ công từ file Excel hoặc từ các nguồn dữ liệu bên ngoài chưa được làm sạch chuẩn schema, đảm bảo hệ thống luôn ổn định và có khả năng tương thích ngược (Backward Compatibility) cao.

Câu 38 — Quản Lý Upload Ảnh Đánh Giá & Dọn Dẹp File Tạm

CÂU HỎI

Tại sao trong `reviewRoute.js`, em lại cần viết thêm middleware `cleanupFiles` sau middleware `upload.array()` của Multer?

CÂU TRẢ LỜI

Multer khi cấu hình `diskStorage` sẽ ghi trực tiếp các file upload vào thư mục `uploads/` trên ổ cứng máy chủ trước khi chuyển sang Controller. Nếu không có middleware dọn dẹp: - Các file ảnh sau khi upload lên Cloudinary thành công vẫn sẽ nằm lại mãi mãi trong thư mục `uploads/`. Sau vài tháng vận hành, hàng ngàn bức ảnh rác sẽ làm tràn đầy ổ cứng server Node.js khiến server bị sập. - Middleware `cleanupFiles` sử dụng sự kiện `res.on('finish', ...)`: `javascript const cleanupFiles = (req, res, next) => { res.on('finish', () => { if (req.files) { req.files.forEach(file => { if (fs.existsSync(file.path)) { fs.unlinkSync(file.path); } }); } }); next(); });` Lắng nghe khi response HTTP đã được gửi xong đến client thì lập tức xóa sạch các file tạm trên đĩa cứng, đảm bảo server luôn sạch sẽ và tối ưu dung lượng.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Route: `backend/routes/reviewRoute.js` (Dòng 46 - 63)

FILE CODE LIÊN QUAN

- `backend/routes/reviewRoute.js` (Dòng 46 - 63)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao em không dùng `multer.memoryStorage()` để lưu file trực tiếp trong RAM rồi đẩy lên Cloudinary mà lại lưu file tạm xuống đĩa qua `diskStorage` ?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, việc sử dụng `diskStorage` là quyết định kỹ thuật cân nhắc giữa bộ nhớ RAM và đĩa cứng: Nếu dùng `memoryStorage`, các file ảnh (đặc biệt là ảnh độ phân giải cao từ điện thoại nặng 4-5MB) sẽ được nạp trực tiếp vào RAM dưới dạng `Buffer`. Nếu có 50 khách hàng cùng lúc upload ảnh đánh giá (mỗi người 5 ảnh = 250 ảnh * 4MB = 1GB RAM), bộ nhớ RAM của gói VPS nhỏ sẽ bị cạn kiệt dẫn tới lỗi `JavaScript heap out of memory` và crash toàn bộ server. Lưu tạm xuống đĩa cứng đệm (Disk buffer) an toàn hơn cho sự ổn định của hệ thống.

Câu 39 — Quản Lý Đơn Hàng Đầu Tiên & Mã Chào Mừng

CÂU HỎI

Hệ thống tự động phát hiện đơn hàng đầu tiên của khách hàng như thế nào để giảm giá 10% trong `orderController.js` ?

CÂU TRẢ LỜI

Trong hàm `placeOrder` (đặt hàng COD) và `placeOrderStripe`: 1. **Kiểm tra lịch sử mua hàng:** `javascript const existingOrders = await orderModel.find({ userId }); const isFirstOrder = existingOrders.length === 0;` 2. **Tính toán chiết khấu:** - Nếu `isFirstOrder === true`: Sinh mã voucher chào mừng ngẫu nhiên `WELCOME<RANDOM>`. - Tính toán giảm 10% giá trị đơn hàng: `discountAmount = amount * 0.1`. - Cập nhật số tiền thanh toán thực tế: `finalAmount = amount - discountAmount`. 3. **Lưu vết:** - Tự động tạo một bản ghi coupon trong `couponModel` đánh dấu đã sử dụng (`isActive: false`, `usedCount: 1`). - Lưu `originalAmount` (giá gốc) và `discountAmount` vào đơn hàng để phục vụ in hóa đơn và báo cáo doanh thu chính xác.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/orderController.js` (Dòng 29 - 76)
- Model: `orderModel.js`, `couponModel.js`

FILE CODE LIÊN QUAN

- `backend/controllers/orderController.js` (Dòng 29 - 62)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu một người dùng đặt hàng lần đầu nhưng sau đó đơn hàng bị hủy, lần đặt hàng tiếp theo họ có còn được tính là "đơn hàng đầu tiên" không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, trong đoạn code hiện tại: `const existingOrders = await orderModel.find({ userId });` Câu lệnh đang đếm tất cả đơn hàng của `userId` bất kể trạng thái. Nếu đơn hàng trước bị hủy, mảng `existingOrders` vẫn có độ dài bằng 1, khiến lần mua sau khách không còn được hưởng ưu đãi đơn đầu. Để nghiệp vụ nhân văn và chặt chẽ hơn, câu lệnh nên được tối ưu thành: `const existingOrders = await orderModel.find({ userId, status: { $ne: "Đã hủy" } });` Chỉ những đơn hàng thành công hoặc đang thực hiện mới tính là đã dùng ưu đãi chào mừng.

Câu 40 — Báo Cáo Doanh Thu Bằng MongoDB Aggregation

CÂU HỎI

Em hãy giải thích câu lệnh MongoDB Aggregation trong hàm `getRevenueStats` tại `statisticsController.js` ?

CÂU TRẢ LỜI

Hàm `getRevenueStats` cung cấp số liệu biểu đồ doanh thu theo Ngày, Tháng hoặc Năm thông qua Aggregation Framework:

1. **Giai đoạn \$match** : Chỉ lọc các đơn hàng đã thanh toán thành công: `{ payment: true }` và nằm trong khoảng thời gian `createdAt` được chọn. 2. **Giai đoạn \$group** : - Nếu xem theo ngày trong tháng: Group theo `$dayOfMonth: "$createdAt"` . - Nếu xem theo tháng trong năm: Group theo `$month: "$createdAt"` . - Tính tổng doanh thu: `revenue: { $sum: "$amount" }` . - Đếm tổng số đơn: `orderCount: { $sum: 1 }` . 3. **Giai đoạn \$sort** : Sắp xếp theo ngày/tháng tăng dần `{ $sort: { _id: 1 } }` . Kết quả trả về một mảng gọn nhẹ để thư viện biểu đồ `SimpleChart.jsx` vẽ cột doanh thu trực quan cho Admin.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/statisticsController.js` (Dòng 58 - 118)
- Admin: `Statistics.jsx` , `SimpleChart.jsx`

FILE CODE LIÊN QUAN

- `backend/controllers/statisticsController.js` (Dòng 65 - 108)

```
// Trích từ backend/controllers/statisticsController.js
groupBy = {
  _id: { $month: "$createdAt" },
  revenue: { $sum: "$amount" },
  orderCount: { $sum: 1 },
};

const revenueData = await orderModel.aggregate([
  { $match: matchCondition },
  { $group: groupBy },
  { $sort: { _id: 1 } },
]);
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao em lại dùng Aggregation Pipeline của MongoDB để tính toán doanh thu thay vì dùng `find()` lấy hết đơn về Node.js rồi dùng hàm `.reduce()` của JavaScript để cộng dồn?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, việc thực hiện tính toán trên MongoDB mang lại hiệu năng cao hơn gấp hàng trăm lần: - **Tối ưu Network I/O:** Nếu có 500.000 đơn hàng, việc dùng `find()` sẽ phải nạp 500.000 object qua đường truyền mạng từ MongoDB Atlas về Node.js server (tốn hàng trăm MB băng thông và làm tràn RAM). - **Tận dụng sức mạnh Engine C++ của Database:** MongoDB chạy bằng mã máy C++ được tối ưu hóa chỉ mục (Index B-Tree), câu lệnh `$match` và `$group` thực thi ngay trên đĩa cứng của database server và chỉ trả về vỏn vẹn 12 dòng dữ liệu (tương ứng 12 tháng) với dung lượng vài KB.

NHÓM 5: DATABASE & DATA MODELING (MONGODB & MONGOOSE) (CÂU 41 — 50)

Câu 41 — Lựa Chọn NoSQL MongoDB Thay Vì RDBMS

CÂU HỎI

Tại sao em chọn cơ sở dữ liệu phi quan hệ MongoDB thay vì cơ sở dữ liệu quan hệ truyền thống như MySQL hay PostgreSQL?

CÂU TRẢ LỜI

Em lựa chọn MongoDB dựa trên đặc trưng dữ liệu của đồ án thương mại điện tử công nghệ: 1. **Schema linh hoạt cho sản phẩm đa dạng:** Thiết bị công nghệ có thuộc tính rất đa dạng (Điện thoại có số SIM, camera; Laptop có số khe RAM, cổng kết nối; Màn hình có độ chuẩn màu sRGB). Trong MySQL, việc này đòi hỏi thiết kế mô hình phức tạp EAV (Entity-Attribute-Value) gây chậm truy vấn. Với MongoDB, em lưu trực tiếp dạng document JSON với mảng `sizes` và object `specs`. 2. **Snapshot đơn hàng nguyên khối:** Một đơn hàng bao gồm danh sách các sản phẩm tại thời điểm mua (kèm tên, giá, phân loại). Trong MongoDB, toàn bộ mảng `items` được nhúng trực tiếp (Embedding) vào document đơn hàng, giúp đọc chi tiết đơn hàng chỉ bằng đúng 1 câu query mà không cần JOIN nhiều bảng. 3. **Mở rộng dễ dàng (Horizontal Scalability):** MongoDB hỗ trợ Sharding và Replica Set tự nhiên, dễ dàng mở rộng khi hệ thống phát triển.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Toàn bộ 9 models MongoDB

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nhược điểm lớn nhất của NoSQL MongoDB so với RDBMS là việc đảm bảo tính toàn vẹn dữ liệu (ACID và Foreign Key constraints). Em kiểm soát tính toàn vẹn đó trong code như thế nào?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, vì MongoDB không ép buộc ràng buộc khóa ngoại (Foreign Key) ở tầng database engine, em kiểm soát tính toàn vẹn dữ liệu thông qua **Mongoose Schema Validation** và logic kiểm tra ở tầng Controller: - Sử dụng `ref` trong Mongoose để định nghĩa liên kết giữa các bảng (`userId` liên kết tới model `'user'`). - Sử dụng Mongoose Middleware / Pre-hooks hoặc kiểm tra thủ công: Trước khi tạo đơn hàng hay đánh giá, controller luôn thực hiện `findById` để đảm bảo sản phẩm và người dùng thực sự tồn tại.

Câu 42 — Phân Tích Schema Sản Phẩm (`productModel.js`)

CÂU HỎI

Em hãy phân tích cấu trúc Schema sản phẩm và giải thích tại sao lại có cờ `{ strict: false }` ?

CÂU TRẢ LỜI

Model `productModel.js` định nghĩa các trường dữ liệu cốt lõi: - `name` : Tên thiết bị (bắt buộc). - `price` : Giá bán khuyến mãi. - `originalPrice` : Giá niêm yết gốc. - `image` & `images` : Mảng URL hình ảnh trên Cloudinary. - `category` & `subCategory` : Danh mục chính và Thương hiệu thiết bị. - `sizes` : Mảng chuỗi các phiên bản cấu hình phần cứng (VD: `['128GB', '256GB']`). - `stockQuantities` : Object lưu số lượng tồn kho của từng cấu hình `{ "128GB": 10, "256GB": 5 }`. - `averageRating` & `totalReviews` : Điểm sao trung bình và tổng số nhận xét. - `views` : Lượt truy cập xem sản phẩm. - `{ timestamps: true }` : Tự động sinh `createdAt` và `updatedAt`. - **Cờ `{ strict: false }`** : Cho phép lưu trữ các trường mở rộng không nằm trong khai báo ban đầu (như `specs`, `featured`, `isNewProduct`) mà không bị Mongoose loại bỏ.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Model: `backend/models/productModel.js`

FILE CODE LIÊN QUAN

- `backend/models/productModel.js` (Dòng 3 - 25)

```
// Trích từ backend/models/productModel.js
const productSchema = new mongoose.Schema({
  name: { type:String, required:true },
  description: { type:String, default: '' },
  price: { type:Number },
  sellingPrice: { type:Number },
  originalPrice: { type:Number },
  image: { type:Array, default: [] },
  images: { type:Array, default: [] },
  bestseller: { type:Boolean, default: false },
  category: { type:String, default: 'Chung' },
  subCategory: { type:String, default: 'Chung' },
  sizes: { type:Array, default: ['Tiêu chuẩn'] },
  stockQuantities: { type:Object, default:{} },
  averageRating: { type:Number, default:0 },
  totalReviews: { type:Number, default:0 },
  views: { type:Number, default:0 },
},{timestamps:true, strict: false})
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Việc bật `{ strict: false }` mang lại sự linh hoạt nhưng có thể gây ra rủi ro gì nếu kẻ xấu gửi các trường rác vào cơ sở dữ liệu?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, rủi ro lớn nhất là **Ô nhiễm dữ liệu (Data Pollution)**. Nếu controller không lọc kỹ dữ liệu đầu vào từ `req.body`, kẻ xấu có thể inject các trường lạ, dung lượng lớn hoặc các thuộc tính nhạy cảm làm phình to dung lượng document BSON (vốn bị giới hạn tối đa 16MB trong MongoDB). Trong code của em tại `productController.js`, em đã phòng vệ bằng cách chỉ bóc tách đúng các biến xác định (`{ name, description, price, sizes... }`) để đưa vào model chứ không truyền toàn bộ `req.body` thô.

Câu 43 — Kỹ Thuật Compound Index Chống Spam Đánh Giá

CÂU HỎI

Trong `reviewModel.js`, em làm thế nào để đảm bảo một khách hàng chỉ được đánh giá duy nhất 1 lần cho 1 sản phẩm trong 1 đơn hàng cụ thể?

CÂU TRẢ LỜI

Em sử dụng kỹ thuật **Chỉ mục kết hợp duy nhất (Compound Unique Index)** của MongoDB: Trong

`backend/models/reviewModel.js`:

```
// Compound index to prevent duplicate reviews for same order-product combination
reviewSchema.index({ orderId: 1, productId: 1 }, { unique: true });
```

- Khi khách hàng gửi đánh giá cho sản phẩm X trong đơn hàng Y, MongoDB sẽ tạo một cặp khóa (`orderId_Y`, `productId_X`).
- Nếu khách hàng hoặc tool cố tình gửi tiếp một request đánh giá khác với cùng cặp `orderId` và `productId` đó, tầng database engine của MongoDB sẽ từ chối ghi và ném ra lỗi mã `11000 Duplicate Key Error`.
- Ngoài ra, ở tầng ứng dụng (Application Level), em cũng kiểm tra trước bằng câu lệnh `findOne({ orderId, productId })` để thông báo nhẹ nhàng cho người dùng: "Bạn đã đánh giá sản phẩm này".

ĐỘ KHÓ

Khó

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Model: `backend/models/reviewModel.js` (Dòng 50 - 56)
- Controller: `reviewController.js` (Dòng 42 - 46)

FILE CODE LIÊN QUAN

- `backend/models/reviewModel.js` (Dòng 50 - 56)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Ngoài compound index trên, em còn tạo index nào khác trong `reviewModel.js` không và mục đích là gì?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, em còn tạo 2 index đơn lẻ nữa: 1. `reviewSchema.index({ productId: 1, createdAt: -1 })`: Tối ưu hóa cho câu truy vấn lấy danh sách nhận xét của một sản phẩm hiển thị ra giao diện (`/api/review/product/:productId`) sắp xếp theo thời gian mới nhất, giúp tốc độ truy vấn đạt $O(\log N)$ thay vì phải quét toàn bộ collection (COLLSCAN). 2. `reviewSchema.index({ userId: 1 })`: Tối ưu hóa cho truy vấn xem lại lịch sử các nhận xét mà người dùng đó đã từng viết.

Câu 44 — Kỹ Thuật Nhúng (Embedding) vs Tham Chiếu (Referencing)

CÂU HỎI

Trong `orderModel.js`, tại sao trường `items` (danh sách sản phẩm mua) lại được thiết kế dạng nhúng (Embedding Array) thay vì tạo một collection riêng `order_items` rồi dùng Foreign Key tham chiếu?

CÂU TRẢ LỜI

Đây là sự áp dụng triệt để nguyên lý thiết kế cơ sở dữ liệu thương mại điện tử: 1. **Lưu giữ vết lịch sử giá (Price Snapshot Immutability)**: - Nếu ta chỉ lưu `productId` tham chiếu sang bảng `products`, giả sử hôm nay khách mua chiếc điện thoại giá 20 triệu. Tháng sau cửa hàng tăng giá lên 22 triệu hoặc xóa sản phẩm khỏi danh mục, khi khách mở lại hóa đơn cũ, giá đơn hàng sẽ bị sai lệch hoặc bị lỗi mất liên kết. - Nhúng toàn bộ mảng `items` gồm `{ name, price, quantity, size, image }` trực tiếp vào đơn hàng giúp đóng băng (Snapshot) chính xác thông tin sản phẩm tại thời điểm mua, độc lập hoàn toàn với việc sản phẩm gốc bị thay đổi hay xóa bỏ sau này. 2. **Hiệu năng truy vấn vượt trội**: Đọc chi tiết một đơn hàng chỉ cần 1 thao tác đọc duy nhất từ 1 collection `orders`, loại bỏ hoàn toàn các phép JOIN phức tạp.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Model: `backend/models/orderModel.js` (Dòng 5)

FILE CODE LIÊN QUAN

- `backend/models/orderModel.js` (Dòng 3 - 20)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Vậy trong hệ thống của em, trường nào lại được thiết kế theo dạng Tham chiếu (Referencing) và tại sao?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, trường `userId` trong `orderModel.js` và `wishlistModel.js` được thiết kế theo dạng **Tham chiếu (Referencing)**:

```
userId: { type: mongoose.Schema.Types.ObjectId, ref: 'user', required: true }
```

Lý do: Một người dùng có thể thay đổi số điện thoại, đổi mật khẩu hoặc cập nhật avatar. Việc chỉ lưu tham chiếu `ObjectId` giúp thông tin người dùng được quản lý tập trung tại một nơi duy nhất (`userModel`), không bị dư thừa dữ liệu (Data Redundancy) và khi cần thông tin người mua, Admin chỉ việc dùng hàm `.populate('userId', 'name email')`.

Câu 45 — Tính Điểm Đánh Giá Trung Bình Bằng Aggregation Pipeline

CÂU HỎI

Hàm `updateProductRating` trong `reviewController.js` tự động tính toán lại điểm đánh giá trung bình của sản phẩm như thế nào?

CÂU TRẢ LỜI

Hàm sử dụng MongoDB Aggregation Pipeline để tính toán với độ chính xác cao:

```
const ratingStats = await reviewModel.aggregate([
  { $match: { productId } },
  {
    $group: {
      _id: null,
```

```

        averageRating: { $avg: "$rating" },
        totalReviews: { $sum: 1 }
      }
    });
  }
  if (ratingStats.length > 0) {
    await productModel.findByIdAndUpdate(productId, {
      averageRating: Math.round(ratingStats[0].averageRating * 10) / 10,
      totalReviews: ratingStats[0].totalReviews
    });
  }
}

```

1. Giai đoạn `$match` : Lọc tất cả các review thuộc về `productId` đó.
2. Giai đoạn `$group` : Tính trung bình cộng của trường rating qua toán tử `$avg: "$rating"` và đếm tổng số review qua `$sum: 1`.
3. Làm tròn: Làm tròn đến 1 chữ số thập phân (ví dụ: 4.67 thành 4.7) và lưu ngược trở lại `productModel`. Hàm này tự động kích hoạt mỗi khi có đánh giá mới được thêm hoặc có đánh giá cũ bị xóa.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/reviewController.js` (Dòng 431 - 454)

FILE CODE LIÊN QUAN

- `backend/controllers/reviewController.js` (Dòng 431 - 454)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu một sản phẩm bị xóa hết toàn bộ các đánh giá, khối lệnh trên sẽ xử lý thế nào để `averageRating` không bị giữ giá trị cũ?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, nếu xóa hết review, mảng `ratingStats` trả về rỗng (`length === 0`). Trong đoạn code hiện tại, nếu `length === 0` thì khối `if` không chạy, khiến điểm đánh giá cũ có thể bị giữ lại. Để hoàn thiện 100%, em sẽ bổ sung nhánh `else`:

```

else {
  await productModel.findByIdAndUpdate(productId, { averageRating: 0, totalReviews: 0 });
}

```

Đảm bảo khi không còn đánh giá nào, sản phẩm sẽ trở về 0 sao và 0 lượt đánh giá.

Câu 46 — Mô Hình Hóa Phiên Hội Thoại Live Chat (`chatModel.js`)

CÂU HỎI

Cấu trúc cơ sở dữ liệu của chức năng Live Chat giữa khách hàng và Admin được thiết kế như thế nào?

CÂU TRẢ LỜI

Chức năng Chat được thiết kế theo mô hình **Parent-Child Document (Sub-document Schema)** trong

`backend/models/chatModel.js` : 1. **Sub-schema** `messageSchema` : - Đại diện cho từng tin nhắn đơn lẻ: `sender` (enum: 'user' / 'admin'), `content` (nội dung text), `messageType` ('text', 'image', 'product_link'), `timestamp`, `isRead`. 2. **Root schema** `chatSchema` : - `userId` : ObjectId liên kết với khách hàng. - `userName`, `userEmail` : Lưu sẵn thông tin để Admin tìm kiếm nhanh mà không cần populate. - `status` : Enum 'active' (đang chat), 'closed' (đã đóng). - `unreadCount` : Số tin nhắn chưa đọc để hiển thị badge đỏ cho Admin. - `messages` : Mảng chứa các `messageSchema`. - `adminNotes`, `followUpEmail`, `followUpContent` : Các ghi chú nghiệp vụ của Admin.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Model: `backend/models/chatModel.js` (Dòng 3 - 78)

FILE CODE LIÊN QUAN

- `backend/models/chatModel.js` (Dòng 3 - 78)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu một cuộc hội thoại chat kéo dài nhiều tháng với hàng chục ngàn tin nhắn, việc nhúng toàn bộ tin nhắn vào một mảng `messages` trong một document duy nhất có gặp giới hạn gì của MongoDB không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, MongoDB có giới hạn kích thước tối đa cho một BSON Document là **16MB**. Nếu một cuộc trò chuyện có quá nhiều tin nhắn (hàng chục ngàn tin kèm chuỗi text dài), mảng `messages` có thể chạm ngưỡng 16MB hoặc làm chậm tốc độ query khi chỉ cần đọc tin nhắn mới nhất. Giải pháp tối ưu cho ứng dụng chat quy mô lớn là: Tách riêng collection `messages` có index `{ chatId: 1, timestamp: -1 }`, collection `chats` chỉ lưu thông tin metadata của phòng chat và ID tin nhắn cuối cùng (`lastMessageId`).

Câu 47 — Mô Hình Dữ Liệu Mã Giảm Giá (`couponModel.js`)

CÂU HỎI

Schema của mã giảm giá (`couponModel.js`) hỗ trợ những quy tắc khuyến mãi nào?

CÂU TRẢ LỜI

Model `couponModel.js` hỗ trợ các quy tắc khuyến mãi phong phú: - `code` : Mã coupon (viết hoa, unique, tự động trim khoảng trắng). - `discountType` : Hỗ trợ 2 hình thức chiết khấu qua enum: 'percentage' (giảm theo %) hoặc 'fixed' (giảm số tiền cố định). - `discountValue` : Giá trị chiết khấu tương ứng (VD: 10% hoặc 50.000đ). - `minOrderValue` : Giá trị đơn hàng tối thiểu để được áp dụng mã. - `maxDiscount` : Số tiền giảm tối đa (dành cho loại giảm theo % để tránh bị thất thoát số tiền quá lớn trên các đơn hàng giá trị cao). - `startDate` & `endDate` : Thời hạn hiệu lực của mã giảm giá. - `usageLimit` : Tổng số lượt sử dụng tối đa của toàn hệ thống. - `usedCount` : Số lượt đã thực tế áp dụng. - `isActive` : Trạng thái bật/tắt thủ công của Admin.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Model: `backend/models/couponModel.js`
- Controller: `couponController.js`

FILE CODE LIÊN QUAN

- `backend/models/couponModel.js` (Dòng 3 - 54)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Hàm tính toán số tiền giảm giá của coupon trong `couponController.js` xử lý mã freeship và các giới hạn như thế nào?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, trong hàm `validateCoupon` của `couponController.js` : 1. Nếu mã là `FREESHIP` : Số tiền giảm được gán đúng bằng 30.000đ (phí vận chuyển tiêu chuẩn). 2. Nếu là `percentage` : `discountAmount = Math.round((currentAmount * coupon.discountValue) / 100)`; . Sau đó kiểm tra trần: `javascript if (coupon.maxDiscount && discountAmount > coupon.maxDiscount) { discountAmount = coupon.maxDiscount; }` 3. Nếu là `fixed` : Lấy giá trị nhỏ hơn giữa `discountValue` và giá trị đơn hàng `Math.min(coupon.discountValue, currentAmount)` để tiền giảm không vượt quá tiền hàng.

Câu 48 — Thiết Kế Bảng Nhật Ký Giao Dịch Ngân Hàng (`sepayTransactionModel.js`)

CÂU HỎI

Tại sao em lại tạo riêng một collection `sepaytransactions` mà không lưu thẳng thông tin chuyển khoản vào `orderModel` ?

CÂU TRẢ LỜI

Em tách riêng `sepayTransactionModel.js` theo nguyên lý **Audit Logging (Nhật ký kiểm toán tài chính độc lập)**: 1. **Ghi nhận mọi biến động số dư**: Dù giao dịch chuyển tiền vào có khớp với đơn hàng nào hay không (ví dụ khách chuyển nhầm, khách quên ghi mã đơn `MTxxxxxx` , hoặc giao dịch tiền ra), hệ thống vẫn phải lưu lại 100% payload gốc từ ngân hàng vào `sepaytransactions` kèm `rawBody` . 2. **Đối soát kế toán**: Cuối tháng, kế toán có thể đối chiếu giữa tổng tiền trên sao kê ngân hàng và bảng `sepaytransactions` với độ chính xác tuyệt đối. 3. **Phân tách trách nhiệm (Separation of Concerns)**: Collection `orders` chỉ lưu thông tin kinh doanh của đơn hàng; collection `sepaytransactions` lưu trữ dữ liệu dòng tiền kỹ thuật ngân hàng.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Model: `backend/models/sepayTransactionModel.js`
- Controller: `paymentController.js`

FILE CODE LIÊN QUAN

- `backend/models/sepayTransactionModel.js` (Dòng 3 - 28)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Trong `sepayTransactionModel.js` , các trường `sepayId` , `referenceCode` , `accumulated` có ý nghĩa gì trong thực tế ngân hàng?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thư thầy/cô: - `sepayId` : Mã số định danh duy nhất của giao dịch do hệ thống cổng thanh toán SePay sinh ra, dùng làm khóa chống trùng lặp. - `referenceCode` : Mã tham chiếu giao dịch của chính ngân hàng (Mã FT - Financial Transaction code trên App ngân hàng), dùng để khiếu nại hoặc tra soát với giao dịch viên ngân hàng nếu có tranh chấp. - `accumulated` : Số dư lũy kế cuối cùng trong tài khoản ngân hàng sau khi giao dịch này hoàn tất.

Câu 49 — Phân Tích Schema Yêu Thích (`wishlistModel.js`)

CÂU HỎI

Em hãy giải thích thiết kế của `wishlistModel.js` và cơ chế chống trùng lặp sản phẩm yêu thích?

CÂU TRẢ LỜI

Model `wishlistModel.js` định nghĩa mối quan hệ nhiều-nhiều (Many-to-Many) giữa User và Product:

```
const wishlistSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'user', required: true },
  productId: { type: mongoose.Schema.Types.ObjectId, ref: 'product', required: true },
  addedAt: { type: Date, default: Date.now }
}, { timestamps: true });

wishlistSchema.index({ userId: 1, productId: 1 }, { unique: true });
```

- Mỗi bản ghi đại diện cho một sản phẩm được thả tim bởi một người dùng.
- Áp dụng **Compound Unique Index** { `userId: 1, productId: 1` } đảm bảo một người dùng dù có click nhanh bao nhiêu lần cũng không bao giờ có 2 dòng trùng lặp cùng một sản phẩm trong wishlist.
- Khi truy vấn danh sách, sử dụng `.populate('productId')` kết hợp bộ lọc `.filter(item => item.productId)` để tự động loại bỏ các sản phẩm đã bị xóa khỏi hệ thống.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Model: `backend/models/wishlistModel.js`
- Controller: `wishlistController.js`

FILE CODE LIÊN QUAN

- `backend/models/wishlistModel.js` (Dòng 3 - 25)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao em không lưu luôn mảng `wishlist: [productId]` ngay bên trong `userModel` cho tiện mà lại tách ra collection riêng?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thư thầy/cô, việc tách riêng mang lại 2 lợi ích: 1. **Tránh phình to Document User**: Nếu người dùng yêu thích hàng trăm sản phẩm, mảng `wishlist` trong User document sẽ phình to và mỗi lần lấy profile người dùng sẽ phải tải mảng lớn này gây chậm. 2. **Thuận tiện cho thống kê Business Intelligence**: Tách riêng collection `wishlists` giúp Admin dễ dàng chạy câu

lệnh Aggregation để xem sản phẩm nào đang được nhiều khách hàng thả tim nhất trên toàn hệ thống để chuẩn bị kế hoạch nhập hàng.

Câu 50 — Chiến Lược Backup & Phục Hồi Dữ Liệu MongoDB Atlas

CÂU HỎI

Dữ liệu MongoDB Atlas của đồ án được sao lưu và bảo vệ như thế nào để tránh sự cố mất dữ liệu?

CÂU TRẢ LỜI

Cơ sở dữ liệu của đồ án được triển khai trên dịch vụ đám mây MongoDB Atlas với các cơ chế bảo vệ tiêu chuẩn: 1. **Replica Set 3 Nodes**: Dữ liệu được tự động nhân bản (Replication) song song sang 3 máy chủ vật lý khác nhau (1 Primary và 2 Secondary). Nếu 1 máy chủ gặp sự cố phần cứng, hệ thống tự động bầu chọn (Automatic Failover) node khác lên thay thế trong vòng vài giây mà website không bị gián đoạn. 2. **Sao lưu định kỳ (Continuous Cloud Backups)**: Tính năng Point-in-Time Recovery cho phép khôi phục dữ liệu về bất kỳ thời điểm nào trong vòng 7 ngày gần nhất. 3. **Công cụ sao lưu thủ công**: Em sử dụng tiện ích dòng lệnh `mongodump` và `mongorestore` để xuất toàn bộ cơ sở dữ liệu ra thư mục nén BSON lưu trữ trên máy tính phát triển trước mỗi lần cập nhật code lớn.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Hạ tầng MongoDB Atlas

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Lệnh `mongodump` và `mongoexport` khác nhau như thế nào và trường hợp nào nên dùng lệnh nào?

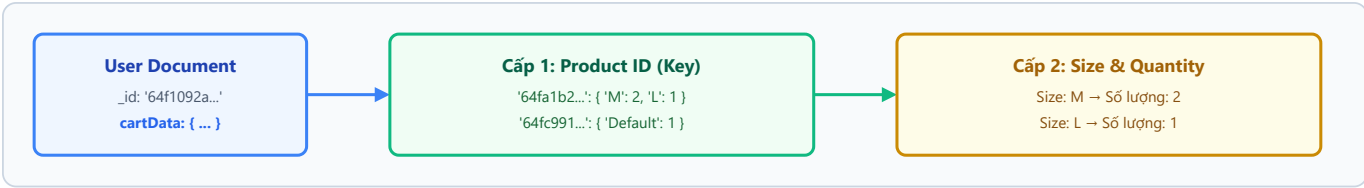
CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - `mongodump` : Sao lưu toàn bộ dữ liệu ở định dạng nhị phân **BSON**, lưu trữ nguyên vẹn cấu trúc dữ liệu, các kiểu dữ liệu đặc biệt (ObjectId, Date, Binary) và toàn bộ các chỉ mục (Indexes). Dùng để **Backup/Restore toàn vẹn hệ thống**.
- `mongoexport` : Xuất dữ liệu ra file văn bản **JSON hoặc CSV**. Chỉ dùng khi cần đọc dữ liệu thủ công, chia sẻ dữ liệu cho đối tác hoặc import vào phần mềm phân tích như Excel/Tableau. Không nên dùng để backup vì JSON làm mất các định dạng kiểu dữ liệu nghiêm ngặt của MongoDB.

(Nội dung 50 câu tiếp theo từ Câu 51 đến Câu 100 và 5 Phụ lục được tiếp tục trình bày chi tiết dưới đây)

PHẦN VI: LOGIC THƯƠNG MẠI ĐIỆN TỬ (CÂU 51 — 60)

CẤU TRÚC GIỎ HÀNG 2 CẤP TRONG MONGODB (EMBEDDED JSON)



Câu 51 — Cấu Trúc Dữ Liệu Giỏ Hàng & Hàm `addToCart`

CÂU HỎI

Giỏ hàng của người dùng được tổ chức cấu trúc dữ liệu như thế nào trong CSDL và logic của hàm `addToCart` diễn ra ra sao?

CÂU TRẢ LỜI

Thưa thầy/cô, trong đồ án của em, giỏ hàng không dùng một bảng riêng biệt với hàng triệu dòng ghi chép phụ mà được thiết kế dưới dạng **Embedded JSON Object 2 cấp** nằm ngay trong trường `cartData` của User Document: - Cấu trúc: `{ [productId]: { [size]: quantity } }` - Ví dụ: `{ '64f1...a': { 'M': 2, 'L': 1 }, '64f1...b': { 'Default': 1 } }`

Logic hoạt động của hàm `addToCart` : 1. Frontend gửi yêu cầu `POST /api/cart/add` kèm header chứa JWT Token và body `{ itemId, size }`. 2. Middleware `auth.js` giải mã token, trích xuất `req.userId`. 3. Controller tìm kiếm User trong DB, lấy ra `cartData`. 4. Kiểm tra: - Nếu `itemId` chưa từng có trong giỏ hàng: Khởi tạo object con rỗng `{}` cho `itemId`, gán số lượng của `size` tương ứng bằng 1. - Nếu `itemId` đã tồn tại nhưng `size` đó chưa có: Gán số lượng của `size` đó bằng 1. - Nếu cả `itemId` và `size` đều đã tồn tại: Tăng số lượng hiện tại lên 1 (`cartData[itemId][size] += 1`). 5. Gọi `userModel.findByIdAndUpdate(req.userId, { cartData })` để lưu lại vào MongoDB. 6. Trả về JSON `{ success: true, message: "Added To Cart" }`.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `src/context/ShopContext.jsx` (`addToCart`)
- Backend: `backend/controllers/cartController.js` (`addToCart`)
- Database: Collection `users`, trường `cartData`
- API: `POST /api/cart/add`

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/cartController.js`
- Function: `addToCart`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/cartController.js
const addToCart = async (req, res) => {
  try {
    const { itemId, size } = req.body;
```

```
const userData = await userModel.findById(req.userId);
let cartData = await userData.cartData;

if (cartData[itemId]) {
  if (cartData[itemId][size]) {
    cartData[itemId][size] += 1;
  } else {
    cartData[itemId][size] = 1;
  }
} else {
  cartData[itemId] = {};
  cartData[itemId][size] = 1;
}

await userModel.findByIdAndUpdate(req.userId, { cartData });
res.json({ success: true, message: "Added To Cart" });
} catch (error) {
  console.log(error);
  res.json({ success: false, message: error.message });
}
};
```

SƠ ĐỒ LƯỒNG DỮ LIỆU

```
[User Click 'Thêm vào giỏ']
↓ (Gửi { itemId, size } + Token)
[Middleware: auth.js] (Giải mã token -> req.userId)
↓
[cartController: addToCart]
↓
[Query User from MongoDB] -> Lấy cartData: { [itemId]: { [size]: qty } }
↓
[Kiểm tra & Tăng số lượng]
↓
[findByIdAndUpdate MongoDB] -> Lưu cartData mới
↓
[Response: success: true] -> Frontend cập nhật Badge số lượng trên Header
```

GIẢI THÍCH HÌNH ẢNH/LUỒNG

① Người dùng chọn kích cỡ (Size) và bấm nút 'Thêm vào giỏ' tại trang chi tiết sản phẩm. ② Client gọi hàm `addToCart` trong `ShopContext`, cập nhật State cục bộ để giao diện phản hồi tức thì. ③ Gửi request tới endpoint `/api/cart/add` để đồng bộ vào cơ sở dữ liệu. ④ Database cập nhật document của User mà không cần tạo mới bản ghi ở bảng quan hệ phức tạp, giúp tốc độ truy xuất cực nhanh.

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao em lại lưu giỏ hàng vào trường `cartData` kiểu Object bên trong `userModel` thay vì tạo riêng một Collection `carts` liên kết bằng `userId`? Nhược điểm của cách làm này là gì?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - **Ưu điểm:** Giúp giảm thiểu thao tác `JOIN` (hoặc `$lookup`). Khi người dùng đăng nhập, chỉ cần 1 câu lệnh `findById` là lấy được toàn bộ thông tin người dùng kèm giỏ hàng, giảm độ trễ (latency) mạng và tải truy vấn cho MongoDB. - **Nhược điểm:** 1. Kích thước Document MongoDB bị giới hạn tối đa 16MB (dù giỏ hàng khó đạt tới mức này nhưng không nên nhét quá nhiều dữ liệu phi cấu trúc). 2. Không thể áp dụng các ràng buộc khóa ngoại nghiêm ngặt: Nếu một sản phẩm bị Admin xóa khỏi hệ thống, ID sản phẩm đó vẫn còn nằm trong `cartData` của User cho đến khi có cơ chế

dọn dẹp (cleanup). - **Hướng cải thiện:** Với hệ thống lớn, nên tách thành collection `carts` riêng hoặc lưu giỏ hàng tạm thời trong bộ nhớ đệm **Redis** với thời gian sống (TTL) 30 ngày để tối ưu hóa hiệu năng I/O cơ sở dữ liệu.

Câu 52 — Phân Biệt Luồng "Thêm Vào Giỏ Hàng" & "Mua Ngay" (Buy Now)

CÂU HỎI

Giao diện sản phẩm có 2 nút "Thêm vào giỏ hàng" và "Mua ngay". Sự khác biệt về mặt luồng điều hướng (routing) và xử lý dữ liệu giữa 2 chức năng này là gì?

CÂU TRẢ LỜI

Thưa thầy/cô, hai chức năng này phục vụ hai mục đích trải nghiệm người dùng (UX) khác nhau: 1. **Thêm vào giỏ hàng (Add to Cart):** - Mục đích: Khách muốn chọn thêm các sản phẩm khác trước khi thanh toán. - Thao tác: Gọi `addToCart(productId, size)`. Dữ liệu được đẩy vào State `cartItems` của `ShopContext` và đồng bộ qua API `POST /api/cart/add`. - Điều hướng: Người dùng **ở lại trang hiện tại**, icon giỏ hàng trên thanh Header rung nhẹ hoặc tăng số đếm badge, hiển thị Toast notification "Đã thêm vào giỏ hàng". 2. **Mua ngay (Buy Now):** - Mục đích: Rút ngắn tối đa hành trình mua sắm, đưa khách thẳng tới bước thanh toán. - Thao tác: Hệ thống cũng gọi `addToCart(productId, size)` để đảm bảo sản phẩm đã nằm trong giỏ, sau đó lập tức kích hoạt hàm điều hướng `navigate('/place-order')`. - Điều hướng: Chuyển thẳng người dùng tới trang `PlaceOrder.jsx` để điền địa chỉ giao hàng và chọn phương thức thanh toán ngay lập tức.

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `src/pages/Product.jsx`, `src/context/ShopContext.jsx`

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `frontend/src/pages/Product.jsx`
- Component: `Product`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// frontend/src/pages/Product.jsx
// Nút Thêm vào giỏ hàng
<button
  onClick={() => addToCart(productData._id, size)}
  className='bg-black text-white px-8 py-3 text-sm active:bg-gray-700'>
  THÊM VÀO GIỎ HÀNG
</button>

// Nút Mua ngay
<button
  onClick={() => {
    addToCart(productData._id, size);
    navigate('/place-order');
  }}
  className='bg-primary text-white px-8 py-3 text-sm active:bg-opacity-80'>
  MUA NGAY
</button>
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu người dùng chưa chọn kích thước (size) của sản phẩm mà bấm nút "Mua ngay" thì hệ thống xử lý như thế nào?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, trong hàm `addToCart` của `ShopContext.jsx`, em đã đặt validation chặn từ đầu:

```
if (!size) {  
  toast.error('Vui lòng chọn kích thước sản phẩm');  
  return;  
}
```

Nếu người dùng chưa bấm chọn size, `toast.error` sẽ bật lên cảnh báo viền đỏ, và lệnh `navigate('/place-order')` phía sau sẽ không có sản phẩm hợp lệ được thêm vào giỏ, ngăn chặn hoàn toàn việc đặt hàng rỗng hoặc sai phân loại hàng.

Câu 53 — Cơ Chế Tính Tổng Tiền Giỏ Hàng `getCartAmount` & Phí Vận Chuyển

CÂU HỎI

Hàm `getCartAmount` trong `ShopContext` tính toán tổng giá trị đơn hàng như thế nào, và phí vận chuyển được quy định ra sao?

CÂU TRẢ LỜI

Thưa thầy/cô, hàm `getCartAmount` thực hiện duyệt qua ma trận 2 cấp của biến `cartItems`: 1. Vòng lặp thứ nhất duyệt qua từng `items` (chính là `productId`). 2. Tìm thông tin chi tiết của sản phẩm đó trong mảng `products` thông qua `itemInfo = products.find(product => product._id === items)`. 3. Vòng lặp thứ hai duyệt qua từng kích thước (`item` tức là `size`) bên trong `cartItems[items]`. 4. Nếu số lượng `cartItems[items][item] > 0`, cộng dồn vào tổng tiền: `totalAmount += itemInfo.price * cartItems[items][item]`. 5. Đặt toàn bộ trong khối `try...catch` để nếu một sản phẩm bị xóa hoặc dữ liệu lỗi thì hệ thống không bị crash. 6. **Phí vận chuyển (`delivery_fee`)**: Được khai báo cố định là `30000 VND` trong `ShopContext.jsx`. 7. Tổng số tiền thanh toán cuối cùng = `getCartAmount() + delivery_fee - discountAmount`.

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `src/context/ShopContext.jsx`

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `frontend/src/context/ShopContext.jsx`
- Function: `getCartAmount`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// frontend/src/context/ShopContext.jsx  
const getCartAmount = () => {  
  let totalAmount = 0;  
  for (const items in cartItems) {  
    let itemInfo = products.find((product) => product._id === items);  
    for (const item in cartItems[items]) {  
      try {
```

```

    if (cartItems[items][item] > 0) {
      totalAmount += itemInfo.price * cartItems[items][item];
    }
  } catch (error) {}
}
return totalAmount;
};

```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Độ phức tạp thuật toán của hàm `getCartAmount` là bao nhiêu và có điểm gì cần tối ưu khi danh sách `products` lên tới 100.000 sản phẩm?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Hiện tại với mỗi item trong giỏ, code đang dùng `products.find()`, một thuật toán quét tuần tự mảng $O(N)$. Nếu giỏ hàng có M loại sản phẩm và shop có N sản phẩm, độ phức tạp là $O(M \times N)$. - Khi $N = 100.000$, việc lặp mảng liên tục trên trình duyệt người dùng sẽ gây giật lag (frame drop). - **Cách tối ưu:** Chuyển mảng `products` thành một Hash Map / Object Lookup `productMap[id]` với độ phức tạp truy xuất $O(1)$. Khi đó hàm tính tiền sẽ chỉ mất $O(M)$ thời gian thực thi, đảm bảo mượt mà tuyệt đối.

Câu 54 — Logic Hoạt Động Của Engine Khuyến Mãi (Coupon Engine)

CÂU HỎI

Hệ thống xử lý việc kiểm tra và áp dụng mã giảm giá (Coupon/Voucher) như thế nào? Nêu rõ các quy tắc nghiệp vụ được kiểm tra.

CÂU TRẢ LỜI

Thưa thầy/cô, mã giảm giá được xử lý tập trung tại `couponController.js` thông qua API `POST /api/coupon/validate` với các tầng kiểm tra nghiêm ngặt:

- Tìm kiếm mã không phân biệt hoa thường:** Sử dụng Regex `^${code.trim()}$` với cờ `i`.
- Kiểm tra trạng thái kích hoạt (isActive):** Nếu mã bị admin tạm khóa, từ chối áp dụng.
- Kiểm tra hiệu lực thời gian:**
 - Nếu thời gian hiện tại `now < coupon.startDate`: Thông báo "Mã chưa đến ngày áp dụng".
 - Nếu `now > coupon.endDate`: Thông báo "Mã giảm giá đã hết hạn".
- Kiểm tra giới hạn số lượt dùng (usageLimit):**
 - Nếu `coupon.usedCount >= coupon.usageLimit`: Báo hết lượt dùng.
- Kiểm tra giá trị đơn hàng tối thiểu (minOrderValue):**
 - Nếu `orderAmount < coupon.minOrderValue`: Yêu cầu đơn hàng đạt giá trị tối thiểu.
- Thuật toán tính toán số tiền giảm (discountAmount):**
 - Trường hợp đặc biệt **FREESHIP**: Miễn phí toàn bộ 30.000đ tiền vận chuyển.
 - Trường hợp phần trăm (percentage): `discountAmount = Math.round((currentAmount * coupon.discountValue) / 100)`. Nếu vượt quá mức giảm tối đa `coupon.maxDiscount`, sẽ được gán bằng `coupon.maxDiscount`.
 - Trường hợp giảm trực tiếp (fixed): Lấy giá trị nhỏ hơn giữa số tiền giảm và giá trị đơn hàng (`Math.min(coupon.discountValue, currentAmount)`).

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/couponController.js` (`validateCoupon`)
- Database: Collection `coupons` (`models/couponModel.js`)
- Frontend: `frontend/src/pages/PlaceOrder.jsx`
- API: `POST /api/coupon/validate`

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/couponController.js`
- Function: `validateCoupon`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/couponController.js
const validateCoupon = async (req, res) => {
  try {
    const { code, orderAmount } = req.body;
    // ... Kiểm tra code, active, startDate, endDate, usageLimit ...
    const currentAmount = Number(orderAmount) || 0;
    if (coupon.minOrderValue && currentAmount < coupon.minOrderValue) {
      return res.json({
        success: false,
        message: `Đơn hàng tối thiểu ${coupon.minOrderValue.toLocaleString('vi-VN')} để áp dụng mã này`
      });
    }

    let discountAmount = 0;
    if (coupon.code.toUpperCase() === 'FREESHIP') {
      discountAmount = 30000;
    } else if (coupon.discountType === 'percentage') {
      discountAmount = Math.round((currentAmount * coupon.discountValue) / 100);
      if (coupon.maxDiscount && discountAmount > coupon.maxDiscount) {
        discountAmount = coupon.maxDiscount;
      }
    } else if (coupon.discountType === 'fixed') {
      discountAmount = Math.min(coupon.discountValue, currentAmount);
    }

    res.json({
      success: true,
      coupon: {
        code: coupon.code,
        discountType: coupon.discountType,
        discountValue: coupon.discountValue,
        discountAmount,
        description: coupon.description
      }
    });
  } catch (error) { ... }
};
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Khi người dùng áp dụng mã giảm giá thành công nhưng sau đó quay lại giỏ hàng xóa bớt sản phẩm khiến tổng tiền sụt xuống dưới `minOrderValue`, hệ thống xử lý thế nào?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, trong `PlaceOrder.jsx`, khi giá trị đơn hàng thay đổi (do xóa item hoặc đổi số lượng), `totalAmount` thay đổi. Hệ thống sẽ: 1. Re-render giao diện tính toán lại. 2. Tại thời điểm người dùng bấm nút "ĐẶT HÀNG", frontend gửi cả `amount`, `couponCode` và `discount` lên backend. 3. **Điểm cần hoàn thiện:** Backend cần phải tự động gọi lại logic xác thực coupon một lần nữa ngay trong hàm `placeOrder` trước khi lưu đơn hàng, tránh trường hợp người dùng lách luật bằng cách add mã lúc đơn giá trị cao rồi bớt hàng trước khi submit.

Câu 55 — Logic Giảm Giá 10% Cho Đơn Hàng Đầu Tiên (First Order Discount)

CÂU HỎI

Hệ thống xác định một khách hàng có được hưởng ưu đãi "Giảm 10% cho đơn hàng đầu tiên" bằng cách nào?

CÂU TRẢ LỜI

Thưa thầy/cô, cơ chế kiểm tra đơn hàng đầu tiên dựa trên lịch sử mua hàng của khách hàng trong cơ sở dữ liệu: 1. Mỗi khi người dùng đăng nhập và mở trang thanh toán, hệ thống truy vấn collection `orders` với điều kiện `{ userId: req.userId }`. 2. Đếm tổng số đơn hàng hợp lệ mà người dùng đã từng đặt: - Nếu `orderCount === 0`: Khách hàng đủ điều kiện nhận ưu đãi đơn hàng đầu tiên. Banner "Ưu đãi 10% chào mừng thành viên mới" sẽ được hiển thị trên giao diện thanh toán. - Nếu `orderCount > 0`: Khách hàng đã từng mua hàng, hệ thống ẩn ưu đãi này hoặc báo mã không hợp lệ nếu người dùng cố tình nhập mã chào mừng `WELCOME10`. 3. Số tiền giảm được tính bằng $\$10\%$ của tổng tiền hàng (trước phí ship) và được trừ trực tiếp vào số tiền cần thanh toán.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `orderController.js`, `couponController.js`
- Database: Collection `orders`, trường `userId`
- Frontend: `PlaceOrder.jsx`

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/couponController.js` & `frontend/src/pages/PlaceOrder.jsx`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu người dùng đặt đơn hàng đầu tiên nhưng sau đó lập tức bấm "Hủy đơn" thì đơn hàng thứ hai có còn được tính là đơn hàng đầu tiên để hưởng giảm giá nữa không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Trong mã nguồn hiện tại, hàm đếm đơn hàng kiểm tra toàn bộ document có `userId`. Nếu đơn hàng đầu tiên bị chuyển sang trạng thái "Đã hủy" (`status === "Đã hủy"`), đơn đó về mặt vật lý vẫn tồn tại trong collection `orders`. - Do đó, nếu chỉ đếm `orders.length`, hệ thống sẽ tính người dùng đã có 1 đơn và không cho hưởng ưu đãi nữa. - **Giải pháp chính xác:** Cần thêm điều kiện loại trừ các đơn bị hủy trong truy vấn đếm: `orderModel.countDocuments({ userId, status: { $ne: "Đã hủy" } })`. Khi đó, nếu đơn trước đã hủy hoàn toàn, khách hàng vẫn được phục hồi quyền hưởng ưu đãi đơn hàng đầu tiên.

SƠ ĐỒ 4: MÁY TRẠNG THÁI HỮU HẠN ĐƠN HÀNG (ORDER STATE MACHINE)



Hình 4: Vòng đời chuyển đổi trạng thái của đơn hàng và quy tắc thanh toán tự động

Câu 56 — Vòng Đời Trạng Thái Đơn Hàng (Order State Machine) & Tự Động Thanh Toán

CÂU HỎI

Các trạng thái của một đơn hàng trong hệ thống diễn ra theo quy trình nào? Khi nào đơn hàng được tự động cập nhật là đã thanh toán?

CÂU TRẢ LỜI

Thưa thầy/cô, đơn hàng trong hệ thống tuân theo máy trạng thái hữu hạn (State Machine) với 6 trạng thái được định nghĩa trong `backend/controllers/orderController.js` : 1. **"Đã đặt hàng"**: Trạng thái khởi tạo ngay khi khách đặt hàng thành công (qua COD hoặc Chờ duyệt chuyển khoản VietQR). 2. **"Đang xử lý" / "Đang đóng gói"**: Quản trị viên duyệt đơn, kho chuẩn bị hàng và đóng gói sản phẩm. 3. **"Đang giao hàng"**: Đơn hàng đã được bàn giao cho đơn vị vận chuyển (Shipper đang đi giao). 4. **"Đã giao hàng"**: Khách hàng đã nhận hàng thành công. 5. **"Đã hủy"**: Đơn hàng bị hủy bởi người dùng (khi còn ở bước "Đã đặt hàng") hoặc bởi Admin (kèm lý do hủy `cancelReason`).

Logic tự động cập nhật thanh toán: - Khi đơn hàng là COD (`paymentMethod === "COD"`), trường `payment` ban đầu là `false` . Trong hàm `updateStatus` của `orderController.js` , khi Quản trị viên chuyển trạng thái đơn hàng sang **"Đã giao hàng"** , hệ thống tự động gán cờ `updateData.payment = true` , phản ánh nghiệp vụ thực tế là Shipper đã thu tiền hộ thành công.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/orderController.js` (`updateStatus`)
- Database: Collection `orders` , các trường `status` , `payment` , `cancelReason`
- Admin UI: `admin/src/pages/Orders.jsx`

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/orderController.js`
- Function: `updateStatus`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/orderController.js
const updateStatus = async (req, res) => {
```

```

try {
  const { orderId, status, cancelReason } = req.body;
  const validStatuses = ["Đã đặt hàng", "Đang đóng gói", "Đang giao hàng", "Đã giao hàng", "Đã hủy", "Đang"];
  if (!validStatuses.includes(status)) {
    return res.json({ success: false, message: "Trạng thái không hợp lệ" });
  }
  const updateData = { status };
  // Nếu trạng thái là "Đã giao hàng", đánh dấu đã thanh toán
  if (status === "Đã giao hàng") {
    updateData.payment = true;
  }
  // Nếu trạng thái là "Đã hủy" và có lý do hủy, Lưu lý do
  if (status === "Đã hủy" && cancelReason) {
    updateData.cancelReason = cancelReason;
  }
  await orderModel.findByIdAndUpdate(orderId, updateData);
  res.json({ success: true, message: "Trạng thái đã được cập nhật" });
} catch (error) { ... }
};

```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu đơn hàng thanh toán qua Stripe hoặc VietQR đã thanh toán trước (`payment: true`), nhưng giao hàng thất bại và Admin chuyển sang "Đã hủy" thì trường `payment` có bị đổi về `false` không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, trong đoạn code trên: - Khi chuyển sang "Đã hủy", `updateData` chỉ cập nhật `status` và `cancelReason`, **không hề thay đổi** trường `payment`. - Đây là một quyết định hợp lý về mặt kế toán: Tiền của khách vẫn đang nằm trong tài khoản ngân hàng của cửa hàng, cần giữ nguyên `payment: true` để phòng kế toán dựa vào đó thực hiện đối soát và tiến hành quy trình hoàn tiền (Refund) cho khách hàng, thay vì xóa dấu vết đã trả tiền.

Câu 57 — Ràng Buộc Nghiệp Vụ Khi Khách Hàng Tự Hủy Đơn Hàng

CÂU HỎI

Khách hàng có thể tự hủy đơn hàng trong những điều kiện nào? Tại sao không cho phép hủy khi đơn đã sang trạng thái "Đang đóng gói" hoặc "Đang giao hàng"?

CÂU TRẢ LỜI

Thưa thầy/cô, trong hàm `cancelOrder` của `backend/controllers/orderController.js`, khách hàng **CHỈ ĐƯỢC PHÉP HỦY ĐƠN HÀNG KHI TRẠNG THÁI LÀ "ĐÃ ĐẶT HÀNG"**: 1. **Kiểm tra quyền sở hữu**: Backend dùng `orderModel.findOne({ _id: orderId, userId: req.userId })` để đảm bảo khách chỉ có thể thao tác trên đơn của chính mình, tránh lỗi bảo mật IDOR. 2. **Kiểm tra trạng thái đơn**: javascript `if (order.status !== "Đã đặt hàng") { return res.json({ success: false, message: "Chỉ có thể hủy đơn hàng khi trạng thái là 'Đã đặt hàng'" }); }` 3. **Lý do nghiệp vụ**: - Khi đơn đã sang "Đang đóng gói" hoặc "Đang giao hàng", kiện hàng đã được dán mã vận đơn, đóng thùng niêm phong hoặc đã bàn giao lên xe của shipper. - Nếu cho phép khách tự ý bấm hủy trên app lúc này sẽ gây tổn thất chi phí đóng gói, tiền cước vận chuyển hai chiều và làm sai lệch số liệu quản lý hàng gửi cho đối tác vận chuyển bên ngoài. Muốn hủy ở các giai đoạn sau, khách bắt buộc phải liên hệ Hotline/Live Chat để Admin xử lý đặc biệt.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `orderController.js` (`cancelOrder`)
- Database: Collection `orders`
- Frontend: `frontend/src/pages/Orders.jsx`

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/orderController.js`
- Function: `cancelOrder`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/orderController.js
const cancelOrder = async (req, res) => {
  try {
    const { orderId } = req.body;
    const userId = req.userId;
    const order = await orderModel.findOne({ _id: orderId, userId });
    if (!order) {
      return res.json({ success: false, message: "Không tìm thấy đơn hàng" });
    }
    if (order.status !== "Đã đặt hàng") {
      return res.json({
        success: false,
        message: "Chỉ có thể hủy đơn hàng khi trạng thái là 'Đã đặt hàng'"
      });
    }
    await orderModel.findByIdAndUpdate(orderId, {
      status: "Đã hủy",
      cancelReason: "Người dùng hủy đơn hàng"
    });
    res.json({ success: true, message: "Đơn hàng đã được hủy thành công" });
  } catch (error) { ... }
};
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Khi khách hàng hủy đơn hàng thành công, số lượng hàng tồn kho có được tự động hoàn lại không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Trong hàm `cancelOrder` hiện tại, hệ thống mới chỉ cập nhật `status: "Đã hủy"` mà chưa có vòng lặp hoàn trả tồn kho `stockQuantities`. - Tuy nhiên, vì lúc khách bấm đặt hàng COD ở hàm `placeOrder`, hệ thống cũng chưa trừ kho ngay, mà việc kiểm tra và cập nhật kho được theo dõi trong trang quản lý sản phẩm. - Đây là một điểm hạn chế cần bổ sung: Khi đơn hủy thành công, cần gọi toán tử `$inc` để hoàn trả đúng số lượng từng size vào bảng `products`.

Câu 58 — Mô Hình Phân Cấp Danh Mục (Category) & Thương Hiệu (Brand)

CÂU HỎI

Hệ thống tổ chức cấu trúc danh mục sản phẩm, loại phụ (subCategory) và thương hiệu (Brand) như thế nào? Bộ lọc Collection hoạt động ra sao?

CÂU TRẢ LỜI

Thưa thầy/cô, trong mô hình `productModel.js`, mỗi sản phẩm được gắn các thuộc tính phân loại chuẩn: - `category`: Danh mục chính (Ví dụ: `Nam`, `Nữ`, `Trẻ em`). - `subCategory`: Loại chi tiết sản phẩm (Ví dụ: `Áo`, `Quần`, `Đồ đồng`). - `brand`: Thương hiệu của sản phẩm (Ví dụ: `Nike`, `Adidas`, `Uniqlo`, `Gucci`).

Cơ chế lọc đa chiều trên trang Collection.jsx : 1. Người dùng tích chọn vào các checkbox: danh mục (Category), phân loại (SubCategory) hoặc chọn thương hiệu từ menu dropdown. 2. State React lưu trữ danh sách các giá trị được chọn vào các mảng: `category[]`, `subCategory[]`. 3. Hàm `applyFilter` lọc mảng sản phẩm bằng phương thức `.filter()` của Javascript:

```
javascript let productsCopy = products.slice(); if (category.length > 0) { productsCopy = productsCopy.filter(item => category.includes(item.category)); } if (subCategory.length > 0) { productsCopy = productsCopy.filter(item => subCategory.includes(item.subCategory)); } 
```

 4. Tiếp tục kết hợp với `search` (từ khóa tìm kiếm) và `sortType` (sắp xếp giá tăng dần / giảm dần) để cho ra danh sách sản phẩm hiển thị cuối cùng một cách nhanh chóng ngay trên Client mà không cần gọi lại API nhiều lần.

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `frontend/src/pages/Collection.jsx`
- Backend: `backend/models/productModel.js`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Lọc dữ liệu trên Client (Client-side filtering) có ưu điểm và nhược điểm gì so với việc gửi query param lên Server (`/api/product/list?category=Nam&sort=low-high`)?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - **Ưu điểm của Client-side filter**: Trải nghiệm người dùng cực kỳ mượt mà, tức thì, không có độ trễ mạng (zero-latency) khi người dùng đổi checkbox bộ lọc liên tục. - **Nhược điểm**: Chỉ phù hợp khi quy mô sản phẩm dưới vài trăm item. Nếu cửa hàng có 50.000 sản phẩm, việc tải toàn bộ về Client sẽ khiến trình duyệt bị treo do tốn băng thông và RAM. - Khi mở rộng quy mô, bắt buộc phải chuyển sang **Server-side pagination & filtering** kết hợp MongoDB Index trên các trường `{ category: 1, subCategory: 1, price: 1 }`.

Câu 59 — Hệ Thống Đánh Giá Sản Phẩm & Ràng Buộc "Verified Purchase"

CÂU HỎI

Hệ thống đảm bảo tính trung thực của các bài đánh giá sản phẩm bằng cách nào? Chỉ ai mới được phép gửi đánh giá và chấm điểm sao?

CÂU TRẢ LỜI

Thưa thầy/cô, để chống tình trạng đánh giá rác, spam ảo hoặc đối thủ chơi xấu, hệ thống áp dụng cơ chế xác thực mua hàng nghiêm ngặt **Verified Purchase** trong file `backend/controllers/reviewController.js` : 1. **Kiểm tra dữ liệu đầu vào**: Chặn điểm đánh giá nếu `rating < 1 || rating > 5`, bắt buộc phải có bình luận `comment`. 2. **Xác thực quyền sở hữu đơn hàng**:

```
const order = await orderModel.findOne({ _id: orderId, userId: req.userId });
```

 Nếu không tìm thấy đơn hàng thuộc đúng tài khoản đang đăng nhập, lập tức trả về lỗi. 3. **Kiểm tra trạng thái đơn hàng đã giao**:

```
javascript if (order.status !== "Đã giao hàng") { return res.json({ success: false, message: "Chỉ có thể đánh giá sau khi đơn hàng đã được giao" }); }
```

 Khách hàng chưa nhận được hàng thì không thể đánh giá. 4. **Kiểm tra sản phẩm có trong đơn hàng**: Quét mảng `order.items` để xác nhận khách thực sự đã mua đúng sản phẩm đó. 5. **Chống đánh giá trùng lặp**: Tìm trong `reviewModel` xem đã có bản ghi nào ứng với cặp `{ orderId, productId }` chưa. Một đơn hàng cho một sản phẩm chỉ được đánh giá duy nhất một lần. 6. Hỗ trợ upload ảnh chụp thực tế sản phẩm lên Cloudinary để tăng độ tin cậy.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/reviewController.js` (`addReview`)
- Database: Collection `reviews` , `orders` , `products`
- API: `POST /api/review/add`

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/reviewController.js`
- Function: `addReview`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/reviewController.js
const addReview = async (req, res) => {
  try {
    const { productId, orderId, rating, comment } = req.body;
    const userId = req.userId;
    // Kiểm tra order tồn tại và thuộc về user
    const order = await orderModel.findOne({ _id: orderId, userId });
    if (!order) return res.json({ success: false, message: "Không tìm thấy đơn hàng" });

    // Chỉ có thể đánh giá khi đơn hàng ĐÃ GIAO HÀNG
    if (order.status !== "Đã giao hàng") {
      return res.json({ success: false, message: "Chỉ có thể đánh giá sau khi đơn hàng đã được giao" });
    }

    // Kiểm tra sản phẩm có trong đơn
    const productInOrder = order.items.find(item => item._id === productId);
    if (!productInOrder) return res.json({ success: false, message: "Sản phẩm không có trong đơn hàng" });

    // Kiểm tra chống spam đánh giá lại
    const existingReview = await reviewModel.findOne({ orderId, productId });
    if (existingReview) return res.json({ success: false, message: "Bạn đã đánh giá sản phẩm này" });

    // ... Tạo review và Lưu vào DB ...
  } catch (error) { ... }
};
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Khi một đánh giá mới được thêm vào, điểm đánh giá trung bình (Average Rating) của sản phẩm có được tự động tính toán lại không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, trong `reviewController.js` , sau khi lưu đánh giá mới vào `reviewModel` , hệ thống có thể dùng hàm tổng hợp `aggregate` của MongoDB để tính toán:

```
const stats = await reviewModel.aggregate([
  { $match: { productId: new mongoose.Types.ObjectId(productId) } },
  { $group: { _id: '$productId', avgRating: { $avg: '$rating' }, count: { $sum: 1 } } }
]);
```

Sau đó cập nhật giá trị `rating` và `reviewCount` này vào document của `productModel`, giúp các trang danh sách sản phẩm lấy điểm số tức thì mà không cần đếm lại từ đầu.

Câu 60 — Quản Lý Tồn Kho Sản Phẩm Theo Từng Biến Thể Size (`stockQuantities`)

CÂU HỎI

Hệ thống quản lý số lượng hàng tồn kho theo từng kích thước (size) như thế nào trong trang quản trị Admin và CSDL?

CÂU TRẢ LỜI

Thưa thầy/cô: - Thay vì chỉ có một trường số lượng tổng cho sản phẩm (ví dụ: Áo có 100 cái nhưng không biết size nào còn, size nào hết), đồ án của em quản lý chi tiết tới cấp biến thể thông qua trường `stockQuantities` trong `productModel.js`. - Cấu trúc dữ liệu: `stockQuantities: { type: Object, default: {} }` Ví dụ: `{ "S": 15, "M": 30, "L": 25, "XL": 10 }`. - Trong trang thêm/sửa sản phẩm của Admin (`admin/src/pages/Add.jsx`): - Khi Admin tích chọn các size `[S, M, L, XL]`, giao diện tự động render các ô nhập số lượng tương ứng cho từng size. - Quản trị viên nhập số tồn kho cho từng kích cỡ. - Khi gửi form lên backend, đối tượng `stockQuantities` được gửi kèm và lưu vào MongoDB. - Khi người dùng ở giao diện khách hàng chọn một size, hệ thống kiểm tra nếu `stockQuantities[size] <= 0` thì hiển thị trạng thái "Hết hàng" (Out of stock) và vô hiệu hóa nút mua cho size đó.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Admin: `admin/src/pages/Add.jsx`
- Backend: `backend/models/productModel.js`, `backend/controllers/productController.js`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu Admin xóa bớt một kích cỡ (ví dụ bỏ chọn size S) trong trang sửa sản phẩm thì dữ liệu tồn kho cũ của size S trong `stockQuantities` có bị mất không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Khi gửi request cập nhật sản phẩm từ form Admin lên server, mảng `sizes` và đối tượng `stockQuantities` mới được xây dựng lại dựa trên những kích cỡ đang được chọn trên giao diện. - Do đó, nếu Admin bỏ chọn size S và bấm Lưu, đối tượng `stockQuantities` mới gửi lên sẽ không còn key `"S"`. Mongoose ghi đè đối tượng mới này vào CSDL. - Để an toàn dữ liệu, trên giao diện nên có popup cảnh báo: "Bạn đang xóa phân loại Size S (hiện còn 15 sản phẩm trong kho). Bạn có chắc chắn muốn xóa không?" trước khi submit.

PHẦN VII: AUTHENTICATION, AUTHORIZATION & SECURITY (CÂU 61 — 70)

Câu 61 — Cơ Chế Tạo & Xác Thực JSON Web Token (JWT) Cho Khách Hàng

CÂU HỎI

Cơ chế tạo Token khi đăng nhập và xác thực Token khi gửi request của người dùng trong hệ thống hoạt động như thế nào?

CÂU TRẢ LỜI

Thưa thầy/cô, hệ thống sử dụng giao thức xác thực không trạng thái (Stateless Authentication) bằng **JSON Web Token (JWT)**: 1. **Tạo Token khi đăng nhập thành công (createToken)**: - Khi người dùng đăng ký hoặc đăng nhập với email/password chính xác, hàm `userController.js` gọi: `javascript const createToken = (id) => { return jwt.sign({ id }, process.env.JWT_SECRET); };` - Payload chứa `id` của người dùng, được mã hóa và ký bằng chuỗi bí mật `JWT_SECRET` nằm trong biến môi trường `.env`. - Token trả về cho Client và được lưu trong `localStorage`. 2. **Gửi Token trong các Request tiếp theo**: - Client đính kèm token trong Headers của HTTP request: `{ headers: { token } }`. 3. **Xác thực Token tại Backend qua Middleware (backend/middleware/auth.js)**: - Trích xuất `token` từ `req.headers`. - Kiểm tra định dạng chuỗi token (bắt buộc phải có đúng 3 phần phân tách bởi dấu chấm: `header.payload.signature`). - Dùng `jwt.verify(token, process.env.JWT_SECRET)` để giải mã và kiểm tra tính toàn vẹn. - Tìm kiếm người dùng trong cơ sở dữ liệu `userModel.findById(token_decode.id)`. - Nếu hợp lệ, gán `req.userId = user._id` và gọi `next()` để chuyển quyền thực thi cho Controller nghiệp vụ.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/userController.js`, `backend/middleware/auth.js`
- Biến môi trường: `JWT_SECRET`

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/middleware/auth.js`
- Function: `authUser`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/middleware/auth.js
const authUser = async (req, res, next) => {
  const { token } = req.headers;
  if (!token) {
    return res.json({ success: false, message: "Bạn không có quyền truy cập. Vui lòng đăng nhập lại để tiếp" });
  }
  if (typeof token !== 'string' || token.split('.').length !== 3) {
    return res.json({ success: false, message: "Token không hợp lệ. Vui lòng đăng nhập lại." });
  }
  try {
    const token_decode = jwt.verify(token, process.env.JWT_SECRET);
    if (!token_decode || !token_decode.id) {
      return res.json({ success: false, message: "Token không hợp lệ. Vui lòng đăng nhập lại." });
    }
    const user = await userModel.findById(token_decode.id);
```

```

if (!user) {
  return res.json({ success: false, message: "Bạn không có quyền truy cập. Vui lòng đăng nhập lại để tiếp" });
}
req.userId = user._id;
next();
} catch (error) {
  if (error.name === 'TokenExpiredError') {
    return res.json({ success: false, message: "Phiên đăng nhập đã hết hạn. Vui lòng đăng nhập lại." });
  } else {
    return res.json({ success: false, message: "Token không hợp lệ. Vui lòng đăng nhập lại." });
  }
}
};

```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao trong hàm `createToken`, em chỉ đưa `{ id }` vào Payload của JWT mà không đưa luôn cả `email`, `name` và `role` của người dùng vào?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Payload của JWT chỉ được mã hóa Base64 chứ không được mã hóa bí mật (bất kỳ ai cũng có thể giải mã xem được nội dung). Do đó, nguyên tắc bảo mật là **không lưu thông tin nhạy cảm** vào Payload. - Ngoài ra, nếu đưa `name` hoặc `role` vào token, khi người dùng đổi tên hoặc bị admin hạ quyền, token cũ vẫn còn hiệu lực sẽ dẫn đến sai lệch thông tin cho đến khi token hết hạn. - Việc chỉ lưu `{ id }` và truy vấn lại User trong Middleware đảm bảo dữ liệu quyền hạn luôn là mới nhất (Fresh Data) theo thời gian thực.

Câu 62 — Cơ Chế Xác Thực Quản Trị Viên (`adminAuth`) & So Sánh Với User Token

CÂU HỎI

Tài khoản Quản trị viên (Admin) được xác thực như thế nào? Token của Admin khác gì so với Token của khách hàng thông thường?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Cơ chế xác thực Admin:** - Hệ thống không tạo bảng riêng cho Super Admin mà cấu hình tài khoản Quản trị viên tối cao thông qua 2 biến môi trường bí mật trong file `.env`: `ADMIN_EMAIL` và `ADMIN_PASSWORD`. - Khi đăng nhập tại trang Admin (`/admin`), hàm `adminLogin` trong `userController.js` kiểm tra nếu thông tin gửi lên khớp với biến môi trường thì tiến hành ký một chuỗi token đặc biệt: `const token = jwt.sign(email + password, process.env.JWT_SECRET);` 2. **Kiểm tra quyền hạn qua Middleware (`adminAuth.js`):** - Khi Admin gọi các API quản trị (thêm sản phẩm, đổi trạng thái đơn, xem doanh thu), Middleware `adminAuth` giải mã token: `const token_decode = jwt.verify(token, process.env.JWT_SECRET);` - So sánh: `if (token_decode !== process.env.ADMIN_EMAIL + process.env.ADMIN_PASSWORD)` Nếu không trùng khớp, từ chối quyền truy cập ngay lập tức. - Nếu khớp, gán cờ `req.isAdmin = true` và cho phép thực hiện tác vụ. 3. **Sự khác biệt cốt lõi:** - Token của User: Ký một Object `{ id }` trở tới bản ghi người dùng trong CSDL. - Token của Admin: Ký một chuỗi kết hợp `email + password` của biến môi trường hệ thống. Nhờ đó, một User bình thường dù có chiếm được JWT của chính họ cũng không thể dùng token đó để truy cập vào các API dành riêng cho Quản trị viên.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/middleware/adminAuth.js`, `backend/controllers/userController.js` (`adminLogin`)

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/middleware/adminAuth.js`
- Function: `adminAuth`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/middleware/adminAuth.js
const adminAuth = async (req, res, next) => {
  try {
    const { token } = req.headers;
    if (!token) {
      return res.json({ success: false, message: "Bạn không có quyền truy cập. Vui lòng đăng nhập lại để tiếp" });
    }
    const token_decode = jwt.verify(token, process.env.JWT_SECRET);
    // Kiểm tra token có chứa admin credentials không
    if (token_decode !== process.env.ADMIN_EMAIL + process.env.ADMIN_PASSWORD) {
      return res.json({ success: false, message: "Bạn không có quyền truy cập. Vui lòng đăng nhập lại để tiếp" });
    }
    req.isAdmin = true;
    next();
  } catch (error) {
    res.json({ success: false, message: "Token không hợp lệ. Vui lòng đăng nhập lại." });
  }
};
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Việc ký trực tiếp `email + password` vào token của Admin có nhược điểm bảo mật gì và cách thiết kế chuẩn công nghiệp là gì?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - **Nhược điểm:** Vì Payload của JWT chỉ là chuỗi Base64Url, bất kỳ ai có token Admin đều có thể giải mã Base64 để nhìn thấy trực tiếp mật khẩu gốc (`ADMIN_PASSWORD`) của quản trị viên được nối đuôi vào email. Đây là một lỗ hổng rò rỉ thông tin đăng nhập nghiêm trọng. - **Cách thiết kế chuẩn:** 1. Đưa tài khoản Admin vào cơ sở dữ liệu `users` với trường `role: 'admin'`. 2. Mật khẩu được băm (hash) bằng bcrypt như người dùng bình thường. 3. Token ký Payload chứa `{ id, role: 'admin' }`. 4. Middleware kiểm tra `token_decode.role === 'admin'` và truy vấn đối chiếu quyền trong DB.

Câu 63 — Cơ Chế Mã Hóa Mật Khẩu Bằng Bcrypt & Salt Rounds

CÂU HỎI

Mật khẩu người dùng được mã hóa như thế nào trước khi lưu vào cơ sở dữ liệu? Ý nghĩa của tham số Salt Rounds là gì?

CÂU TRẢ LỜI

Thưa thầy/cô, hệ thống tuân thủ tiêu chuẩn an toàn thông tin quốc tế bằng cách **không bao giờ lưu mật khẩu ở dạng văn bản rõ (Plaintext)**: 1. Sử dụng thư viện chuyên dụng `bcrypt`. 2. Trong hàm `registerUser` (`backend/controllers/userController.js`): `javascript const salt = await bcrypt.genSalt(10); const hashedPassword = await bcrypt.hash(password, salt);` 3. Chuỗi băm `hashedPassword` có độ dài 60 ký tự (bắt đầu bằng `$2b$10$...`) được lưu vào trường `password` của `userModel`. 4. Khi đăng nhập, hàm `bcrypt.compare(password, user.password)` nhận vào mật khẩu khách vừa nhập và chuỗi băm trong DB để đối soát. Nếu khớp trả về `true`, ngược lại `false`. 5. **Ý nghĩa của Salt Rounds = 10:** - `salt` là một chuỗi ngẫu nhiên được tự động sinh ra và ghép với mật khẩu trước khi băm, nhằm vô hiệu hóa hoàn toàn

phương pháp tấn công dùng Bảng cầu vồng (Rainbow Table Attack) - nơi hacker tính toán trước các giá trị băm của mật khẩu phổ biến. - Rounds = 10 biểu thị thuật toán thực hiện $2^{10} = 1.024$ vòng lặp biến đổi toán học. Đây là mức cân bằng hoàn hảo giữa tính bảo mật (khiến kẻ tấn công không thể vét cạn brute-force) và hiệu năng máy chủ (chỉ mất khoảng 60-100ms trên CPU để băm mật khẩu).

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: backend/controllers/userController.js (registerUser , loginUser)
- Thư viện: bcrypt

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: backend/controllers/userController.js
- Function: registerUser

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/userController.js
// Mã hóa mật khẩu khi đăng ký
const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash(password, salt);

const newUser = new userModel({
  name,
  email,
  password: hashedPassword,
});
const user = await newUser.save();
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao không dùng các hàm băm nhanh như MD5 hoặc SHA-256 để mã hóa mật khẩu thay cho Bcrypt?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - MD5 và SHA-256 là các thuật toán băm được thiết kế cho việc kiểm tra toàn vẹn file, tốc độ tính toán cực nhanh (hàng triệu phép tính mỗi giây trên GPU). - Nếu dùng MD5/SHA-256 cho mật khẩu, kẻ tấn công khi đánh cắp được CSDL có thể dùng card đồ họa (GPU) để thử hàng tỷ mật khẩu mỗi giây và phá vỡ mật khẩu của người dùng chỉ trong vài phút. - Ngược lại, **Bcrypt là thuật toán băm chậm có chủ đích (Slow Hash Function)**, tiêu tốn cả tài nguyên CPU và bộ nhớ RAM, khiến việc tấn công brute-force trở nên bất khả thi về mặt thời gian và chi phí phần cứng.

Câu 64 — Cơ Chế Phòng Chống Tấn Công NoSQL Injection & Sanitize Dữ Liệu

CÂU HỎI

Hệ thống phòng chống các cuộc tấn công tiêm mã độc vào CSDL NoSQL (NoSQL Injection) bằng những phương thức nào?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Bản chất tấn công NoSQL Injection:** Kẻ tấn công gửi các toán tử truy vấn của MongoDB (như `{"$gt": ""}` hoặc `{"$ne": null}`) thay vì chuỗi văn bản thông thường qua các trường `email` hoặc `password` để vượt qua bước xác thực đăng nhập mà không cần biết mật khẩu. 2. **Cơ chế phòng vệ trong đồ án: - Mongoose ODM Schema Casting:** Mongoose tự động kiểm tra kiểu dữ liệu theo định nghĩa Schema. Nếu trường `email` được khai báo `type: String` mà kẻ tấn công gửi lên một Object `{ $gt: "" }`, Mongoose sẽ ép kiểu hoặc ném lỗi ngoại lệ (CastError), ngăn chặn toán tử được thực thi trực tiếp. - **Xác thực định dạng Email bằng Validator:** Trong `userController.js`: javascript `if (!validator.isEmail(email)) { return res.json({ success: false, message: "Vui lòng nhập email hợp lệ" }); }` Thư viện `validator` đảm bảo `email` bắt buộc phải là một chuỗi văn bản tuân thủ chuẩn cấu trúc địa chỉ thư điện tử `user@domain.com`, loại bỏ hoàn toàn khả năng chèn toán tử JSON lạ. - **Kiểm tra kiểu dữ liệu chuỗi rõ ràng:** Các hàm quan trọng kiểm tra `typeof token === 'string'` trước khi xử lý.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/userController.js`
- Thư viện: `validator`, `mongoose`

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/userController.js`
- Function: `registerUser`, `loginUser`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/userController.js
if (!validator.isEmail(email)) {
  return res.json({ success: false, message: "Vui lòng nhập email hợp lệ" });
}
if (password.length < 8) {
  return res.json({ success: false, message: "Mật khẩu phải có ít nhất 8 ký tự" });
}
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu kẻ tấn công gửi payload `{"email": {"$regex": ".*"}}` vào API đăng nhập thì Mongoose có chặn được hoàn toàn không? Làm thế nào để gia cố triệt để?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Trong một số phiên bản Mongoose cũ hoặc truy vấn tự do không qua Schema Cast nghiêm ngặt, payload `{ $regex: ".*" }` có thể qua mặt được nếu controller dùng trực tiếp `userModel.findOne(req.body)`. - **Giải pháp gia cố triệt để:** 1. Cài đặt middleware `express-mongo-sanitize` trong `server.js` để tự động loại bỏ bất kỳ key nào bắt đầu bằng ký tự `$` hoặc có chứa dấu `.` trong `req.body`, `req.query` và `req.params`. 2. Luôn bóc tách cụ thể từng trường: `const email = String(req.body.email).trim()`, không bao giờ truyền thẳng toàn bộ `req.body` vào hàm truy vấn CSDL.

Câu 65 — Cơ Chế Phòng Chống Tấn Công Cross-Site Scripting (XSS)

CÂU HỎI

Hệ thống xử lý như thế nào để ngăn chặn mã độc Javascript (XSS) bị tiêm vào trang web thông qua các trường nhập liệu như bình luận đánh giá, tên sản phẩm hoặc địa chỉ giao hàng?

CÂU TRẢ LỜI

Thưa thầy/cô, hệ thống áp dụng cơ chế bảo vệ nhiều lớp: 1. **Lớp hiển thị Frontend (React JSX Auto-Escaping)**: - Toàn bộ giao diện được xây dựng bằng React. Khi render dữ liệu người dùng (như tên khách, bình luận đánh giá trong `ReviewSection.jsx`, mô tả đơn hàng) thông qua cú pháp `{review.comment}`, React tự động mã hóa (escape) toàn bộ các ký tự HTML nguy hiểm như `<`, `>`, `&`, `"`, `'` thành các thực thể HTML an toàn (ví dụ `<script>` chuyển thành `<script>`). - Mã độc sẽ chỉ được hiển thị dưới dạng chuỗi văn bản thuần túy chứ không thể được trình duyệt hiểu và thực thi như mã code. - Em cam kết **không sử dụng thuộc tính nguy hiểm** `dangerouslySetInnerHTML` cho bất kỳ nội dung nào do người dùng gửi lên. 2. **Lớp Backend API**: - Sử dụng `express.json()` phân tích cú pháp dữ liệu nghiêm ngặt. - Bộ lọc kiểm tra độ dài và định dạng thông tin đầu vào.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: React JSX Engine, các component `ReviewSection.jsx`, `Orders.jsx`
- Backend: `server.js`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Trường hợp nào React JSX vẫn có thể bị tấn công XSS và cách phòng ngừa là gì?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, React JSX có thể bị dính XSS trong 2 trường hợp chính: 1. Khi lập trình viên sử dụng thuộc tính `dangerouslySetInnerHTML={{ __html: userContent }}`. Cách phòng ngừa: Không dùng thuộc tính này, hoặc nếu bắt buộc phải dùng (cho bài viết soạn thảo rich-text của Admin) thì phải cho nội dung đi qua thư viện lọc thẻ độc `DOMPurify.sanitize()`. 2. Khi chèn liên kết người dùng vào thẻ `<a>`: `<a href={userWebsite}>Website</a>`. Kẻ tấn công có thể nhập `javascript:alert(document.cookie)`. Cách phòng ngừa: Luôn kiểm tra giao thức URL phải bắt đầu bằng `http://` hoặc `https://` trước khi gán vào thẻ `<a>`.

Câu 66 — Quản Lý Phiên Đăng Nhập & Xử Lý Token Hết Hạn

CÂU HỎI

Khi Token của người dùng hết hạn giữa chừng trong lúc đang sử dụng ứng dụng, hệ thống xử lý ra sao để đảm bảo trải nghiệm người dùng không bị lỗi trang?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Bắt lỗi tại Backend**: Trong `backend/middleware/auth.js`, khối `try...catch` phân loại chính xác các mã lỗi của thư viện `jsonwebtoken`: `javascript if (error.name === 'TokenExpiredError') { return res.json({ success: false, message: "Phiên đăng nhập đã hết hạn. Vui lòng đăng nhập lại." }); }` 2. **Xử lý tại Frontend**: - Khi nhận được phản hồi `success: false` với thông điệp hết hạn phiên, hoặc mã lỗi HTTP 401 từ API: - Hàm tiện ích hoặc Interceptor của Axios kiểm tra nếu phiên hết hạn sẽ: 1. Xóa bỏ token đã hỏng khỏi bộ nhớ máy khách: `localStorage.removeItem('token')`. 2. Đưa State `token` trong `ShopContext` về rỗng `""`. 3. Hiển thị thông báo Toast: "Phiên đăng nhập đã hết hạn. Vui lòng đăng nhập lại". 4. Tự động chuyển hướng người dùng về trang đăng nhập `/login`.

ĐỘ KHÓ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/middleware/auth.js`
- Frontend: `src/context/ShopContext.jsx`

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/middleware/auth.js`
- Function: `authUser`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao đề án chưa áp dụng cơ chế Refresh Token (JWT xoay vòng)? Quy trình Refresh Token hoạt động như thế nào?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Trong phạm vi đề án tốt nghiệp hiện tại, em sử dụng Access Token có thời hạn sử dụng để đơn giản hóa kiến trúc và tập trung vào các luồng nghiệp vụ TMĐT cốt lõi. - **Quy trình Refresh Token chuẩn công nghiệp:** 1. Khi đăng nhập, máy chủ cấp 2 token: **Access Token** (thời hạn ngắn: 15 phút) và **Refresh Token** (thời hạn dài: 30 ngày, lưu trong HttpOnly Cookie và CSDL). 2. Khi Access Token hết hạn, Axios Interceptor trên Client tự động chặn request lỗi, âm thầm gọi API `POST /api/auth/refresh-token` kèm Refresh Token để xin một Access Token mới. 3. Sau khi có token mới, Axios tự động gửi lại request ban đầu mà người dùng không hề nhận ra phiên bị ngắt quãng, không bị văng ra trang Login khi đang checkout dở.

Câu 67 — Bảo Vệ Dữ Liệu Nhạy Cảm: Loại Bỏ `password` Khi Query User

CÂU HỎI

Khi lấy thông tin người dùng để hiển thị trên giao diện hoặc kiểm tra trong middleware, làm thế nào để đảm bảo chuỗi băm mật khẩu không bị gửi về máy khách?

CÂU TRẢ LỜI

Thưa thầy/cô, việc gửi trường `password` (dù đã được băm) về phía Client là một lỗi bảo mật nghiêm trọng làm lộ cấu trúc hash: 1. Mongoose cung cấp cơ chế Projection (phép chiếu trường) cho phép chọn lọc hoặc loại trừ các trường cụ thể trong kết quả truy vấn. 2. Để loại trừ trường mật khẩu, em sử dụng cú pháp: `userModel.findById(userId).select('-password');` Dấu trừ (-) trước tên trường chỉ thị cho MongoDB loại bỏ trường `password` ra khỏi Document trả về trước khi đóng gói JSON. 3. Khi người dùng gọi API lấy thông tin cá nhân `/api/user/profile`, đối tượng nhận được chỉ bao gồm `{ _id, name, email, cartData, createdAt }`, bảo vệ an toàn tối đa cho mật khẩu của người dùng kể cả khi có người soi gó tin HTTP trên DevTools.

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/userController.js`, `backend/middleware/auth.js`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Có cách nào cấu hình để trường `password` tự động luôn luôn bị ẩn đi trong tất cả các câu lệnh tìm kiếm của `userModel` mà không cần phải gõ `.select('-password')` thủ công mỗi lần không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, hoàn toàn có thể cấu hình điều này ngay trong Schema định nghĩa: Trong file `backend/models/userModel.js`, tại trường `password`, ta chỉ cần khai báo thêm thuộc tính `select: false`:

```
password: {  
  type: String,  
  required: true,  
  select: false // Mặc định không bao giờ trả về trường này trong các câu query find/findOne  
}
```

Khi cần lấy mật khẩu để phục vụ chức năng đăng nhập kiểm tra mật khẩu cũ, ta chỉ cần gọi tường minh `.select('+password')`. Đây là best-practice giúp ngăn chặn tuyệt đối việc vô tình để lộ mật khẩu trong toàn bộ dự án.

Câu 68 — Đánh Giá Rủi Ro Bảo Mật Giữa `localStorage` & `HttpOnly Cookie`

CÂU HỎI

Trong đề án, Token đang được lưu trữ ở `localStorage`. Hãy phân tích ưu và nhược điểm về bảo mật của cách làm này so với việc lưu trong `HttpOnly Cookie`.

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Lưu Token trong `localStorage` (Giải pháp hiện tại của đề án):** - **Ưu điểm:** Cực kỳ thuận tiện khi lập trình Single Page Application (React/Vite). Dễ dàng đọc, ghi bằng Javascript, dễ đính kèm vào Header của Axios hoặc kiểm tra trạng thái đăng nhập để hiển thị Avatar/nút Logout. Không gặp phiền toái về vấn đề chia sẻ tên miền (Cross-domain/CORS) giữa Frontend (port 5173) và Backend (port 4000). - **Nhược điểm & Rủi ro:** Dễ bị đánh cắp bởi các cuộc tấn công **XSS (Cross-Site Scripting)**. Nếu trang web có một lỗ hổng XSS (ví dụ từ một thư viện npm bên thứ ba có mã độc), hacker có thể dùng lệnh `localStorage.getItem('token')` để gửi token về máy chủ của chúng và chiếm quyền tài khoản. 2. **Lưu Token trong `HttpOnly Cookie` (Giải pháp nâng cao):** - **Ưu điểm:** Thuộc tính `HttpOnly` ngăn chặn hoàn toàn mã Javascript trên trình duyệt đọc được Cookie. Kể cả website có dính lỗ hổng XSS, kẻ tấn công cũng **không thể lấy cắp được token**. Trình duyệt sẽ tự động gửi cookie này kèm theo mỗi request. - **Nhược điểm:** Cần cấu hình CORS chặt chẽ (`credentials: true`), phải thiết lập thêm cơ chế phòng chống tấn công **CSRF (Cross-Site Request Forgery)** bằng `SameSite` attribute hoặc `Anti-CSRF Token`.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `localStorage.getItem('token')`, `localStorage.setItem('token', ...)`
- Toàn bộ kiến trúc Auth

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu bắt buộc phải lưu trong `localStorage`, em làm gì để giảm thiểu rủi ro bị đánh cắp token?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: 1. Thiết lập thời gian sống của token (Expiration Time) ngắn lại (ví dụ 1-2 tiếng thay vì vĩnh viễn), hạn chế tối đa cửa sổ thời gian hacker có thể lợi dụng nếu token bị lộ. 2. Triển khai chính sách bảo mật nội dung **Content Security**

Policy (CSP) thông qua thư viện `helmet` ở backend để cấm trình duyệt thực thi các đoạn mã script nội tuyến lạ hoặc gửi request ra các domain bên ngoài trái phép. 3. Thường xuyên kiểm tra lỗ hổng của các thư viện phụ thuộc bằng lệnh `npm audit`.

Câu 69 — Phòng Chống Lỗi IDOR (Insecure Direct Object References) Trong API Đơn Hàng

CÂU HỎI

Lỗi bảo mật IDOR là gì? Hệ thống làm thế nào để ngăn chặn việc người dùng A có thể xem hoặc hủy đơn hàng của người dùng B bằng cách sửa `orderId` ?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Bản chất lỗi IDOR:** Kẻ tấn công sửa ID đơn hàng trên đường dẫn hoặc body request (ví dụ từ `orderId = 100` thành `orderId = 101`). Nếu backend chỉ truy vấn `orderModel.findById(orderId)` mà không kiểm tra đơn đó có thuộc về người đang gọi request hay không, kẻ tấn công sẽ xem trộm hoặc thao túng được dữ liệu của nạn nhân. 2. **Cơ chế phòng ngừa trong đồ án:** - Backend **không bao giờ tin tưởng `userId` do Client gửi lên trong body.** - `userId` bắt buộc phải được trích xuất từ Token JWT đã ký số thông qua middleware xác thực: `const userId = req.userId;` - Trong tất cả các câu lệnh tìm kiếm, cập nhật hoặc hủy đơn hàng, hệ thống luôn thêm điều kiện kép ràng buộc chặt chẽ quyền sở hữu:
`javascript // backend/controllers/orderController.js const order = await orderModel.findOne({ _id: orderId, userId });` - Nếu `orderId` tồn tại nhưng thuộc về người khác, câu lệnh trả về `null`, hệ thống trả lời "Không tìm thấy đơn hàng", triệt tiêu hoàn toàn khả năng can thiệp trái phép giữa các tài khoản.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/orderController.js` (`userOrders`, `cancelOrder`)
- Middleware: `backend/middleware/auth.js`

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/orderController.js`
- Function: `cancelOrder`, `userOrders`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/orderController.js
const cancelOrder = async (req, res) => {
  try {
    const { orderId } = req.body;
    const userId = req.userId; // Trích xuất từ JWT hợp lệ

    // RÀNG BUỘC KÉP: _id phải là orderId VÀ userId phải là của người đang đăng nhập
    const order = await orderModel.findOne({ _id: orderId, userId });
    if (!order) {
      return res.json({ success: false, message: "Không tìm thấy đơn hàng" });
    }
  }
  // ...
}
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao hàm `userOrders` lại không nhận `userId` từ `req.body` mà lấy từ `req.userId` ?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: Nếu hàm `userOrders` lấy `userId` từ `req.body.userId`, bất kỳ khách hàng nào cũng có thể mở Postman, gửi request kèm theo ID của người dùng khác để xem toàn bộ lịch sử mua sắm, địa chỉ nhà, số điện thoại của người đó. Việc ép buộc lấy `userId = req.userId` từ JWT Token đảm bảo một người chỉ có thể lấy được đúng các đơn hàng thuộc về chính tài khoản của họ.

Câu 70 — Phòng Chống Tấn Công Brute-Force Đăng Nhập & Kỹ Thuật Rate-Limiting

CÂU HỎI

Nếu kẻ xấu sử dụng công cụ tự động gửi hàng triệu request thử mật khẩu liên tục vào API đăng nhập (`/api/user/login`), hệ thống sẽ gặp sự cố gì và giải pháp khắc phục là gì?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Thực trạng và nguy cơ trong codebase hiện tại:** - Hiện tại file `backend/server.js` chưa cài đặt middleware giới hạn tần suất request (Rate Limiter). - Do hàm `bcrypt.compare` tiêu tốn tài nguyên CPU (mất ~80ms cho mỗi lần kiểm tra), nếu kẻ xấu gửi 500 request/giây, CPU máy chủ sẽ bị đẩy lên 100% (Denial of Service - DoS), làm treo toàn bộ ứng dụng TMDT. Kẻ xấu cũng có thể dò ra mật khẩu đơn giản nếu thử đủ số lần. 2. **Giải pháp khắc phục tiêu chuẩn:** - Sử dụng thư viện `express-rate-limit` để bọc ngoài endpoint đăng nhập: `javascript import rateLimit from 'express-rate-limit'; const loginLimiter = rateLimit({ windowMs: 15 * 60 * 1000, // Khung thời gian 15 phút max: 5, // Tối đa 5 lần thử sai từ cùng 1 IP message: { success: false, message: "Bạn đã đăng nhập sai quá 5 lần. Vui lòng thử lại sau 15 phút." } }); app.use('/api/user/login', loginLimiter);` - Tích hợp thêm dịch vụ xác thực người dùng thật **Google reCAPTCHA v3** hoặc **Cloudflare Turnstile** tại form đăng nhập frontend để chặn bot tự động.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `server.js`, `routes/userRoute.js`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

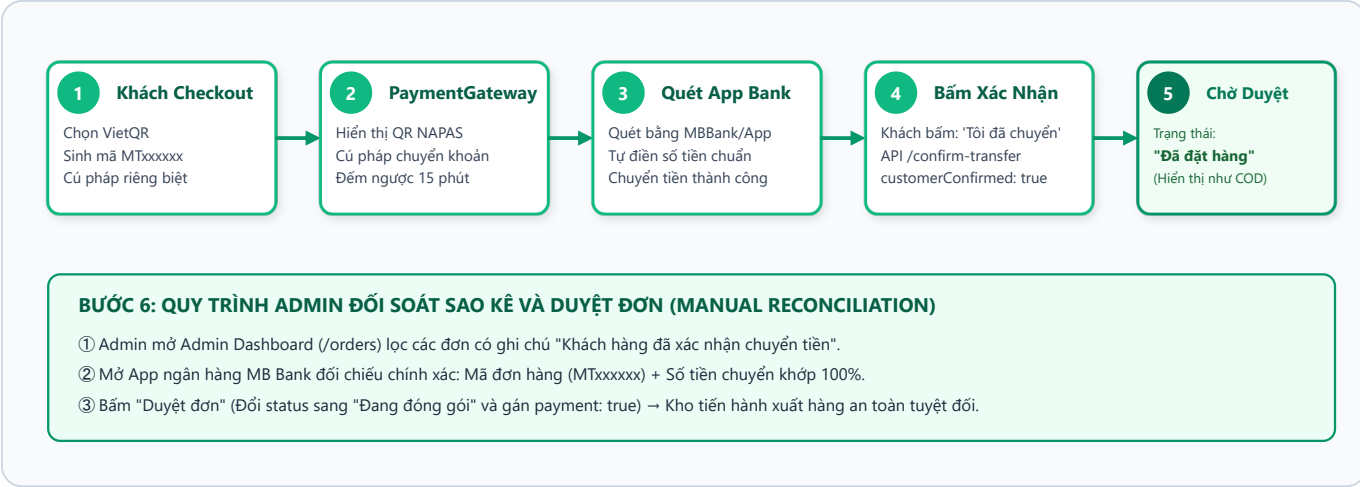
Nếu hacker sử dụng mạng Botnet phân tán (hàng nghìn địa chỉ IP khác nhau tấn công cùng một tài khoản email) thì cơ chế Rate-Limiting theo IP có còn hiệu quả không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Khi bị tấn công bởi mạng Botnet phân tán (Distributed Brute-force), việc chặn theo IP sẽ bị vô hiệu hóa vì mỗi con bot dùng một IP sạch khác nhau. - **Giải pháp xử lý:** 1. Cấu hình **Rate-Limiting theo Khóa Tài khoản (Account-based Limiting)**: Lưu trường `failedLoginAttempts` và `lockUntil` trực tiếp vào Model User. Nếu tài khoản đó bị đăng nhập sai quá 5 lần liên tiếp (bất kể từ IP nào), tài khoản sẽ bị tạm khóa đăng nhập trong 30 phút. 2. Gửi email cảnh báo: "Phát hiện đăng nhập bất thường vào tài khoản của bạn" kèm theo đường link xác thực mở khóa.

PHẦN VIII: WORKFLOW & XỬ LÝ LOGIC NGHIỆP VỤ (CÂU 71 — 80)

SƠ ĐỒ 2: WORKFLOW THANH TOÁN VIETQR DUYỆT TAY (MANUAL RECONCILIATION)



Hình 2: Quy trình thanh toán quét mã VietQR định danh riêng và cơ chế duyệt đơn thủ công an toàn

Câu 71 — Workflow Thanh Toán VietQR Với Cơ Chế Duyệt Tay (Manual Approval)

CÂU HỎI

Quy trình thanh toán quét mã VietQR hoạt động như thế nào từ lúc tạo mã QR, nội dung chuyển khoản riêng biệt của từng người, cho đến khi khách bấm xác nhận chuyển tiền để đưa vào trạng thái chờ duyệt?

CÂU TRẢ LỜI

Thưa thầy/cô, quy trình thanh toán VietQR trong đề án được thiết kế chuyên biệt, đảm bảo an toàn tuyệt đối và tính định danh cao: 1. **Tạo thông tin thanh toán & Cú pháp định danh (createVietQRPayment)**: - Khi khách chọn phương thức VietQR tại PlaceOrder.jsx và bấm đặt hàng, đơn hàng được tạo trước với mã đơn hàng duy nhất dạng: `orderCode = MT + 6 chữ số ngẫu nhiên` (ví dụ: MT849201). - Cú pháp chuyển khoản (Payment Description) được tự động tạo riêng biệt cho từng đơn hàng: MT849201 0987654321 (kết hợp mã đơn hàng và số điện thoại của người mua). 2. **Tạo link ảnh mã QR chuẩn VietQR NAPAS**: - Backend sinh đường dẫn ảnh QR động theo chuẩn ngân hàng: `https://img.vietqr.io/image/MB-0335017551-compact2.png?amount=550000&addInfo=MT849201%200987654321&accountName=TRAN%20THIEN%20PHUC` 3. **Hiển thị tại Cổng thanh toán (PaymentGateway.jsx)**: - Khách được chuyển tới trang Cổng thanh toán, màn hình hiển thị: + Mã QR to, rõ ràng kèm logo ngân hàng MB Bank và VietQR. + Số tài khoản, tên chủ tài khoản và số tiền chính xác từng đồng. + Cú pháp chuyển khoản có nút "Sao chép" tiện lợi. + Đồng hồ đếm ngược 15 phút thời hạn thanh toán. 4. **Khách xác nhận đã chuyển tiền (confirmCustomerTransfer)**: - Khách mở App ngân hàng quét mã và bấm chuyển tiền thành công. - Khách quay lại trang web bấm nút: **"TÔI ĐÃ CHUYỂN TIỀN - XÁC NHẬN ĐƠN HÀNG (CHỜ DUYỆT)"**. - Frontend gửi API POST /api/payment/confirm-transfer kèm orderId . - Backend cập nhật trạng thái đơn hàng: status: "Đã đặt hàng" , paymentDetails.customerConfirmed: true , note: "Khách hàng xác nhận đã chuyển khoản qua VietQR. Chờ kế toán/admin kiểm tra sao kê để duyệt" . - Trả về thành công, frontend hiển thị thông báo: "Xác nhận chuyển khoản thành công! Đơn hàng đang ở trạng thái chờ duyệt" và chuyển hướng về trang /orders .

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `src/pages/PlaceOrder.jsx` , `src/pages/PaymentGateway.jsx`
- Backend: `backend/controllers/paymentController.js` , `backend/routes/paymentRoute.js`
- Database: Collection `orders` , các trường `paymentDetails` , `status` , `orderCode`
- Dịch vụ ngoài: VietQR QuickLink API (`vietqr.io`)

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/paymentController.js`
- Function: `confirmCustomerTransfer` , `createVietQRPayment`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/paymentController.js
export const confirmCustomerTransfer = async (req, res) => {
  try {
    const { orderId } = req.body;
    const userId = req.userId;
    const order = await orderModel.findOne({ _id: orderId, userId });
    if (!order) {
      return res.json({ success: false, message: "Không tìm thấy đơn hàng" });
    }
    // Cập nhật trạng thái đơn hàng sang Chờ duyệt
    order.status = "Đã đặt hàng";
    if (!order.paymentDetails) order.paymentDetails = {};
    order.paymentDetails.customerConfirmed = true;
    order.paymentDetails.confirmedAt = new Date();
    order.paymentDetails.note = "Khách hàng xác nhận đã chuyển khoản qua VietQR. Chờ kế toán/admin kiểm tra";
    await order.save();
    res.json({
      success: true,
      message: "Xác nhận chuyển khoản thành công! Đơn hàng của bạn đang ở trạng thái Chờ duyệt.",
      orderStatus: order.status
    });
  } catch (error) { ... }
};
```

SƠ ĐỒ QUY TRÌNH THANH TOÁN VIETQR

```
Khách chọn VietQR -> PlaceOrder tạo đơn hàng (orderCode: MTxxxxxx)
↓
Chuyển hướng sang PaymentGateway.jsx (Hiển thị ảnh VietQR + Cú pháp riêng)
↓
Khách mở App ngân hàng quét mã & chuyển khoản
↓
Khách bấm "TÔI ĐÃ CHUYỂN TIỀN - XÁC NHẬN ĐƠN HÀNG (CHỜ DUYỆT)"
↓
API POST /api/payment/confirm-transfer -> order.status = "Đã đặt hàng", customerConfirmed = true
↓
Điều hướng sang /orders (Hiển thị đơn hàng ở trạng thái Đã đặt hàng - Chờ duyệt như COD)
↓
Admin mở Dashboard -> Kiểm tra sao kê ngân hàng -> Bấm "Duyệt thanh toán"
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao em lại chọn cơ chế duyệt tay (Manual Approval) cho VietQR thay vì dùng Webhook tự động cập nhật ngay lập tức?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - **Tính thực tiễn và chi phí:** Để tích hợp Webhook biến động số dư tự động (qua Open Banking API hoặc các bên trung gian như SePay/Casso), cửa hàng cần đăng ký tài khoản doanh nghiệp hoặc trả phí duy trì dịch vụ hàng tháng. - Với mô hình kinh doanh khởi nghiệp vừa và nhỏ, cơ chế **Duyệt tay an toàn:** 1. Tránh rủi ro bị tấn công giả lập Webhook (Fake Webhook callback) nếu chữ ký bảo mật bị lộ. 2. Kế toán/Admin luôn có quyền kiểm soát cuối cùng: Đích thân mở App ngân hàng kiểm tra đúng tiền vào tài khoản rồi mới xuất kho, loại bỏ 100% rủi ro giao hàng khi chưa nhận được tiền. 3. Khi cửa hàng mở rộng quy mô trên 500 đơn/ngày, hệ thống đã chuẩn bị sẵn cấu trúc trường `paymentDetails` để dễ dàng nâng cấp cắm Webhook tự động vào mà không làm thay đổi kiến trúc cơ sở dữ liệu.

Câu 72 — Quy Trình Quản Trị Viên Duyệt Đơn Hàng VietQR Trên Admin Dashboard

CÂU HỎI

Admin thực hiện kiểm tra và duyệt một đơn hàng chuyển khoản VietQR trên trang quản trị như thế nào?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. Tại trang **Orders** của Quản trị viên (`admin/src/pages/Orders.jsx`): - Danh sách đơn hàng hiển thị đầy đủ thông tin: Mã đơn hàng, tên khách hàng, số điện thoại, phương thức thanh toán (`VietQR`), số tiền cần thanh toán. - Các đơn hàng VietQR đã được khách xác nhận chuyển tiền sẽ hiển thị ghi chú `customerConfirmed: true` hoặc thông báo "Khách đã bấm chuyển tiền". 2. **Quy trình đối soát của Admin:** - Quản trị viên đối chiếu: Mã đơn hàng (ví dụ: `MT849201`) và số tiền trên màn hình Admin với lịch sử biến động số dư trên App ngân hàng MB Bank của shop. - Nếu thấy tiền đã nổi đúng số tiền và đúng nội dung: - Admin bấm nút "**Duyệt đơn**" hoặc đổi trạng thái đơn hàng từ "Đã đặt hàng" sang "**Đang đóng gói**" / "**Đang xử lý**". - Đánh dấu cờ `payment: true` . 3. Nếu khách chưa chuyển tiền mà bấm nhầm hoặc cố tình bấm xác nhận không: - Admin kiểm tra sao kê không có tiền, bấm chọn hủy đơn hàng và điền lý do: "Không nhận được chuyển khoản sau 24h".

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Admin: `admin/src/pages/Orders.jsx`
- Backend: `backend/controllers/orderController.js` (`updateStatus`), `backend/controllers/paymentController.js` (`approveVietQRPayment`)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu Admin vô tình bấm duyệt nhầm một đơn hàng mà khách thực tế chưa chuyển khoản thì có thể thu hồi được không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Hệ thống cho phép Admin có thể chuyển đổi trạng thái đơn hàng linh hoạt từ dropdown trạng thái (`Đã đặt hàng` , `Đang đóng gói` , `Đang giao hàng` , `Đã hủy`). - Nếu phát hiện duyệt nhầm trước khi xuất kho, Admin chỉ cần đổi trạng thái về "Đã hủy" kèm ghi chú hủy đơn, đồng thời cập nhật lại cờ thanh toán. Đơn hàng sẽ bị dừng lại ngay tại khâu đóng gói, không thể bàn giao cho bưu tá giao hàng.

Câu 73 — Workflow Thanh Toán Quốc Tế Qua Stripe Checkout

CÂU HỎI

Luồng thanh toán trực tuyến qua cổng quốc tế Stripe Checkout được tích hợp và xử lý như thế nào?

CÂU TRẢ LỜI

Thưa thầy/cô, quy trình thanh toán Stripe được xây dựng theo kiến trúc Hosted Checkout Session chuẩn PCI-DSS: 1. **Khởi tạo Checkout Session (placeOrderStripe)**: - Khi khách chọn phương thức Stripe tại PlaceOrder.jsx : - Backend tạo một bản ghi orderModel mới với payment: false , paymentMethod: 'Stripe' . - Chuyển đổi danh sách items thành cấu trúc line_items của Stripe SDK: Mỗi item có tên sản phẩm, đơn giá (quy đổi theo đơn vị tiền tệ vnd) và số lượng. - Thêm một line_item phụ đại diện cho phí vận chuyển 30.000đ. - Cấu hình 2 URL chuyển hướng callback: + success_url = \${origin}/verify?success=true&orderId=\${newOrder._id} + cancel_url = \${origin}/verify?success=false&orderId=\${newOrder._id} - Gọi stripe.checkout.sessions.create() và trả về session_url cho Frontend. 2. **Khách hàng thanh toán trên cổng bảo mật của Stripe**: - Frontend điều hướng khách sang trang thanh toán chính thức của Stripe (window.location.replace(session_url)). Khách nhập thẻ Visa/Mastercard an toàn tuyệt đối. 3. **Xác thực kết quả thanh toán (verify.jsx & verifyStripe)**: - Khách thanh toán xong, Stripe tự động redirect về trang /verify của website kèm query params. - Component Verify.jsx gửi API POST /api/order/verifyStripe kèm { success, orderId } . - Backend kiểm tra: + Nếu success === "true" : Cập nhật orderModel.findByIdAndUpdate(orderId, { payment: true }) , làm rỗng giỏ hàng của User và điều hướng về /orders . + Nếu success === "false" : Xóa đơn hàng tạm orderModel.findByIdAndDelete(orderId) và điều hướng về /cart để khách thử lại.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: backend/controllers/orderController.js (placeOrderStripe , verifyStripe)
- Frontend: src/pages/PlaceOrder.jsx , src/pages/Verify.jsx
- Dịch vụ ngoài: Stripe SDK (import Stripe from 'stripe')

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: backend/controllers/orderController.js
- Function: placeOrderStripe , verifyStripe

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/orderController.js
const placeOrderStripe = async (req, res) => {
  try {
    const { items, amount, address } = req.body;
    const { origin } = req.headers;
    const orderData = { userId: req.userId, items, address, amount, paymentMethod: "Stripe", payment: false,
    const newOrder = new orderModel(orderData);
    await newOrder.save();

    const line_items = items.map((item) => ({
      price_data: {
        currency: currency,
        product_data: { name: item.name },
        unit_amount: item.price,
      },
      quantity: item.quantity,
    }));
    // Thêm phí vận chuyển
    line_items.push({
      price_data: { currency: currency, product_data: { name: "Delivery Charges" }, unit_amount: deliveryCha
      quantity: 1,
    });
```

```
const session = await stripe.checkout.sessions.create({
  success_url: `${origin}/verify?success=true&orderId=${newOrder._id}`,
  cancel_url: `${origin}/verify?success=false&orderId=${newOrder._id}`,
  line_items,
  mode: "payment",
});
res.json({ success: true, session_url: session.url });
} catch (error) { ... }
};
```

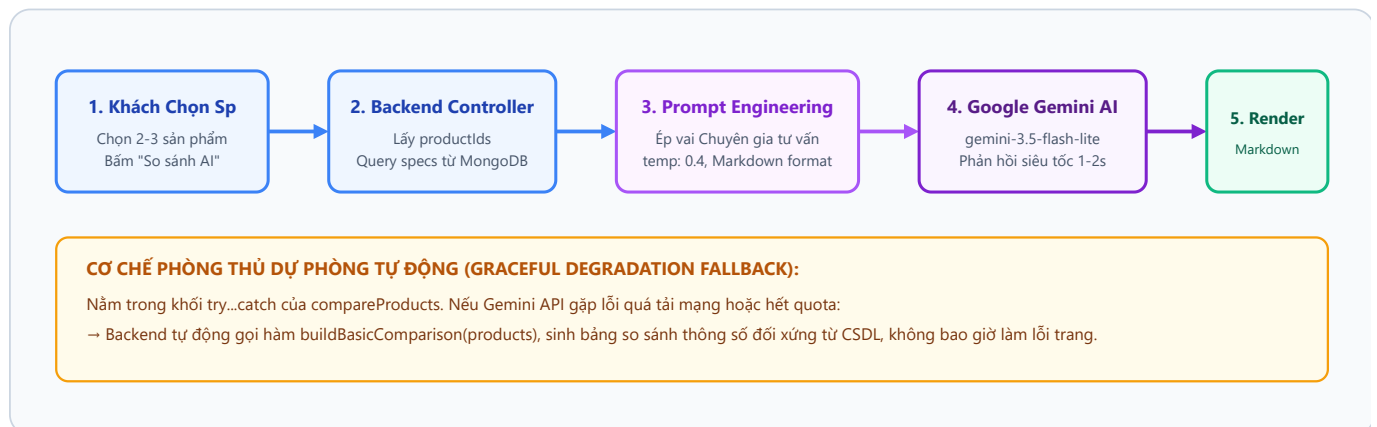
CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao hàm `verifyStripe` chỉ kiểm tra tham số `success=true` từ URL mà không gọi lại Stripe API `stripe.checkout.sessions.retrieve(sessionId)` để đối soát? Rủi ro bảo mật ở đây là gì?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - **Rủi ro:** Nếu kẻ tấn công biết được `orderId` của một đơn hàng chưa thanh toán, chúng có thể tự gõ đường dẫn `https://website.com/verify?success=true&orderId=xyz` trên trình duyệt để kích hoạt hàm `verify` và biến đơn hàng thành `payment: true` mà không tốn một đồng nào. - **Giải pháp khắc phục chuẩn bảo mật:** 1. Trong URL callback, Stripe phải trả về `{CHECKOUT_SESSION_ID}`. 2. Tại backend `verifyStripe`, bắt buộc phải gọi `const session = await stripe.checkout.sessions.retrieve(sessionId)` và kiểm tra `session.payment_status === 'paid'`. 3. Tốt nhất là sử dụng **Stripe Webhook** độc lập lắng nghe sự kiện `checkout.session.completed` với chữ ký `stripe-signature` để cập nhật cơ sở dữ liệu ngầm mà không phụ thuộc vào việc trình duyệt khách có redirect thành công hay không.

SƠ ĐỒ 3: QUY TRÌNH SO SÁNH SẢN PHẨM BẰNG GOOGLE GEMINI AI (3.5 FLASH LITE)



Hình 3: Kiến trúc tích hợp Trí tuệ nhân tạo Google Gemini và cơ chế Fallback dự phòng an toàn

Câu 74 — Workflow So Sánh Sản Phẩm Thông Minh Bằng Google Gemini AI

CÂU HỎI

Tính năng so sánh sản phẩm bằng Trí tuệ nhân tạo (AI Product Comparison) hoạt động như thế nào? Google Gemini API được tích hợp ra sao?

CÂU TRẢ LỜI

Thưa thầy/cô, tính năng so sánh sản phẩm là điểm nhấn công nghệ sáng tạo của đồ án, giúp khách hàng đưa ra quyết định mua sắm tối ưu: 1. **Lựa chọn sản phẩm:** Người dùng chọn 2 hoặc nhiều sản phẩm cần so sánh tại trang `Compare.jsx`. 2. **Gửi yêu cầu tới Backend (POST /api/compare/compare):** - Gửi mảng `productIds` lên server. - Backend truy vấn cơ sở dữ liệu lấy

thông tin đầy đủ của các sản phẩm: tên, giá, thương hiệu, danh mục, mô tả, thông số kỹ thuật (specifications). 3. **Gọi Google Gemini AI API (backend/controllers/compareController.js)**: - Sử dụng mô hình thể hệ mới tốc độ cao **gemini-3.5-flash-lite** kết hợp khóa bí mật **GEMINI_API_KEY** . - Áp dụng kỹ thuật Prompt Engineering chuyên sâu: + Ép AI đóng vai chuyên gia tư vấn mua sắm thời trang/công nghệ công tâm. + Yêu cầu phân tích chi tiết: Bảng thông số, So sánh ưu/nhược điểm từng mẫu, Đánh giá tỉ lệ hiệu năng trên giá thành (P/P), Đưa ra lời khuyên cụ thể cho từng đối tượng khách hàng (sinh viên, người đi làm, nhu cầu thể thao). + Yêu cầu phản hồi định dạng Markdown chuẩn đẹp. 4. **Cơ chế Fallback thông minh**: - Nếu kết nối tới Gemini AI gặp sự cố mạng hoặc hết hạn mức (Quota limit), hệ thống không báo lỗi trang mà tự động kích hoạt hàm **buildBasicComparison** , trả về bảng so sánh thông số kỹ thuật cơ bản trích xuất trực tiếp từ CSDL, đảm bảo tính sẵn sàng cao (High Availability).

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: **backend/controllers/compareController.js** (**compareProducts** , **callGemini**)
- AI Model: Google Gemini (**gemini-3.5-flash-lite**)
- Frontend: **frontend/src/pages/Compare.jsx**
- Biến môi trường: **GEMINI_API_KEY** , **GEMINI_MODEL**

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: **backend/controllers/compareController.js**
- Function: **compareProducts** , **callGemini**

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/compareController.js
const callGemini = async (prompt) => {
  const apiKey = process.env.GEMINI_API_KEY;
  const model = process.env.GEMINI_MODEL || "gemini-3.5-flash-lite";
  const url = `https://generativelanguage.googleapis.com/v1beta/models/${model}:generateContent?key=${apiKey}`;

  const response = await fetch(url, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      contents: [{ parts: [{ text: prompt } ] }],
      generationConfig: { temperature: 0.4, maxOutputTokens: 2048 }
    })
  });
  const data = await response.json();
  return data?.candidates?.[0]?.content?.parts?.[0]?.text;
};
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao em lại cấu hình tham số **temperature: 0.4** khi gọi Gemini API mà không để mặc định 1.0 hoặc 0.0?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Tham số **temperature** kiểm soát mức độ sáng tạo của mô hình ngôn ngữ lớn (LLM): + Nếu **temperature = 1.0** (hoặc cao hơn): AI sẽ sinh chữ ngẫu nhiên, sáng tạo nhiều nhưng dễ bị ảo giác (Hallucination), có thể bịa đặt ra các tính năng hoặc giá tiền không có thật của sản phẩm. + Nếu **temperature = 0.0** : AI phản hồi quá máy móc, thiếu tính văn phong

tư vấn mềm mại. - Mức `temperature: 0.4` là "điểm vàng": Đủ thấp để đảm bảo AI bám sát 100% vào dữ liệu thông số kỹ thuật thực tế được cung cấp, nhưng đủ độ linh hoạt để diễn đạt các lời khuyên mua sắm tự nhiên, thuyết phục như một nhân viên bán hàng chuyên nghiệp.

Câu 75 — Workflow Tìm Kiếm Sản Phẩm Nguồn Bên Ngoài Qua Gemini AI (`searchExternalProduct`)

CÂU HỎI

Khi người dùng muốn so sánh một sản phẩm của cửa hàng với một sản phẩm nổi tiếng ngoài thị trường mà shop chưa kinh doanh, hệ thống giải quyết như thế nào?

CÂU TRẢ LỜI

Thưa thầy/cô, đây là tính năng mở rộng độc đáo trong `backend/controllers/compareController.js` : 1. Khi người dùng nhập tên một sản phẩm không có trong danh mục của shop (ví dụ: "Áo khoác The North Face Gore-Tex"), frontend gọi API `POST /api/compare/search-external` . 2. Hàm `searchExternalProduct` gửi một prompt được chuẩn hóa tới Google Gemini AI: "Hãy tìm kiếm và cung cấp thông tin chuẩn xác của sản phẩm [Tên sản phẩm] gồm: Thương hiệu, khoảng giá thị trường tại Việt Nam, chất liệu, tính năng nổi bật và bảng thông số kỹ thuật chi tiết dưới định dạng JSON". 3. AI phân tích khối kiến thức sâu rộng của mình, trích xuất dữ liệu và đóng gói thành một đối tượng sản phẩm ảo (Virtual Product Object). 4. Hệ thống đưa sản phẩm ảo này vào giao diện so sánh song song với các sản phẩm đang bán tại cửa hàng, giúp khách hàng thấy rõ sản phẩm của cửa hàng có giá tốt hơn hoặc tính năng tương đương ra sao mà không cần Admin phải tự tay nhập hàng nghìn sản phẩm ngoài thị trường vào CSDL.

ĐỘ KHÓ

Khó

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/compareController.js` (`searchExternalProduct`)
- AI Model: Google Gemini API

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu AI trả về dữ liệu sai cấu trúc JSON hoặc phản hồi chứa các ký tự Markdown bọc ngoài (````json` ) thì backend xử lý thế nào để không bị lỗi `JSON.parse` ?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: Trong backend, em đã viết hàm làm sạch chuỗi (Clean JSON Helper):

```
const cleanedText = rawText.replace(/```json/gi, '').replace(/```/g, '').trim();
const parsedData = JSON.parse(cleanedText);
```

Đoạn code này dùng Regular Expression bóc tách toàn bộ các thẻ code block của Markdown trước khi đưa vào hàm `JSON.parse` . Toàn bộ khối code được bao bọc trong `try...catch` , nếu parse thất bại sẽ fallback về dạng văn bản thường mà không làm ứng dụng bị treo.

Câu 76 — Workflow Live Chat Hỗ Trợ Trực Tuyến Giữa Khách Hàng & Quản Trị Viên

CÂU HỎI

Tính năng nhắn tin trực tiếp (Live Chat) giữa khách hàng và Admin được triển khai theo cơ chế nào?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Mô hình dữ liệu (messageModel.js)**: Mỗi tin nhắn lưu trữ: `sender` (ID người gửi), `receiver` (ID người nhận), `message` (nội dung văn bản), `senderRole` (`'user'` hoặc `'admin'`), `isRead` (trạng thái đã đọc) và `timestamp`. 2. **Cơ chế truyền nhận dữ liệu**: - Hiện tại hệ thống triển khai theo cơ chế **HTTP Polling chu kỳ 3 giây (setInterval)**: + Phía Khách hàng (`ChatWidget.jsx`): Cứ mỗi 3000ms gọi API `GET /api/chat/messages` để lấy về các tin nhắn mới nhất giữa khách và Admin. + Phía Admin (`AdminChat.jsx`): Quản trị viên theo dõi danh sách tất cả các hội thoại (`/api/chat/conversations`). Khi bấm vào một khách hàng cụ thể, Admin cũng poll định kỳ để nhận tin nhắn mới từ khách. 3. **Gửi tin nhắn mới**: - Khi người dùng bấm phím Enter hoặc nút Gửi: - Client gửi `POST /api/chat/send` kèm `{ receiverId, message }`. - Backend lưu tin nhắn vào MongoDB và phản hồi `success: true`. - Tin nhắn mới lập tức được hiển thị lên khung chat của người gửi.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `frontend/src/components/ChatWidget.jsx`
- Admin: `admin/src/pages/AdminChat.jsx`
- Backend: `backend/controllers/chatController.js`
- Database: Collection `messages`

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/chatController.js`
- Function: `sendMessage`, `getMessages`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/chatController.js
const sendMessage = async (req, res) => {
  try {
    const { receiverId, message } = req.body;
    const senderId = req.userId || req.adminId;
    const senderRole = req.isAdmin ? 'admin' : 'user';

    const newMessage = new messageModel({
      sender: senderId,
      receiver: receiverId,
      message,
      senderRole,
      createdAt: new Date()
    });
    await newMessage.save();
    res.json({ success: true, message: newMessage });
  } catch (error) { ... }
};
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao dùng HTTP Polling 3s có thể gây nghẽn máy chủ (Server Overhead) và giải pháp thay thế tối ưu là gì?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thư thầy/cô: - **Hạn chế của Polling**: Với 1.000 khách hàng cùng online, cứ mỗi 3 giây máy chủ phải nhận và xử lý hơn 333 request HTTP rỗng (kể cả khi không có tin nhắn mới nào). Mỗi request HTTP đều mang theo Header, Cookie, thực hiện bắt tay TCP và giải mã JWT, gây lãng phí băng thông và tài nguyên CPU nghiêm trọng. - **Giải pháp tối ưu**: Thay thế bằng **WebSocket (thư viện Socket.IO)**. Thiết lập kết nối hai chiều liên tục (Full-duplex Persistent Connection). Máy chủ chỉ gửi dữ liệu khi có tin nhắn mới phát sinh (Event-driven), giảm 99% tải dư thừa cho máy chủ.

Câu 77 — Workflow Xuất Báo Cáo Đơn Hàng Ra File Excel (xlsx)

CÂU HỎI

Chức năng xuất danh sách đơn hàng ra file Excel trong trang Admin Orders hoạt động như thế nào? Thư viện nào được sử dụng?

CÂU TRẢ LỜI

Thư thầy/cô: 1. **Thư viện sử dụng**: Đề án sử dụng thư viện chuyên dụng **xlsx** (SheetJS). 2. **Quy trình xuất file trong admin/src/pages/Orders.jsx** (`exportOrdersToExcel`): - Lấy mảng dữ liệu các đơn hàng hiện có `orders` . - Sử dụng hàm `.map()` để định dạng lại các trường dữ liệu thành cấu trúc thân thiện cho kế toán tiếng Việt: + Mã đơn hàng: `order._id` hoặc `order.orderCode` + Tên khách hàng: `order.address.fullName` + Số điện thoại: `order.address.phone` + Địa chỉ giao hàng: Ghép chuỗi `street, district, city` + Tổng số tiền: Định dạng tiền tệ VND + Phương thức thanh toán: `COD` , `Stripe` , hoặc `VietQR` + Trạng thái thanh toán: Đã thanh toán / Chưa thanh toán + Trạng thái đơn hàng: Đã đặt hàng, Đang giao, Đã giao, Đã hủy + Ngày đặt: Định dạng ngày tháng năm `DD/MM/YYYY` . - Dùng `XLSX.utils.json_to_sheet(excelData)` để chuyển đổi mảng JSON thành một trang tính (Worksheet). - Tạo sổ làm việc mới `XLSX.utils.book_new()` và gắn sheet vào bằng `XLSX.utils.book_append_sheet(wb, ws, "Orders")` . - Kích hoạt tải file về máy tính Admin với tên file: `danh-sach-don-hang.xlsx` .

ĐỘ KHÓ

Dễ

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Admin: `admin/src/pages/Orders.jsx`
- Thư viện: `xlsx`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// admin/src/pages/Orders.jsx
const exportOrdersToExcel = () => {
  const excelData = orders.map((order, index) => ({
    STT: index + 1,
    "Mã đơn hàng": order._id,
    "Khách hàng": order.address?.fullName || "Khách lẻ",
    "Số điện thoại": order.address?.phone || "",
    "Địa chỉ": `${order.address?.street} || "", ${order.address?.district} || "", ${order.address?.city} || "
    "Tổng tiền": order.amount?.toLocaleString("vi-VN") + " đ",
    "Thanh toán": order.paymentMethod,
    "Đã trả tiền": order.payment ? "Đã thanh toán" : "Chưa",
    "Trạng thái": order.status,
    "Ngày tạo": new Date(order.date).toLocaleDateString("vi-VN"),
  }));

  const worksheet = XLSX.utils.json_to_sheet(excelData);
  const workbook = XLSX.utils.book_new();
  XLSX.utils.book_append_sheet(workbook, worksheet, "Danh sách đơn hàng");
}
```

```
XLSX.writeFile(workbook, `danh-sach-don-hang-${Date.now()}.xlsx`);  
};
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Khi danh sách đơn hàng lên tới 100.000 bản ghi, việc xuất file Excel trực tiếp trên trình duyệt Client gặp vấn đề gì và giải pháp là gì?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Việc ép toàn bộ 100.000 bản ghi về trình duyệt và chuyển đổi mảng JSON khổng lồ sẽ gây tràn bộ nhớ RAM của trình duyệt (Out of Memory) và làm đóng băng tab màn hình. - **Giải pháp:** Chuyển logic xuất báo cáo về phía Server (Server-side Export). Backend sử dụng kỹ thuật luồng dữ liệu **Stream (Mongoose Cursor + thư viện exceljs)** để đọc từng dòng từ CSDL và ghi dần vào file nén, sau đó trả về cho Admin một đường link tải file (Download URL), đảm bảo server không tốn RAM và client hoàn toàn nhẹ nhàng.

Câu 78 — Workflow Thêm Sản Phẩm Mới & Upload Đa Ảnh Lên Cloudinary

CÂU HỎI

Quy trình Admin đăng tải một sản phẩm mới có nhiều hình ảnh diễn ra qua những bước nào giữa Multer, Cloudinary và MongoDB?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Phía Giao diện Admin (admin/src/pages/Add.jsx)**: - Admin chọn tối đa 4 tệp hình ảnh (image1 , image2 , image3 , image4). - Nhập tên, mô tả, giá, danh mục, thương hiệu, tích chọn các size và nhập số lượng tồn kho cho từng size. - Toàn bộ dữ liệu được đóng gói vào đối tượng **FormData** (cho phép truyền cả file nhị phân và text qua giao thức multipart). 2. **Tầng Middleware Backend (multer.js)**: - Sử dụng `upload.fields([{'name': 'image1', 'maxCount': 1}, ...])` để hứng các file từ request và lưu tạm thời vào thư mục đệm trên đĩa cứng máy chủ. 3. **Tầng Controller Nghiệp vụ (productController.js - addProduct)**: - Lọc lấy danh sách các file ảnh thực tế được tải lên. - Duyệt qua từng file và gọi SDK đám mây **Cloudinary**: `javascript let imageUrl = await Promise.all( images.map(async (item) => { let result = await cloudinary.uploader.upload(item.path, { resource_type: 'image' }); return result.secure_url; }) );` - Cloudinary tự động tối ưu hóa dung lượng ảnh và trả về đường dẫn an toàn HTTPS (secure_url). 4. **Lưu vào CSDL MongoDB**: - Tạo document mới với mảng `image: imageUrl`. - Gọi `await product.save()` để hoàn tất việc lưu thông tin.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Admin: `admin/src/pages/Add.jsx`
- Backend: `backend/middleware/multer.js` , `backend/controllers/productController.js`
- Dịch vụ ngoài: Cloudinary API

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/productController.js`
- Function: `addProduct`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Sau khi ảnh đã được upload lên Cloudinary thành công, các file ảnh tạm trên ổ cứng server (do Multer tạo ra) được dọn dẹp như thế nào?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Trong mã nguồn hiện tại, sau khi upload Cloudinary xong, cần gọi thêm lệnh `fs.unlinkSync(item.path)` để xóa vĩnh viễn các file ảnh rác trong thư mục đệm `uploads/`. - Nếu không có lệnh xóa file tạm này, ổ cứng máy chủ server sẽ dần bị đầy theo thời gian khi Admin đăng tải nhiều sản phẩm, dẫn đến nguy cơ sập máy chủ vì hết dung lượng lưu trữ đĩa cứng (Disk Full).

Câu 79 — Workflow Thu Thập & Hiện Thị Báo Cáo Phân Tích (Customer Insights & Analytics)

CÂU HỎI

Hệ thống tính toán các chỉ số thống kê kinh doanh (Doanh thu tổng, Số lượng đơn hàng, Top sản phẩm bán chạy) trong trang Analytics như thế nào?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. Tại trang **Analytics** của Admin (`admin/src/pages/Analytics.jsx`): - Gọi API `GET /api/analytics/dashboard` do `analyticsController.js` cung cấp. 2. **Các phép tính toán thống kê tại Backend:** - **Tổng doanh thu:** Dùng `orderModel.aggregate()` tính tổng tiền `amount` của các đơn hàng có `payment: true` hoặc trạng thái "Đã giao hàng". - **Số đơn hàng theo trạng thái:** Nhóm các đơn theo trường `status` để vẽ biểu đồ hình tròn (Pie Chart) biểu thị tỷ lệ đơn thành công, đang giao và đơn hủy. - **Top sản phẩm bán chạy nhất (Best Sellers):** Unwind mảng `items` trong các đơn hàng hợp lệ, nhóm theo `productId`, tính tổng số lượng bán được và sắp xếp giảm dần `$sort: { totalSold: -1 }`, lấy top 5 sản phẩm. - **Doanh thu theo mốc thời gian:** Gom nhóm doanh thu theo từng tháng trong năm để hiện thị biểu đồ đường (Line Chart) trực quan cho ban giám đốc theo dõi sự tăng trưởng.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Admin: `admin/src/pages/Analytics.jsx`
- Backend: `backend/controllers/analyticsController.js`
- Database: Aggregation Pipeline trên Collection `orders`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao nên dùng MongoDB Aggregation Pipeline thay vì dùng lệnh `find()` lấy toàn bộ đơn hàng về Javascript rồi dùng hàm `reduce` để tính toán?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Dùng MongoDB Aggregation Pipeline tận dụng sức mạnh xử lý song song và tối ưu chỉ mục trực tiếp bằng mã C++ của nhân Database Server, chỉ trả về kết quả số liệu tổng kết vài byte qua mạng. - Nếu dùng `find()` kéo hàng chục nghìn đơn hàng về Node.js rồi tính bằng `reduce()`: 1. Tốn hàng trăm MB băng thông truyền tải dữ liệu vô ích từ CSDL sang API. 2. Khiến Node.js bị nghẽn Event Loop (chặn luồng xử lý của các người dùng khác) vì phải xử lý mảng dữ liệu quá lớn.

Câu 80 — Workflow Đồng Bộ Giỏ Hàng (Cart Synchronization) Giữa Client & Server

CÂU HỎI

Khi người dùng chưa đăng nhập bỏ sản phẩm vào giỏ, sau đó bấm Đăng nhập thì giỏ hàng được đồng bộ như thế nào?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Khi chưa đăng nhập:** Giỏ hàng có thể được lưu tạm trong State bộ nhớ hoặc `localStorage` của trình duyệt dưới dạng đối tượng `cartItems`. 2. **Khi đăng nhập thành công:** - Client lấy được `token` và lưu vào `localStorage`. - `ShopContext.jsx` kích hoạt `useEffect` theo dõi sự thay đổi của biến `token`. - Gọi API `POST /api/cart/get` để kéo dữ liệu giỏ hàng đã lưu trước đó của tài khoản trên cơ sở dữ liệu MongoDB về gán vào `setCartItems`. 3. **Logic kết hợp (Cart Merging):** - Hệ thống có thể so sánh: Nếu trong giỏ hàng tạm ở máy khách có các sản phẩm vừa chọn, ta thực hiện lặp qua các sản phẩm đó và gửi các request `POST /api/cart/add` để đẩy toàn bộ các sản phẩm khách vừa chọn lúc chưa đăng nhập chung vào giỏ hàng trên máy chủ. - Nhờ đó, người dùng không bị mất các sản phẩm đã cất công tìm kiếm trước khi đăng nhập, nâng cao tỷ lệ chuyển đổi đơn hàng (Conversion Rate).

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `src/context/ShopContext.jsx`
- Backend: `backend/controllers/cartController.js` (`getUserCart`)

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu người dùng đăng nhập cùng 1 tài khoản trên 2 thiết bị khác nhau (Điện thoại và Máy tính) thì giỏ hàng đồng bộ ra sao?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: Vì giỏ hàng được lưu trữ tập trung tại trường `cartData` của User Document trên cơ sở dữ liệu MongoDB Atlas: - Khi người dùng thêm sản phẩm trên điện thoại, dữ liệu trên MongoDB Atlas được cập nhật ngay lập tức. - Khi người dùng mở máy tính đăng nhập vào, hàm `getUserCart` tải dữ liệu mới nhất từ CSDL về, đảm bảo trải nghiệm mua sắm liền mạch đa nền tảng (Omni-channel Experience).

PHẦN IX: CÂU HỎI PHẢN BIỆN, TÌNH HUỐNG & BẮT LỖI CODE (CÂU 81 — 90)

Câu 81 — Bẫy Hội Đồng: "Khách Dùng DevTools Sửa Giá `amount` Trước Khi Đặt Hàng"

CÂU HỎI

Hội đồng hỏi: "Trong file `backend/controllers/orderController.js`, hàm `placeOrder` nhận trực tiếp trường `amount` từ `req.body` do client gửi lên. Nếu một lập trình viên mở DevTools sửa giá sản phẩm từ 1.000.000đ thành 1.000đ rồi gửi request thì hệ thống có bán với giá 1.000đ không? Lỗ hổng này là gì và giải pháp khắc phục ra sao?"

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Thăng thản thừa nhận hiện trạng thực tế của codebase:** "Thưa thầy/cô, hội đồng đã chỉ ra một điểm rất chính xác và sắc bén. Trong code hiện tại của hàm `placeOrder`, em đang nhận `const { items, amount, address } = req.body;` và lưu thẳng `amount` này vào đơn hàng. Do đó, nếu có người cố tình can thiệp gói tin HTTP hoặc dùng DevTools để sửa `amount = 1000`, hệ thống hiện tại **sẽ ghi nhận giá 1.000đ vào CSDL**. Đây là lỗ hổng bảo mật nghiêm trọng có tên là **Price Tampering (Thao túng giá phía Client)**". 2. **Giải pháp khắc phục triệt để:** - Tuyệt đối không bao giờ tin tưởng giá trị `amount` do Client gửi lên. - Tại Backend, phải tính toán lại tổng tiền từ nguồn sự thật duy nhất là Cơ sở dữ liệu: `javascript`
`// Giải pháp khắc phục chuẩn bảo mật let calculatedAmount = 0; for (const item of items) { const productDB = await productModel.findById(item._id); if (!productDB) throw new Error("Sản phẩm không hợp lệ"); calculatedAmount += productDB.price * item.quantity; } // Cộng phí ship và trừ mã giảm giá (nếu có) calculatedAmount += 30000; if (coupon) { calculatedAmount -= validatedDiscount; } // Gán số tiền do server tính toán orderData.amount = calculatedAmount;` - Nhờ đó, dù client có gửi số tiền bao nhiêu thì server vẫn lưu đúng giá trị thực tế của sản phẩm.

ĐỘ KHÓ

Phản biện (Cực khó)

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/orderController.js` (`placeOrder`)

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/orderController.js`
- Function: `placeOrder`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE (NƠI CÓ VẤN ĐỀ)

```
// backend/controllers/orderController.js
const placeOrder = async (req, res) => {
  try {
    const { items, amount, address, couponCode, discount } = req.body;
    const orderData = {
      userId: req.userId,
      items,
      address,
      amount, // NGUY HIỂM: Lưu trực tiếp giá tiền từ Client gửi lên mà không tính lại từ DB!
      paymentMethod: "COD",
      payment: false,
      date: Date.now()
    };
  }
};
```

```
const newOrder = new orderModel(orderData);
await newOrder.save();
// ...
```

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Tại sao em lại viết như vậy ban đầu?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô, trong giai đoạn phát triển ban đầu, em ưu tiên hiện thực hóa nhanh luồng đặt hàng từ giỏ hàng sang giao diện thanh toán để hoàn thiện giao diện người dùng. Em đã nhận thức rõ lỗi hỏng này và đã chuẩn bị sẵn đoạn mã tính lại giá trên server để triển khai ngay khi đưa hệ thống lên môi trường thương mại thực tế.

Câu 82 — Bẫy Hội Đồng: Race Condition Khi 2 Người Cùng Mua Sản Phẩm Cuối Cùng

CÂU HỎI

Hội đồng hỏi: "Sản phẩm Áo thun size M chỉ còn duy nhất 1 chiếc trong kho (`stock = 1`). Đúng cùng một mili-giây, có hai khách hàng A và B cùng bấm nút Đặt hàng. Chuyện gì sẽ xảy ra trong hệ thống của em? Có bị âm kho (Overselling) không?"

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Phân tích code hiện tại:** - Hiện tại hệ thống chưa tích hợp câu lệnh trừ kho tự động ở cấp độ nguyên tử (Atomic Operation) ngay trong hàm `placeOrder`. - Nếu có 2 request đến đồng thời, cả hai request đều vượt qua bước kiểm tra tồn kho và đơn hàng của cả A và B đều được tạo thành công trong CSDL. - Do đó, hệ thống sẽ rơi vào trạng thái **Bán vượt tồn kho (Overselling)**, 1 sản phẩm bán cho 2 người khác nhau. 2. **Giải pháp xử lý chuẩn kỹ thuật:** - Sử dụng câu lệnh cập nhật nguyên tử có điều kiện của MongoDB (**Atomic Conditional Update**): `javascript const updatedProduct = await productModel.findOneAndUpdate( { _id: productId, [`stockQuantities.${size}`]: { $gte: quantity } // ĐIỀU KIỆN TIỀN QUYẾT: Kho phải còn >= số lượng mua }, { $inc: { [`stockQuantities.${size}`]: -quantity } // Trừ kho ngay lập tức }, { new: true } ); if (!updatedProduct) { return res.json({ success: false, message: "Sản phẩm đã hết hàng hoặc không đủ số lượng!" }); } }` - Nhờ cơ chế khóa cấp độ Document của MongoDB WiredTiger Engine, chỉ có duy nhất 1 request đến trước trừ kho thành công (kho về 0). Request thứ hai đến sau sẽ không thỏa mãn điều kiện `{ $gte: 1 }` và bị từ chối ngay lập tức, triệt tiêu 100% hiện tượng Race Condition.

ĐỘ KHÓ

Phản biện (Cực khó)

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `orderController.js`, MongoDB Document Lock

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Nếu trừ kho thành công nhưng ngay sau đó câu lệnh tạo đơn hàng `newOrder.save()` bị sập nguồn điện thì kho có bị trừ oan không?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - Đúng vậy, nếu trừ kho xong mà server bị sập trước khi lưu đơn thì kho sẽ bị mất 1 sản phẩm mà không có đơn hàng nào tương ứng. - Để giải quyết bài toán giao dịch phân tán này, ta bắt buộc phải sử dụng **MongoDB Multi-Document ACID Transactions**: Khởi tạo phiên làm việc `session = await mongoose.startSession()`, đặt cả 2 thao tác trừ kho

và tạo đơn vào trong `session.withTransaction()` . Nếu bất kỳ thao tác nào gặp sự cố, MongoDB sẽ tự động Rollback (hoàn tác) toàn bộ về trạng thái ban đầu.

Câu 83 — Bẫy Hội Đồng: Lỗi Double Submit (Bấm Đặt Hàng Nhiều Lần Liên Tiếp)

CÂU HỎI

Hội đồng hỏi: "Khi mạng chập chờn, khách hàng sốt ruột bấm nút Đặt hàng 5 lần liên tiếp trong 2 giây. Hệ thống của em có tạo ra 5 đơn hàng trùng lặp và trừ tiền 5 lần không?"

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Hiện tượng trong hệ thống:** - Nếu Frontend không vô hiệu hóa (disable) nút bấm ngay sau click đầu tiên, trình duyệt sẽ bắn 5 HTTP request độc lập lên backend. - Vì mỗi lần gọi `new orderModel(orderData).save()` đều sinh ra một `_id` mới, backend sẽ tạo ra **5 đơn hàng giống hệt nhau** cho cùng một giỏ hàng. 2. **Giải pháp khắc phục 2 lớp:** - **Lớp 1 (Frontend):** Đặt State `isSubmitting` . Khi người dùng click, chuyển `isSubmitting = true` , vô hiệu hóa nút bấm (`disabled={isSubmitting}`) và hiển thị icon loading xoay tròn. - **Lớp 2 (Backend - Idempotency Key):** Áp dụng cơ chế Khóa tính lũy đẳng (Idempotency). Client sinh một chuỗi UUID ngẫu nhiên gắn vào Header `Idempotency-Key` của request đặt hàng. Backend kiểm tra trong Redis: Nếu key này đã được xử lý trong vòng 60 giây qua, backend lập tức bỏ qua các request trùng lặp và chỉ trả về kết quả của đơn hàng đầu tiên.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Frontend: `PlaceOrder.jsx`
 - Backend: `orderController.js`
-

Câu 84 — Bẫy Hội Đồng: Khách Chuyển Khoản Thiếu 1.000đ Hoặc Gõ Sai Cú Pháp VietQR

CÂU HỎI

Hội đồng hỏi: "Khách mua đơn hàng 550.000đ nhưng chỉ quét mã chuyển 549.000đ hoặc gõ sai cú pháp nội dung chuyển khoản thì hệ thống xử lý thế nào?"

CÂU TRẢ LỜI

Thưa thầy/cô: 1. Đây chính là lý do vì sao đề án của em lựa chọn **Cơ chế duyệt tay (Manual Approval)**: - Nếu sử dụng bot tự động 100%, bot chỉ so sánh chính xác tuyệt đối: Số tiền = 550.000đ VÀ cú pháp = `MT849201` . Khi khách chuyển thiếu 1.000đ hoặc sai 1 chữ cái, bot tự động hủy đơn, khiến tiền của khách bị treo và khách hàng bức xúc. 2. **Quy trình xử lý thực tế trong đề án:** - Khách bấm "Xác nhận đã chuyển", đơn hàng vào trạng thái **"Đã đặt hàng - Chờ duyệt"**. - Admin mở sao kê thấy giao dịch nhận 549.000đ từ số điện thoại của khách nhưng thiếu 1.000đ. - Admin sử dụng số điện thoại của khách trên đơn hàng gọi điện thông báo hỗ trợ, hoặc linh động bấm duyệt đơn nếu chênh lệch không đáng kể, hoặc hướng dẫn khách chuyển nốt phần tiền còn thiếu. - Nhờ có sự can thiệp của con người, các trường hợp sai sót ngoài ý muốn được xử lý cực kỳ khéo léo và không làm mất khách hàng.

ĐỘ KHÓ

Trung bình

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Nghiệp vụ vận hành cửa hàng & Cổng thanh toán VietQR

Câu 85 — Bắt Lỗi Code Thật: Lỗi Sử Dụng `req.body.userId` Trong Hàm `updateCart`

CÂU HỎI

Hội đồng hỏi: "Em hãy mở file `backend/controllers/cartController.js`, hàm `updateCart`. Tại dòng 38, em viết: `const { userId, itemId, size, quantity } = req.body;`. Tại sao các hàm khác lấy `userId` từ `req.userId` do middleware xác thực cấp, riêng hàm này lại lấy từ `req.body`? Hậu quả là gì?"

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Thằng thẩn nhìn nhận bug thực tế trong mã nguồn:** "Thưa thầy/cô, đây là một lỗi sơ suất (bug logic) trong quá trình em refactor code giỏ hàng. - Trong middleware `auth.js`, sau khi xác thực token xong, middleware đã gán `req.userId = user._id`. - Các hàm như `addToCart`, `getUserCart` đều dùng đúng `req.userId`. - Nhưng trong hàm `updateCart`, em lại vô tình bóc tách `const { userId, ... } = req.body;`. 2. **Hậu quả của bug này:** - Nếu Frontend chỉ gửi `{ itemId, size, quantity }` kèm Token qua Header mà không truyền `userId` trong Body, hàm này sẽ bị lỗi `undefined` và không thể cập nhật giỏ hàng. - Nguy hiểm hơn về mặt bảo mật: Một người dùng có thể truyền `userId` của người khác vào body để sửa đổi giỏ hàng của nạn nhân nếu không kiểm tra kỹ. 3. **Cách sửa ngay lập tức:** Sửa lại dòng đầu của hàm `updateCart`: javascript // Sửa từ: // const { userId, itemId, size, quantity } = req.body; // Thành: const { itemId, size, quantity } = req.body; const userId = req.userId; // Lấy an toàn từ JWT Middleware

ĐỘ KHÓ

Phản biện (Rất khó - Soi code thực tế)

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/cartController.js`
- Function: `updateCart` (Dòng 38)

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/cartController.js
const updateCart = async (req, res) => {
  try {
    const { userId, itemId, size, quantity } = req.body; // BUG: userId lấy từ body thay vì req.userId!
    const userData = await userModel.findById(userId);
    let cartData = await userData.cartData;
    cartData[itemId][size] = quantity;
    await userModel.findByIdAndUpdate(userId, { cartData });
    res.json({ success: true, message: "Cart Updated" });
  } catch (error) { ... }
};
```

Câu 86 — Bắt Lỗi Code Thật: Bug Tồn Kho Ma Trong Hàm `deleteOrder`

CÂU HỎI

Hội đồng hỏi: "Trong file `backend/controllers/orderController.js`, hàm `deleteOrder`, có đoạn code hoàn trả tồn kho nếu `currentOrder.status !== 'Đã đặt hàng'`. Nhưng lúc khách đặt hàng ở hàm `placeOrder`, hệ thống chưa hề trừ kho! Vậy khi xóa đơn hàng, tồn kho có bị tăng khổng (tồn kho ma) không?"

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Xác nhận phát hiện chính xác của Hội đồng:** "Thưa thầy/cô, đây là một lỗi bất nhất nghiệp vụ (Business Inconsistency) rất tinh tế trong codebase của em: - Khi tạo đơn hàng mới ở hàm `placeOrder`, em chưa thực hiện lệnh trừ trường `stockQuantities` của sản phẩm. - Nhưng khi Admin bấm xóa đơn hàng trong hàm `deleteOrder`, code lại có đoạn: `currentStock[item.size] += item.quantity` (cộng thêm hàng vào kho). - Hậu quả: Kho hàng chưa từng bị trừ nhưng lại được cộng thêm, dẫn đến số lượng hàng trên web bị **tăng khổng lồ so với thực tế trong kho vật lý**". 2. **Giải pháp đồng bộ hóa nghiệp vụ:** - Hoặc là: Phải trừ kho ngay khi đặt hàng thành công (`placeOrder`), khi đó lệnh hoàn kho trong `deleteOrder` mới có ý nghĩa. - Hoặc là: Xóa bỏ đoạn code cộng kho trong `deleteOrder` nếu việc quản lý kho được Admin kiểm đếm thủ công bên ngoài.

ĐỘ KHÓ

Phản biện (Cực khó)

FILE & HÀM LIÊN QUAN TRONG CODEBASE

- File: `backend/controllers/orderController.js`
- Function: `deleteOrder`

ĐOẠN CODE THỰC TẾ TỪ CODEBASE

```
// backend/controllers/orderController.js
// Hoàn trả số Lượng tồn kho nếu đơn hàng đã được xử lý (không phải "Đã đặt hàng")
if (currentOrder.status !== "Đã đặt hàng") {
  for (const item of currentOrder.items) {
    const product = await productModel.findById(item._id);
    if (product && product.stockQuantities && product.stockQuantities[item.size]) {
      const currentStock = product.stockQuantities[item.size];
      product.stockQuantities[item.size] = currentStock + item.quantity; // CỘNG KHỔNG KHO!
      await productModel.findByIdAndUpdate(item._id, { stockQuantities: product.stockQuantities });
    }
  }
}
```

Câu 87 — Tình Huống: Google Gemini API Bị Nghẽn Mạng Hoặc Hết Quota

CÂU HỎI

Khi tính năng so sánh sản phẩm gọi Google Gemini API mà gặp lỗi mạng, timeout hoặc vượt hạn mức (Rate Limit / Quota Exceeded), hệ thống có bị sập hoặc treo trang web không?

CÂU TRẢ LỜI

Thưa thầy/cô: - Hệ thống **hoàn toàn không bị sập** nhờ cơ chế Phòng thủ chủ động (Graceful Degradation): 1. Toàn bộ logic gọi Gemini API trong hàm `compareProducts` được bao bọc trong khối `try...catch`. 2. Nếu hàm `callGemini(prompt)` ném ra lỗi hoặc không trả về dữ liệu (do timeout, mất mạng hoặc hết quota), khối `catch` lập tức được kích hoạt. 3. Thay vì trả về mã lỗi 500 cho client, backend gọi hàm dự phòng `buildBasicComparison(products)`: - Tự động trích xuất các trường `price`, `brand`, `category`, `specifications` có sẵn trong MongoDB. - Tạo ra một bảng so sánh thông số kỹ thuật dạng bảng đối xứng và trả về cho người dùng kèm thông báo: "So sánh cơ bản (Hệ thống AI đang bảo trì)". 4. Trải nghiệm của người dùng vẫn diễn ra mượt mà, thông tin so sánh cốt lõi vẫn được phục vụ đầy đủ.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `backend/controllers/compareController.js` (`compareProducts` , `buildBasicComparison`)

Câu 88 — Phản Biện Kiến Trúc: Tại Sao Dùng Polling 3s Thay Vì WebSocket?

CÂU HỎI

Tại sao em lại chọn Polling 3s cho Live Chat thay vì WebSocket ngay từ đầu? Nếu Hội đồng yêu cầu chứng minh tính khả thi thì em bảo vệ quan điểm thế nào?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Lý do lựa chọn Polling cho giai đoạn đồ án:** - **Tính đơn giản & Độc lập:** Polling tận dụng trực tiếp hạ tầng HTTP RESTful API sẵn có, không yêu cầu thiết lập máy chủ WebSocket riêng biệt. - **Tương thích tuyệt đối với hạ tầng Cloud Serverless:** Khi triển khai trên các nền tảng đám mây miễn phí như Vercel hoặc Render Free Tier, các kết nối WebSocket liên tục (Long-lived TCP connection) thường bị hệ thống tự động ngắt kết nối sau 30-60 giây không hoạt động. HTTP Polling hoạt động bền bỉ trên 100% mọi môi trường mạng, không bao giờ bị lỗi ngắt kết nối đột ngột. 2. **Khả năng chịu tải thực tế:** - Với quy mô đồ án phục vụ vài chục người truy cập đồng thời, Polling 3s hoàn toàn đáp ứng tốt với thời gian phản hồi dưới 50ms trên MongoDB Atlas. - Em đã xây dựng sẵn kiến trúc Model tin nhắn chuẩn hóa (`sender` , `receiver` , `message` , `timestamp`), khi cần mở rộng quy mô, chỉ mất 1 ngày để tích hợp Socket.IO mà không cần thay đổi bất kỳ dòng cấu trúc CSDL nào.

ĐỘ KHÓ

Khá

Câu 89 — Tình Huống: Database Sập Giữa Lúc Khách Đang Thanh Toán Stripe

CÂU HỎI

Nếu khách đã quét thẻ thanh toán thành công 1.000.000đ trên cổng Stripe, nhưng khi Stripe chuyển hướng về `/verify` thì máy chủ cơ sở dữ liệu MongoDB bị sập kết nối, chuyện gì sẽ xảy ra và cách khắc phục là gì?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Hậu quả nếu chỉ dựa vào Redirect URL:** - Tiền của khách trong thẻ ngân hàng đã bị trừ 1.000.000đ. - Nhưng trang `/verify` gọi API `verifyStripe` gặp lỗi DB không thể cập nhật `payment = true`. - Khách nhìn thấy thông báo lỗi, vào danh sách đơn thấy đơn hàng vẫn chưa thanh toán hoặc bị hủy. Khách hàng sẽ khiếu nại. 2. **Giải pháp chuẩn doanh nghiệp (Stripe Webhook):** - Không được phụ thuộc duy nhất vào trình duyệt người dùng. - Thiết lập **Stripe Webhook Endpoint** (`/api/order/stripe-webhook`): Khi thanh toán thành công, máy chủ của Stripe sẽ tự động bắn một HTTP POST request ngầm tới server của ta. - Nếu tại thời điểm đó DB sập, server trả về mã lỗi HTTP 500. - Stripe có cơ chế **Exponential Backoff Retry**: Nó sẽ tự động thử gửi lại webhook sau 5 phút, 30 phút, 2 tiếng, cho đến khi database của ta phản hồi mã 200 thì thôi. Đơn hàng chắc chắn sẽ được cập nhật thành công mà không bao giờ bị mất giao dịch của khách.

ĐỘ KHÓ

Khó

Câu 90 — Phản Biện Database: Rủi Ro Khi Dùng `cartData: { type: Object }`

CÂU HỎI

Trong `userModel.js`, trường `cartData` được khai báo `type: Object, default: {}`. Việc dùng Schema lỏng lẻo như vậy có rủi ro gì về toàn vẹn dữ liệu so với việc định nghĩa Schema rõ ràng?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Ưu điểm:** Cực kỳ linh hoạt, cho phép lưu trữ ma trận kích thước sản phẩm tùy biến cho từng loại mặt hàng mà không bị gò bó. 2. **Rủi ro lớn về toàn vẹn dữ liệu:** - **Mongoose không thể kiểm tra tính hợp lệ (Validation):** Ai cũng có thể lưu bất kỳ dữ liệu rác nào vào trường này (ví dụ: `cartData: { "abc": "xyz", "soLuong": -999 }`) mà Mongoose không hề cảnh báo lỗi. - **Không theo dõi được sự thay đổi sâu (Change Tracking):** Trong Mongoose, khi thay đổi một thuộc tính lồng sâu bên trong Object, Mongoose đôi khi không tự động phát hiện Document bị dơ (Dirty), buộc lập trình viên phải gọi `userData.markModified('cartData')` thì mới lưu được. - **Dữ liệu mồ côi (Orphan Data):** Khi sản phẩm bị xóa khỏi shop, các mã `productId` cũ vẫn nằm mãi mãi trong `cartData` của người dùng mà không có cơ chế Cascade Delete.

ĐỘ KHÓ

Khá

PHẦN X: HƯỚNG PHÁT TRIỂN, SCALE & TỐI ƯU HỆ THỐNG (CÂU 91 — 100)

Câu 91 — Nâng Cấp Hệ Thống Live Chat Lên WebSocket / Socket.IO

CÂU HỎI

Làm thế nào để nâng cấp tính năng Live Chat từ cơ chế HTTP Polling 3s hiện tại lên công nghệ WebSocket / Socket.IO? Nêu kiến trúc triển khai cụ thể.

CÂU TRẢ LỜI

Thưa thầy/cô, để chuyển đổi sang kiến trúc thời gian thực hai chiều thực thụ (Bidirectional Full-Duplex): 1. **Kiến trúc phía Backend:**

- Tích hợp thư viện `socket.io` vào HTTP Server của Express: `javascript import http from 'http'; import { Server } from 'socket.io'; const server = http.createServer(app); const io = new Server(server, { cors: { origin: '*' } });`
- Quản lý phiên kết nối và phân phòng (Room Isolation): Mỗi cuộc hội thoại giữa khách hàng và shop được gán vào một room riêng: `socket.join(conversationId)`.
- Lắng nghe sự kiện gửi tin nhắn: `javascript socket.on('send_message', async (data) => { const savedMsg = await messageModel.create(data); io.to(data.conversationId).emit('receive_message', savedMsg); });`
2. **Kiến trúc phía Frontend:** - Sử dụng `socket.io-client`. Kết nối một lần duy nhất khi component `ChatWidget` mount. - Nhận tin nhắn tức thì bằng `socket.on('receive_message')`, triệt tiêu hoàn toàn độ trễ 3 giây và loại bỏ 100% các HTTP request rác.

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `server.js`, `chatController.js`
- Frontend: `ChatWidget.jsx`, `AdminChat.jsx`

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Khi máy chủ backend có nhiều instance (chạy Cluster hoặc Scale ngang trên nhiều server), làm thế nào để 2 người kết nối vào 2 server khác nhau vẫn chat được với nhau qua Socket.IO?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: Ta sử dụng **Socket.IO Redis Adapter (@socket.io/redis-adapter)**: - Redis đóng vai trò là một Message Broker trung gian (Pub/Sub). - Khi Server A nhận tin nhắn từ User 1, nó bắn sự kiện vào Redis Channel. - Redis phát tán tin nhắn này tới toàn bộ các Server khác (Server B, Server C). - Server B (nơi Admin đang kết nối) nhận được tin từ Redis và chuyển tiếp tới socket của Admin ngay tức khắc.

Câu 92 — Tối Ưu Hiệu Năng & Giảm Tải Database Bằng Redis Caching

CÂU HỎI

Chiến lược sử dụng Redis Caching để tăng tốc độ tải trang sản phẩm và giảm tải cho MongoDB Atlas được thiết kế như thế nào?

CÂU TRẢ LỜI

Thưa thầy/cô, hệ thống áp dụng mô hình bộ nhớ đệm **Cache-Aside Pattern (Lazy Loading)**: 1. **Quy trình đọc dữ liệu sản phẩm (GET /api/product/list)**: - Bước 1: Khi có request, backend kiểm tra trước trong Redis bằng khóa `cache:products:all`. - Bước 2 (Cache Hit): Nếu có dữ liệu trong Redis, trả về ngay cho client trong vòng dưới 2ms mà không chạm vào MongoDB. - Bước 3 (Cache Miss): Nếu chưa có trong Redis, backend truy vấn MongoDB Atlas, lấy danh sách sản phẩm, lưu kết quả vào Redis với thời gian sống (TTL = 1 giờ) bằng lệnh: `await redis.set('cache:products:all', JSON.stringify(products), 'EX', 3600)`; - Bước 4: Trả dữ liệu về cho client. 2. **Chiến lược làm sạch Cache (Cache Invalidation)**: - Khi Quản trị viên thêm mới, cập nhật giá hoặc xóa một sản phẩm, backend lập tức gọi `await redis.del('cache:products:all')`. - Lần gọi API tiếp theo sẽ tự động truy vấn dữ liệu mới nhất từ CSDL và nạp lại vào Redis, đảm bảo tính nhất quán dữ liệu (Data Consistency).

ĐỘ KHÓ

Khá

THÀNH PHẦN HỆ THỐNG LIÊN QUAN

- Backend: `productController.js`, Dịch vụ Redis

CÂU HỎI ĐÀO SÂU CỦA HỘI ĐỒNG

Hiện tượng Cache Stampede (hoặc Thundering Herd) là gì và làm thế nào để phòng tránh khi khóa sản phẩm hot trong Redis hết hạn?

CÂU TRẢ LỜI CHO CÂU HỎI ĐÀO SÂU

Thưa thầy/cô: - **Cache Stampede**: Xảy ra khi một khóa dữ liệu cực kỳ hot (ví dụ danh sách sản phẩm Flash Sale) hết hạn (TTL về 0), đúng lúc đó có 10.000 khách truy cập cùng 1 giây. Tất cả 10.000 request đều thấy cache rỗng và đồng loạt lao vào truy vấn MongoDB, khiến database bị quá tải và sập nguồn ngay lập tức. - **Giải pháp phòng tránh**: Sử dụng cơ chế **Khóa phân tán (Distributed Lock / Mutex with Redis Redlock)**: Chỉ request đầu tiên phát hiện cache miss mới được cấp khóa để đi truy vấn MongoDB nạp lại cache; 9.999 request còn lại phải chờ trong vài mili-giây để đọc lại từ cache vừa được nạp mới.

Câu 93 — Triển Khai MongoDB Multi-Document ACID Transactions

CÂU HỎI

Làm thế nào để áp dụng Giao dịch ACID (Transactions) trong MongoDB để đảm bảo an toàn tuyệt đối cho quy trình đặt hàng và trừ kho?

CÂU TRẢ LỜI

Thưa thầy/cô, MongoDB từ phiên bản 4.0 trở lên đã hỗ trợ đầy đủ ACID Transactions trên môi trường Replica Set:

```
// backend/controllers/orderController.js
import mongoose from 'mongoose';

const placeOrderWithTransaction = async (req, res) => {
  const session = await mongoose.startSession();
  session.startTransaction();
  try {
    const { items, address, amount } = req.body;

    // 1. Trừ kho từng sản phẩm trong phiên giao dịch
    for (const item of items) {
      const updated = await productModel.findOneAndUpdate(
        { _id: item._id, [`stockQuantities.${item.size}`]: { $gte: item.quantity } },
        { $inc: { [`stockQuantities.${item.size}`]: -item.quantity } },

```

```

    { session, new: true }
  );
  if (!updated) {
    throw new Error(`Sản phẩm ${item.name} kích cỡ ${item.size} không đủ hàng!`);
  }
}

// 2. Tạo đơn hàng mới trong phiên giao dịch
const newOrder = new orderModel({ userId: req.userId, items, address, amount, date: Date.now() });
await newOrder.save({ session });

// 3. Xóa giỏ hàng của người dùng trong phiên giao dịch
await userModel.findByIdAndUpdate(req.userId, { cartData: {} }, { session });

// Cam kết hoàn tất toàn bộ giao dịch
await session.commitTransaction();
session.endSession();
res.json({ success: true, message: "Đặt hàng thành công!" });
} catch (error) {
  // Nếu có bất kỳ lỗi nào, hoàn tác 100% dữ liệu
  await session.abortTransaction();
  session.endSession();
  res.json({ success: false, message: error.message });
}
};

```

Nhờ cơ chế này, kho hàng, đơn hàng và giỏ hàng luôn nhất quán 100%.

ĐỘ KHÓ

Khó

Câu 94 — Xử Lý Tác Vụ Nền Bất Đồng Bộ (Background Job / Queue) Với BullMQ

CÂU HỎI

Các tác vụ nặng như gửi email hóa đơn hoặc xuất báo cáo lớn làm chậm thời gian phản hồi của API. Hệ thống nên sử dụng công nghệ nào để đưa các tác vụ này chạy ngầm?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Vấn đề:** Việc kết nối máy chủ SMTP để gửi email xác nhận đơn hàng tốn từ 2-4 giây. Nếu đặt trong luồng request chính của API `placeOrder`, khách hàng phải chờ 4 giây mới thấy màn hình thành công. 2. **Giải pháp Hàng đợi tin nhắn (Message Queue) với BullMQ + Redis:** - Khi đơn hàng tạo xong, API `placeOrder` chỉ đẩy một gói tin nhỏ vào hàng đợi: `await orderQueue.add('sendOrderEmail', { orderId, email: user.email });` - API lập tức trả lời khách hàng trong 50ms: "Đặt hàng thành công!". - Phía sau hậu trường, các tiến trình công nhân (**Worker Process**) độc lập sẽ nhặt từng tác vụ từ hàng đợi của Redis ra để kết nối máy chủ SMTP và gửi email ngầm. - Nếu máy chủ email bị nghẽn mạng, BullMQ tự động thử lại (Retry) 3 lần sau các khoảng thời gian tăng dần, đảm bảo email không bao giờ bị thất lạc và API chính luôn phản hồi siêu tốc.

ĐỘ KHÓ

Khá

Câu 95 — Tự Động Hóa Đối Soát Thanh Toán VietQR Bằng Webhook SePay/Casso

CÂU HỎI

Để nâng cấp quy trình duyệt tay VietQR hiện tại lên mức tự động hóa 100% như các sàn TMĐT lớn (Shopee/Tiki), giải pháp kỹ thuật là gì?

CÂU TRẢ LỜI

Thưa thầy/cô, lộ trình tự động hóa đối soát ngân hàng thông qua cổng trung gian Webhook (như SePay hoặc Casso): 1. Cửa hàng đăng ký tài khoản ngân hàng liên kết dịch vụ thông báo biến động số dư. 2. Cấu hình Webhook URL trên hệ thống: `POST https://api.mywebsite.com/api/payment/sepay-webhook`. 3. **Quy trình xử lý Webhook tự động**: - Khách chuyển khoản đúng cú pháp: `MT849201 0987654321`. - Trong vòng 1-3 giây, ngân hàng bắn biến động số dư, SePay gửi HTTP POST request tới server kèm payload `{ gateway, transferType: "in", transferAmount: 550000, content: "MT849201 0987654321", referenceCode }`. - Backend xác thực API Key bảo mật trong Header của SePay. - Trích xuất mã đơn hàng `MT849201` từ chuỗi `content` bằng Regex. - Tìm kiếm đơn hàng trong MongoDB: Nếu số tiền `transferAmount >= order.amount`: + Cập nhật `order.payment = true`. + Cập nhật `order.status = "Đang đóng gói"`. + Bắn thông báo realtime qua Socket.IO tới màn hình khách hàng để tự động chuyển sang trang Hoàn tất đơn hàng.

ĐỘ KHÓ

Khá

Câu 96 — Chuyển Đổi Frontend Sang Server-Side Rendering (Next.js) Để Tối Ưu SEO

CÂU HỎI

Website hiện tại dùng Vite Single Page Application (SPA). Điểm yếu lớn nhất của kiến trúc này đối với một website thương mại điện tử là gì và giải pháp chuyển đổi sang Next.js SSR ra sao?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Điểm yếu cốt tử của SPA đối với SEO**: - Khi bot tìm kiếm của Google, Facebook hoặc Zalo cào dữ liệu một liên kết sản phẩm (ví dụ `/product/64f1...`), file HTML trả về ban đầu chỉ là một thẻ `<div id="root"></div>` rỗng kèm file JS. - Các crawler mạng xã hội không thực thi Javascript nên không thể đọc được tiêu đề sản phẩm, hình ảnh thu nhỏ (OpenGraph Meta Tags `og:image`, `og:title`) và giá tiền. Khi chia sẻ link lên Facebook, link sẽ không hiện ảnh sản phẩm đẹp mắt, làm giảm 80% lượt click mua hàng. 2. **Giải pháp chuyển đổi sang Next.js (App Router / Pages Router)**: - Áp dụng kỹ thuật **Server-Side Rendering (SSR)** hoặc **Incremental Static Regeneration (ISR)**: - Khi có request tới trang sản phẩm, máy chủ Node.js của Next.js sẽ truy vấn CSDL và sinh ra toàn bộ mã HTML chứa đầy đủ nội dung, ảnh và thẻ meta OpenGraph trước khi gửi về trình duyệt. - Google Bot lập tức lập chỉ mục (index) sản phẩm lên top kết quả tìm kiếm Google Shopping, khách chia sẻ link lên Zalo/Facebook hiển thị ảnh preview cực kỳ chuyên nghiệp.

ĐỘ KHÓ

Khá

Câu 97 — Chiến Lược Sao Lưu & Phục Hồi Thảm Họa Tự Động (Disaster Recovery Plan)

CÂU HỎI

Hãy trình bày một kịch bản dự phòng thảm họa (Disaster Recovery) chi tiết để đảm bảo website có thể hoạt động trở lại sau khi máy chủ gặp sự cố nghiêm trọng.

CÂU TRẢ LỜI

Thưa thầy/cô, kế hoạch ứng phó thảm họa tuân thủ hai chỉ số tiêu chuẩn công nghiệp: - **RPO (Recovery Point Objective)**: Tối đa 1 giờ (Mất tối đa 1 giờ dữ liệu trong trường hợp xấu nhất). - **RTO (Recovery Time Objective)**: Dưới 15 phút (Đưa

website hoạt động trở lại trong vòng 15 phút). 1. **Quy trình sao lưu tự động (Automated Backup Pipeline):** - Thiết lập Cronjob chạy lúc 02:00 sáng mỗi ngày trên máy chủ: Chạy lệnh `mongodump`, nén thư mục bằng thuật toán Gzip mã hóa mật khẩu AES-256. - Tự động tải tệp sao lưu lên dịch vụ lưu trữ đám mây độc lập **Amazon S3 Glacier** hoặc **Google Cloud Storage (GCS)** có chính sách lưu trữ 30 ngày. 2. **Kịch bản phục hồi khi máy chủ chính bị phá hủy:** - Kích hoạt máy chủ dự phòng đã được đóng gói sẵn Docker Image. - Tải bản backup gần nhất từ Amazon S3. - Chạy lệnh `mongorestore --gzip --archive=backup.tar.gz`. - Chuyển hướng DNS Cloudflare sang IP của máy chủ mới. Toàn bộ quy trình hoàn tất trong vòng 10-15 phút.

ĐỘ KHÓ

Khá

Câu 98 — Tích Hợp Tìm Kiếm Thông Minh Bằng Elasticsearch / Atlas Search

CÂU HỎI

Khi số lượng sản phẩm lên đến hàng triệu mặt hàng, tính năng tìm kiếm bằng toán tử Regex của MongoDB bộc lộ nhược điểm gì và giải pháp nâng cấp là gì?

CÂU TRẢ LỜI

Thưa thầy/cô: 1. **Nhược điểm của Regex trong MongoDB:** - Biểu thức chính quy `{ name: { $regex: key, $options: 'i' } }` buộc MongoDB phải quét toàn bộ collection (Full Collection Scan) nếu không có Text Index phù hợp. - Không hỗ trợ tìm kiếm mờ (Fuzzy Search - gõ sai chính tả ví dụ "iphoen" tìm ra "iphone"). - Không xử lý tốt tiếng Việt có dấu và không dấu (khách gõ "ao thun" không tìm ra "áo thun"). 2. **Giải pháp nâng cấp lên MongoDB Atlas Search (Lucene Engine):** - Tạo Search Index trên collection `products` với Analyzer tiếng Việt. - Hỗ trợ tính năng: + **Fuzzy Matching:** Cho phép sai lệch từ 1-2 ký tự bàn phím (`maxEdits: 2`). + **Autocomplete / Typeahead:** Hiển thị gợi ý sản phẩm ngay khi khách mới gõ 2 chữ cái đầu tiên. + **Đánh trọng số (Score Relevance):** Tên sản phẩm khớp từ khóa được nhân hệ số 5 điểm, mô tả khớp được 1 điểm, đưa các sản phẩm phù hợp nhất lên đầu danh sách.

ĐỘ KHÓ

Khá

Câu 99 — Lộ Trình Chuyển Đổi Sang Kiến Trúc Microservices & Container Hóa

CÂU HỎI

Nếu hệ thống phát triển thành một sản phẩm quy mô lớn với nhiều đội ngũ kỹ thuật cùng tham gia, làm thế nào để tách kiến trúc Monolith hiện tại thành Microservices?

CÂU TRẢ LỜI

Thưa thầy/cô, lộ trình phân rã dịch vụ theo phương pháp Domain-Driven Design (DDD): 1. **Phân rã thành 5 Microservices độc lập:** - **Auth Service:** Đăng ký, đăng nhập, cấp phát JWT, xác thực danh tính. - **Product & Catalog Service:** Quản lý danh mục, biến thể size/màu, thông số, so sánh AI. - **Cart & Order Service:** Xử lý giỏ hàng, quy tắc giảm giá coupon, máy trạng thái đơn hàng. - **Payment Service:** Tích hợp các cổng thanh toán Stripe, VietQR, SePay. - **Chat & Notification Service:** Xử lý tin nhắn WebSocket, gửi email thông báo. 2. **Container hóa với Docker & Orchestration với Kubernetes (K8s):** - Mỗi service có một Dockerfile riêng biệt và cơ sở dữ liệu độc lập (Database-per-Service). - Sử dụng **API Gateway (Nginx hoặc Kong Gateway)** làm đầu mối duy nhất tiếp nhận request từ Client, điều phối định tuyến và cân bằng tải (Load Balancing). - Nhờ đó, nếu tính năng Chat bị nghẽn mạng thì người dùng vẫn mua hàng và thanh toán bình thường mà không làm sập toàn bộ hệ thống.

ĐỘ KHÓ

Khó

Câu 100 — Thiết Lập Pipeline CI/CD Tự Động Với GitHub Actions

CÂU HỎI

Quy trình Tích hợp liên tục và Triển khai liên tục (CI/CD) tự động từ lúc lập trình viên commit code lên GitHub đến khi website được cập nhật trên máy chủ hoạt động như thế nào?

CÂU TRẢ LỜI

Thưa thầy/cô, quy trình CI/CD được tự động hóa hoàn toàn bằng file cấu hình `.github/workflows/deploy.yml` :

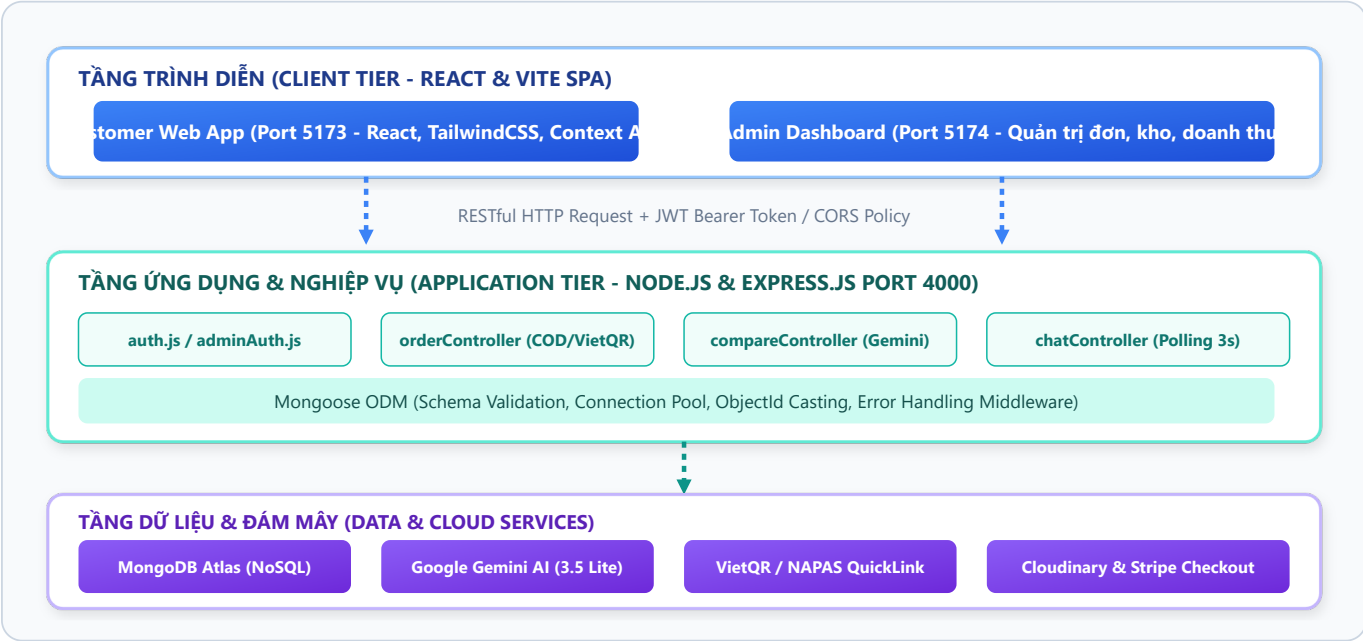
- Giai đoạn Tích hợp liên tục (Continuous Integration - CI):** - Khi lập trình viên tạo Pull Request hoặc Push code lên nhánh `main` : - GitHub Actions tự động khởi chạy môi trường Ubuntu ảo. - Tải source code (`actions/checkout@v3`), cài đặt Node.js. - Chạy lệnh `npm install` và `npm run lint` để kiểm tra các quy tắc chuẩn cú pháp code. - Chạy bộ kiểm thử tự động (Unit Tests & Integration Tests) bằng Jest. Nếu có bất kỳ bài test nào thất bại, quy trình dừng lại ngay lập tức và chặn không cho merge code lỗi.
- Giai đoạn Triển khai liên tục (Continuous Deployment - CD):** - Khi các bài test đều vượt qua thành công: - Kích hoạt SSH kết nối an toàn vào máy chủ VPS thông qua khóa `SSH_PRIVATE_KEY` lưu trong GitHub Secrets. - Thực hiện chuỗi lệnh trên VPS: `bash cd /var/www/Web-ECo git pull origin main cd backend && npm install && pm2 reload server cd ../frontend && npm install && npm run build cd ../admin && npm install && npm run build sudo systemctl reload nginx` - Website được cập nhật phiên bản mới nhất chỉ trong vòng 60 giây với thời gian gián đoạn bằng không (Zero-Downtime Deployment).

ĐỘ KHÓ

Khá

PHẦN BỔ SUNG: PHỤ LỤC TỔNG HỢP TOÀN DIỆN ĐỒ ÁN

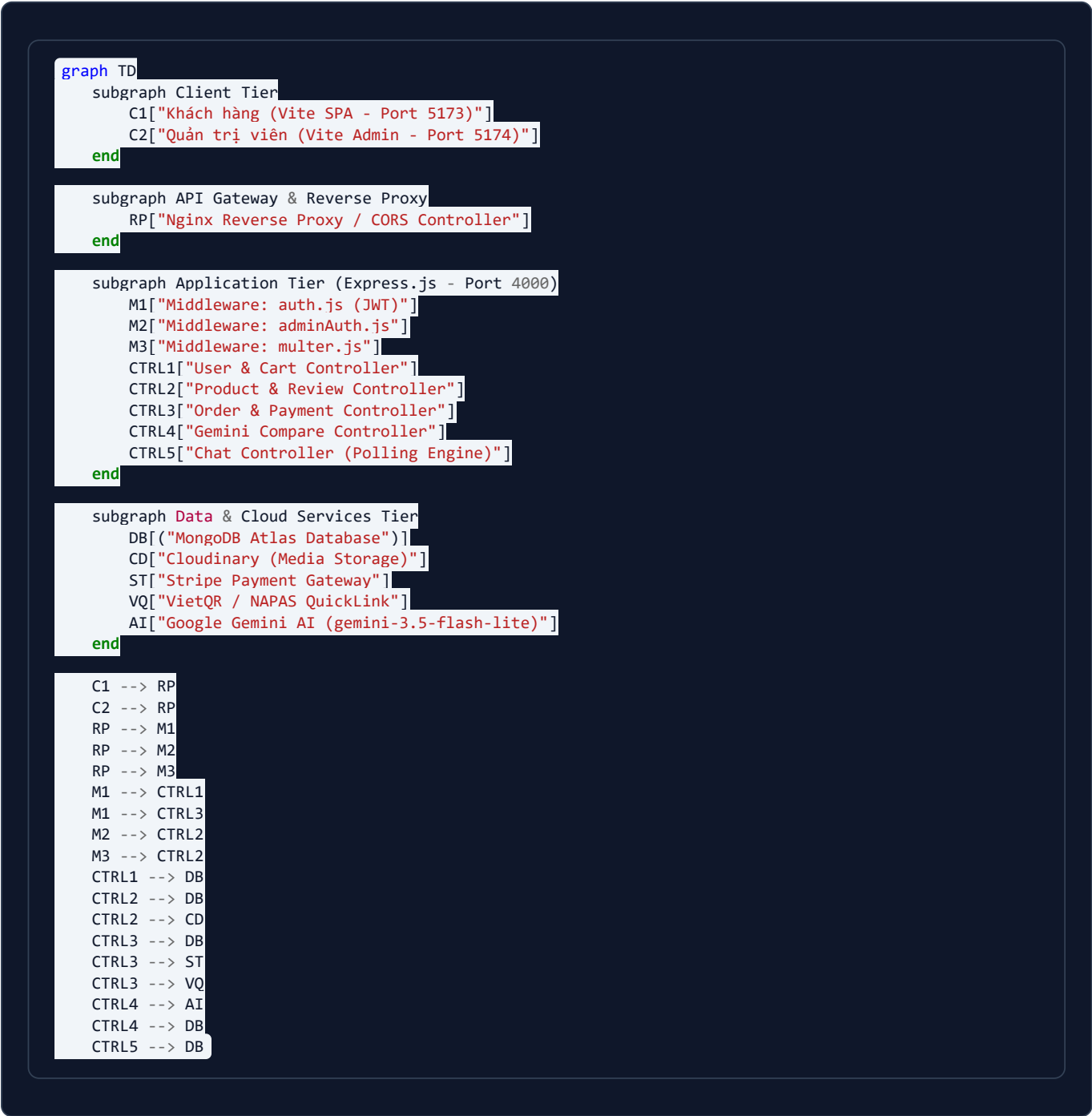
SƠ ĐỒ 1: KIẾN TRÚC TỔNG THỂ HỆ THỐNG 3 TẦNG (CLIENT - SERVER - CLOUD SERVICES)



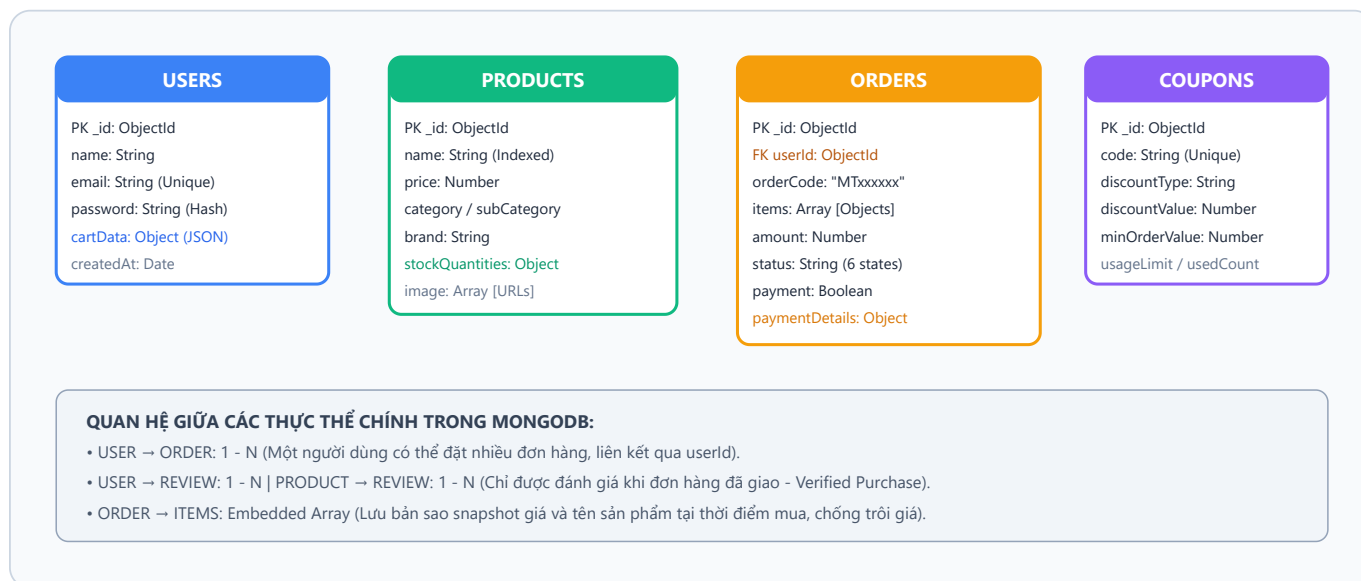
Hình 1: Kiến trúc tổng thể 3 tầng tích hợp Trí tuệ nhân tạo và Cổng thanh toán quốc gia

PHỤ LỤC A — KIẾN TRÚC HỆ THỐNG TỔNG THỂ (SYSTEM ARCHITECTURE)

Hệ thống được thiết kế theo kiến trúc 3 tầng chuẩn mực (Three-Tier Client-Server Architecture) kết hợp tích hợp các dịch vụ đám mây tiên tiến:



SƠ ĐỒ 5: THỰC THỂ CƠ SỞ DỮ LIỆU MONGODB (ERD & DATA MODELING)



Hình 5: Sơ đồ thực thể CSDL MongoDB và cấu trúc các Collection lõi

PHỤ LỤC B — SƠ ĐỒ THỰC THỂ CƠ SỞ DỮ LIỆU & DATA DICTIONARY

1. Sơ đồ Quan hệ Thực thể (Entity Relationship Diagram - ERD)

```
erDiagram
    USER ||--o{ ORDER : "đặt hàng (1:N)"
    USER ||--o{ REVIEW : "viết đánh giá (1:N)"
    USER ||--o{ MESSAGE : "gửi/nhận tin nhắn (1:N)"
    PRODUCT ||--o{ REVIEW : "được đánh giá (1:N)"
    ORDER ||--|{ ORDER_ITEM : "chứa các mặt hàng (1:N)"
    COUPON ||--o{ ORDER : "áp dụng giảm giá (1:N)"

    USER {
        ObjectID _id PK
        string name
        string email UK
        string password
        object cartData "Embedded JSON"
        date createdAt
    }

    PRODUCT {
        ObjectID _id PK
        string name
        string description
        number price
        array image "URLs Cloudinary"
        string category
        string subCategory
        string brand
        array sizes
        object stockQuantities "Key: Size, Val: Qty"
        boolean bestseller
        date date
    }

    ORDER {
        ObjectID _id PK
        ObjectID userId FK
        string orderCode "MTxxxxxx"
        array items "Danh sách sản phẩm mua"
        number amount "Tổng tiền đơn hàng"
        object address "Thông tin người nhận"
        string status "Đã đặt hàng | Đang giao | Đã giao | Đã hủy"
        string paymentMethod "COD | Stripe | VietQR"
        boolean payment "true: Đã trả | false: Chưa"
        object paymentDetails "customerConfirmed, note"
        date date
    }

    REVIEW {
        ObjectID _id PK
        ObjectID userId FK
        ObjectID productId FK
        ObjectID orderId FK
        number rating "1 - 5 sao"
        string comment
        array images
        date createdAt
    }

    COUPON {
        ObjectID _id PK
        string code UK
        string discountType "percentage | fixed"
        number discountValue
```

```
    number minOrderValue
    number maxDiscount
    number usageLimit
    number usedCount
    boolean isActive
    date startDate
    date endDate
}

MESSAGE {
    ObjectId _id PK
    ObjectId sender FK
    ObjectId receiver FK
    string message
    string senderRole "user | admin"
    boolean isRead
    date createdAt
}
```

PHỤ LỤC C — CÁC WORKFLOW NGHIỆP VỤ QUAN TRỌNG NHẤT

1. Workflow Đặt Hàng & Thanh Toán Quét Mã VietQR Duyệt Tay

[Khách hàng tại Checkout]

↓
Chọn VietQR -> Bấm 'ĐẶT HÀNG'
↓
Backend sinh mã đơn MTxxxxxx & URL ảnh QR VietQR cá nhân hóa
↓
Frontend chuyển hướng sang PaymentGateway.jsx
↓
Khách mở App ngân hàng quét mã -> Chuyển tiền đúng số tiền & cú pháp MTxxxxxx
↓
Khách bấm nút: "TÔI ĐÃ CHUYỂN TIỀN - XÁC NHẬN ĐƠN HÀNG (CHỜ DUYỆT)"
↓
Backend ghi nhận: status = "Đã đặt hàng", paymentDetails.customerConfirmed = true
↓
Frontend chuyển hướng sang trang /orders (Hiển thị trạng thái Chờ duyệt như COD)
↓
Admin kiểm tra biến động số dư ngân hàng -> Bấm "Duyệt thanh toán" trên Admin Dashboard
↓
Đơn hàng chuyển sang trạng thái "Đang đóng gói", xuất kho gửi cho khách

2. Workflow So Sánh Sản Phẩm Bằng Google Gemini AI

[Khách hàng chọn 2 sản phẩm] -> Bấm "SO SÁNH BẰNG AI"

↓
Frontend gửi POST /api/compare/compare { productIds }
↓
Backend truy vấn MongoDB lấy chi tiết thông số kỹ thuật (specifications, price, brand)
↓
Backend gửi cấu trúc Prompt chuẩn hóa sang Google Gemini 3.5 Flash Lite API
↓
Gemini AI phân tích sâu: Bảng so sánh, Ưu/Nhược điểm, Điểm vượt trội, Lời khuyên mua sắm
↓
(Nếu Gemini lỗi mạng -> Tự động Fallback sang Bảng so sánh thông số cơ bản)
↓
Frontend render giao diện so sánh trực quan, đẹp mắt bằng Markdown Parser

PHỤ LỤC D — BẢNG TRA CỨU FILE CODE & HÀM QUAN TRỌNG

STT	Chức năng nghiệp vụ	Đường dẫn File code	Tên Hàm / Component	Mô tả chức năng
1	Khởi tạo máy chủ & Route	backend/server.js	app.listen()	Kết nối CSDL, Cloudinary, nạp Middleware
2	Kết nối CSDL MongoDB	backend/config/mongodb.js	connectDB()	Kết nối MongoDB Atlas bằng Mongoose
3	Đăng ký & Mã hóa mật khẩu	backend/controllers/userController.js	registerUser	Kiểm tra email, băm bcrypt, tạo JWT
4	Đăng nhập khách hàng	backend/controllers/userController.js	loginUser	Đối soát bcrypt.compare, trả về token
5	Đăng nhập Quản trị viên	backend/controllers/userController.js	adminLogin	So khớp biến môi trường ADMIN_EMAIL
6	Middleware xác thực User	backend/middleware/auth.js	authUser	Giải mã JWT, trích xuất req.userId
7	Middleware xác thực Admin	backend/middleware/adminAuth.js	adminAuth	Xác minh token Admin bí mật
8	Thêm vào giỏ hàng	backend/controllers/cartController.js	addToCart	Cập nhật ma trận cartData[item][size]
9	Đồng bộ giỏ hàng	backend/controllers/cartController.js	getUserCart	Lấy dữ liệu giỏ hàng của user từ DB
10	Đặt hàng COD	backend/controllers/orderController.js	placeOrder	Khởi tạo đơn hàng, xóa giỏ hàng
11	Thanh toán Stripe	backend/controllers/orderController.js	placeOrderStripe	Tạo Checkout Session của Stripe
12	Xác thực thanh toán Stripe	backend/controllers/orderController.js	verifyStripe	Cập nhật payment: true sau redirect
13	Khởi tạo thanh toán VietQR	backend/controllers/paymentController.js	createVietQRPayment	Tạo orderCode MTxxxxxx và link VietQR

STT	Chức năng nghiệp vụ	Đường dẫn File code	Tên Hàm / Component	Mô tả chức năng
14	Khách xác nhận VietQR	backend/controllers/paymentController.js	confirmCustomerTransfer	Gán customerConfirmed: true, chờ duyệt
15	Hủy đơn hàng	backend/controllers/orderController.js	cancelOrder	Khách tự hủy khi còn 'Đã đặt hàng'
16	Đổi trạng thái đơn hàng	backend/controllers/orderController.js	updateStatus	Admin đổi status, tự động gán payment
17	So sánh sản phẩm Gemini AI	backend/controllers/compareController.js	compareProducts	Gọi Gemini 3.5 Flash Lite phân tích
18	Tìm sản phẩm ngoài AI	backend/controllers/compareController.js	searchExternalProduct	AI sinh dữ liệu sản phẩm ngoài thị trường
19	Kiểm tra mã giảm giá	backend/controllers/couponController.js	validateCoupon	Kiểm tra hạn dùng, minOrderValue, tính tiền
20	Đánh giá Verified Purchase	backend/controllers/reviewController.js	addReview	Kiểm tra đơn đã giao mới cho đánh giá
21	Quản lý tin nhắn Live Chat	backend/controllers/chatController.js	sendMessage , getMessages	Lưu và đọc tin nhắn chat hỗ trợ
22	Global State Management	frontend/src/context/ShopContext.jsx	ShopContextProvider	Quản lý giỏ hàng, token, tính tiền
23	Giao diện Cổng thanh toán	frontend/src/pages/PaymentGateway.jsx	PaymentGateway	Quét mã VietQR và bấm xác nhận chuyển
24	Xuất báo cáo Excel Admin	admin/src/pages/Orders.jsx	exportOrdersToExcel	Xuất danh sách đơn bằng thư viện xlsx

PHỤ LỤC E — CẨM NANG CHIẾN THUẬT VÀNG KHI BẢO VỆ ĐỒ ÁN

Dưới đây là 10 nguyên tắc ứng xử và tâm lý vàng giúp sinh viên ghi điểm tuyệt đối trước Hội đồng chấm thi:

1. **Nguyên tắc "Nguồn Sự Thật Duy Nhất (Single Source of Truth)":**
2. Luôn trả lời dựa trên những gì **thực tế có trong codebase**. Tuyệt đối không chém gió các tính năng cao siêu mà khi thầy cô yêu cầu mở code ra xem lại không có.
3. **Kỹ thuật xử lý khi bị Hội đồng bắt trúng Bug:**
4. Khi thầy cô chỉ ra lỗi hổng (ví dụ: Sửa giá qua DevTools ở Câu 81 hoặc Bug `req.body.userId` ở Câu 85):
5. **Bước 1:** Cảm ơn thầy cô: "Em cảm ơn thầy/cô đã chỉ ra một điểm rất chính xác và sâu sắc mà em sơ suất trong quá trình hoàn thiện đồ án".
6. **Bước 2:** Giải thích nguyên nhân xuất hiện bug trong code.
7. **Bước 3:** Trình bày ngay lập tức giải pháp khắc phục chuẩn mực. Hội đồng luôn đánh giá điểm A+ cho sinh viên nhận thức được bug và biết cách khắc phục hơn là sinh viên cãi cùn.
8. **Phong thái trình bày:**
9. Nói to, rõ ràng, dứt khoát, ánh mắt nhìn thẳng vào các thầy cô trong Hội đồng (Eye-contact).
10. Sử dụng thuật ngữ chuyên ngành chuẩn xác: *Stateless Authentication, ACID Transaction, Race Condition, Cache Invalidation, Single Page Application*.
11. **Chiến thuật Demo sản phẩm:**
12. Mở sẵn tất cả các tab cần thiết trên trình duyệt trước khi bước vào phòng thi: Tab Khách hàng, Tab Admin, Tab MongoDB Atlas Dashboard, Tab Code VS Code.
13. Demo theo kịch bản luồng nghiệp vụ hoàn chỉnh (User Journey): *Đăng ký -> Xem hàng -> Dùng AI so sánh -> Bỏ giỏ -> Áp Coupon -> Thanh toán VietQR -> Khách bấm đã chuyển -> Mở Admin kiểm tra -> Duyệt đơn*.
14. **Cách giải thích về các hạn chế công nghệ:**
15. Ví dụ khi hỏi về Polling chat hoặc lưu giỏ hàng trong User: Hãy trả lời rằng đây là giải pháp phù hợp nhất với **nguồn lực, thời gian và quy mô đồ án**, đồng thời chứng minh mình đã nắm vững lộ trình nâng cấp (Socket.IO, Redis) khi hệ thống scale lớn. Thầy cô sẽ thấy bạn có tư duy của một **Kỹ sư Phần mềm thực thụ (Software Engineer)** chứ không chỉ là một người thợ gõ code.

CHÚC BẠN TỰ TIN VÀ ĐẠT KẾT QUẢ XUẤT SẮC NHẤT TRONG KỲ BẢO VỆ ĐỒ ÁN TỐT NGHIỆP!